DEVOPS ENGINEERING MASTERY

ADVANCED DEVOPS ARCHITECTURE & ENTERPRISE PLATFORM ENGINEERING

Advanced Kubernetes, GitOps, Internal Developer Platforms, Platform Engineering, Multi-Cluster Operations & Production Engineering Capstone

VOLUME 16 • FINAL VOLUME • FIRST EDITION

```
nahid@technova:~$ kubectl get nodes
NAME              STATUS   ROLES         AGE   VERSION
prod-eu-01        Ready    control-plane  41d   v1.31.2
nahid@technova:~$ argocd app sync technova-platform
✓ Application technova-platform synced successfully
nahid@technova:~$ kubectl get pods -n technova-prod
```

Kubernetes • GitOps • Argo CD • Flux • Helm • Backstage • Terraform

First Edition • Published 9 November 2026 • Dhaka, Bangladesh

Companion Workbook • 24 Labs • 3 Projects • Final Capstone

Advanced DevOps Lab Workbook

DEVOPS ENGINEERING MASTERY — Volume 16 (Final Volume, Companion Workbook). First Edition, 9 November 2026. Advanced DevOps architecture and enterprise platform engineering, built around the TechNova MERN example application. This workbook collects all 24 guided labs, 15 assignments, the 3 major projects and the final series capstone, so you can work through them beside your terminal without carrying the whole book. Do the labs in order; each lab states its prerequisites. Local clusters (kind / Minikube / k3d) are used wherever practical so labs cost nothing; cloud labs always warn about charges and end with a cleanup step. Never run a lab step on production infrastructure. The solution guide for the capstone is at the end — attempt the capstone before reading it.

LAB WORKBOOK • VOLUME 16 (FINAL) — Advanced DevOps Architecture & Enterprise Platform Engineering

Lab Workbook — Advanced DevOps Architecture & Enterprise Platform Engineering

এই Workbook হলো **Volume 16**-এর হাতে-কলমে অংশ — এবং পুরো ১৬ খণ্ডের সিরিজের সমাপনী অনুশীলন। এখানে কোনো তত্ত্ব নতুন করে শেখানো হয়নি; বরং আগের ভলিউমগুলোতে শেখা Docker, Kubernetes, Terraform, Ansible, CI/CD, GitOps, Observability ও নিরাপত্তাকে একসাথে জুড়ে একটি **Enterprise Platform** বানানো হয়েছে। চলমান উদাহরণ **TechNova** — ডোমেইন `technova.example`, IP পরিসর `203.0.113.x`, MERN অ্যাপ্লিকেশন, তিনটি পরিবেশ (**dev / staging / prod**)।

প্রতিটি Lab-এ একই কাঠামো অনুসরণ করা হয়েছে: শেখার উদ্দেশ্য, পূর্বশর্ত, প্রয়োজনীয় অবকাঠামো, আনুমানিক সময়, খরচ-ঝুঁকি, আর্কিটেকচার ডায়াগ্রাম, ধাপে ধাপে নির্দেশনা, কমান্ড ও কনফিগারেশন, প্রত্যাশিত আউটপুট, যাচাই, সমস্যা সমাধান, নিরাপত্তা পর্যালোচনা, Cleanup, ডকুমেন্টেশন ডেলিভারেবল ও প্রতিফলনমূলক প্রশ্ন। Lab শেষ না করে পরের Lab-এ যাবেন না — প্রতিটি Lab আগেরটির উপর দাঁড়িয়ে আছে।

> **CONCEPT:** Enterprise Platform মানে অনেক টুলের স্তূপ নয়। এর মানে — ডেভেলপার নিজে সার্ভিস তৈরি করতে পারেন, প্ল্যাটফর্ম দল সেই পথটি নিরাপদ, পর্যবেক্ষণযোগ্য ও পুনরাবৃত্তিযোগ্য করে রাখে, এবং প্রোডাকশন ভাঙলে কে, কীভাবে, কতক্ষণে সারাবে — তা আগে থেকেই লেখা থাকে। এই Workbook-এর প্রতিটি Lab সেই সংজ্ঞার একটি করে টুকরো।

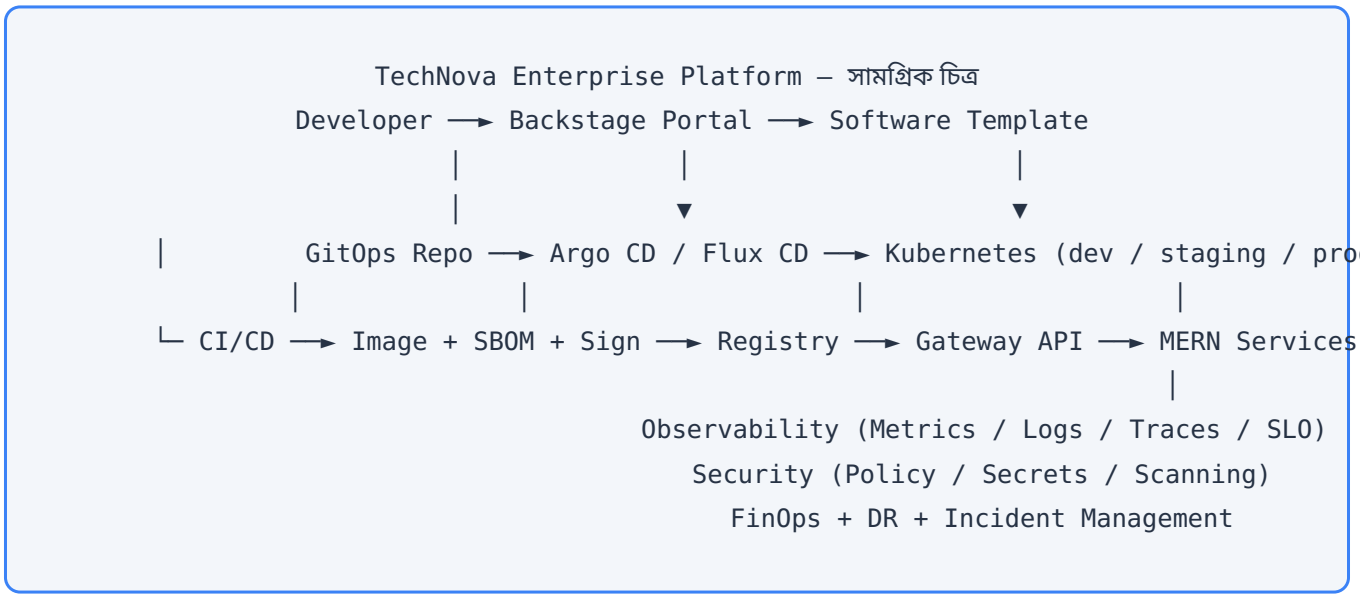
> **WARNING:** এই Workbook-এর বেশিরভাগ কাজ **লোকাল** পরিবেশে করার জন্য ডিজাইন করা হয়েছে — kind, Minikube বা k3d এবং Docker দিয়ে। কোনো ধাপে আসল ক্লাউড অ্যাকাউন্ট প্রয়োজন হলে সেখানে স্পষ্টভাবে **খরচ-সতর্কতা** লেখা আছে। খরচ-সতর্কতা ছাড়া কোনো ক্লাউড রিসোর্স তৈরি করবেন না। সব গোপন তথ্য `CHANGE-ME-...` প্লেসহোল্ডার দিয়ে লিখুন; আসল পাসওয়ার্ড, টোকেন বা কী কখনোই ফাইলে, Git-এ বা স্ক্রিনশটে রাখবেন না।

কীভাবে এই Workbook ব্যবহার করবেন

উপাদান	সংখ্যা	উদ্দেশ্য
Guided Lab	২৪টি (Lab 1–24)	ধাপে ধাপে এন্টারপ্রাইজ প্ল্যাটফর্ম তৈরি
Assignment	১৫টি (Assignment 1–15)	অধ্যায়ের জ্ঞান নিজে প্রয়োগ ও প্রমাণ করা
Professional Project	৩টি (Project 01–03)	পোর্টফোলিও-যোগ্য, সম্পূর্ণ প্ল্যাটফর্ম ডেলিভারি
Final Series Capstone	১টি	পুরো ১৬ খণ্ডের সমন্বিত ইঞ্জিনিয়ারিং প্রকল্প

সাধারণ পূর্বশর্ত ও মান

- ল্যাপটপে Ubuntu 24.04 LTS (বা সমতুল্য Linux), Docker, kubectl, kind অথবা Minikube, Git, Terraform, Ansible, Helm — ইনস্টল করা।
- GitHub অ্যাকাউন্ট ও একটি প্রাইভেট পরীক্ষামূলক রিপোজিটরি; সব কাজ ব্রাঞ্চ ও Pull Request দিয়ে।
- প্রতিটি Lab শেষে `docs/lab-NN-notes.md` -এ কী করলেন, কী ভাঙল, কী শিখলেন — লিখে রাখুন। এই নোটই আপনার ডকুমেন্টেশন ডেলিভারেবল।
- নামকরণের মান: ক্লাস্টার `technova-dev`, `technova-staging`, `technova-prod`; নেমস্পেস `technova-app`, `argocd`, `observability`, `platform`।



Lab ১ • Production Architecture Design

Lab 1 — Production Architecture Design and ADR

১.১ শেখার উদ্দেশ্য

এই Lab শেষে আপনি পারবেন — TechNova-র MERN অ্যাপ্লিকেশনের জন্য একটি বাস্তবসম্মত **Production Architecture** আঁকতে; **Architecture Decision Record (ADR)** লিখে একটি প্রযুক্তিগত সিদ্ধান্তের কারণ, বিকল্প ও পরিণতি নথিভুক্ত করতে; এবং Non-Functional Requirement (Availability, Scalability, Security, Cost) থেকে আর্কিটেকচারগত সিদ্ধান্তে পৌঁছাতে।

- প্রোডাকশন আর্কিটেকচারের মূল উপাদান (Client, Gateway, Service, Data, Observability) চিহ্নিত করা।
- ADR ফরম্যাটে অন্তত দুটি সিদ্ধান্ত লেখা — যেমন Kubernetes বনাম VM, এবং PostgreSQL-এর স্থাপনা।
- Single Point of Failure (SPOF) চিহ্নিত করে তা দূর করার পরিকল্পনা লেখা।

১.২ পূর্বশর্ত

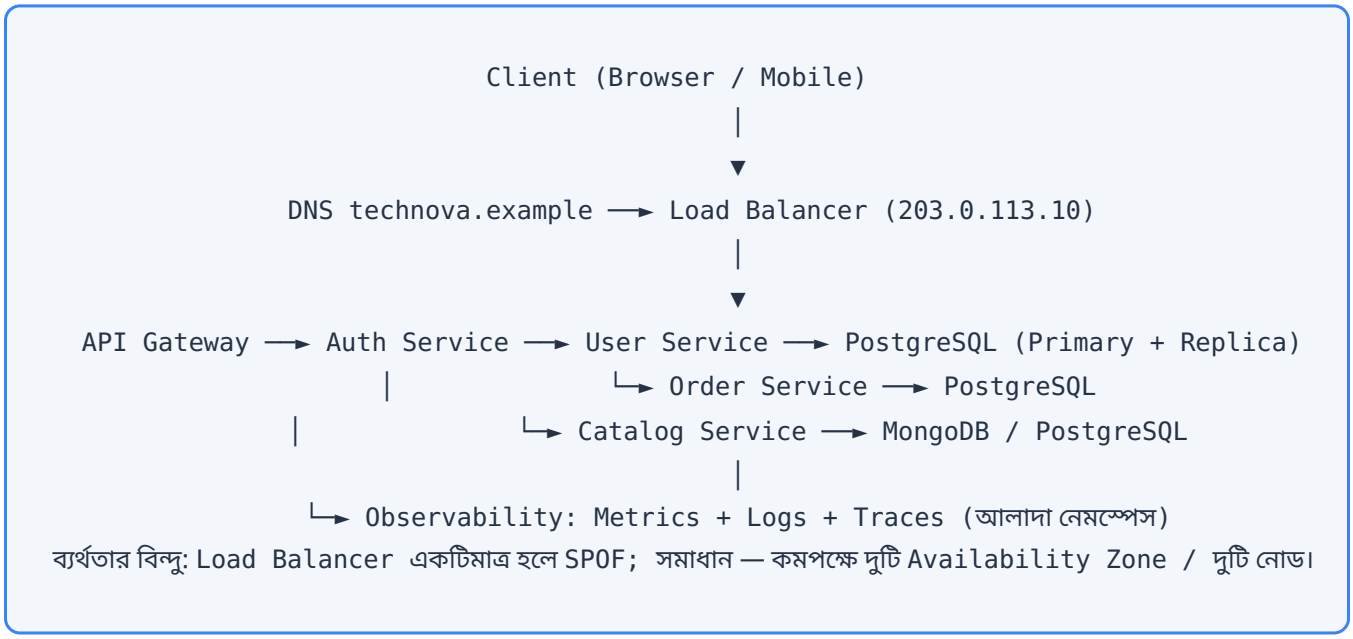
Volume 1–5-এর মৌলিক DevOps ধারণা, MERN স্ট্যাকের পরিচিতি, এবং Markdown লেখার অভ্যাস। কোনো ক্লাস্টার প্রয়োজন নেই — এটি একটি নকশা-ভিত্তিক Lab।

১.৩ প্রয়োজনীয় অবকাঠামো

শুধু একটি টেক্সট এডিটর ও Git রিপোজিটরি। আর্কিটেকচার ডায়াগ্রাম ASCII বা Mermaid দিয়ে আঁকুন; কোনো পেইড ডায়াগ্রাম টুল প্রয়োজন নেই।

আইটেম	বিবরণ	আনুমানিক সময়	খরচ-ঝুঁকি
সময়	২–৩ ঘণ্টা	—	Low
খরচ	শূন্য — লোকাল / কাগজ-ভিত্তিক কাজ	—	Low
ঝুঁকি	কোনো প্রোডাকশন পরিবর্তন নেই	—	Low

১.৪ আর্কিটেকচার ডায়াগ্রাম



১.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — প্রয়োজন লিখুন। TechNova-র জন্য পাঁচটি Non-Functional Requirement লিখুন: যেমন — দৈনিক ব্যবহারকারী, সর্বোচ্চ একসাথে অনুরোধ, অনুমোদিত Downtime, ডেটা হারানো চলবে কি না, বাজেটের সীমা। সংখ্যা অনুমান করলে “অনুমান” লিখে রাখুন; ভুয়া নিশ্চয়তা দেবেন না।

ধাপ ২ — উপাদান ভাগ করুন। MERN অ্যাপকে অন্তত তিনটি সার্ভিসে ভাগ করুন — Frontend, API, Database — এবং প্রতিটির দায়িত্ব এক লাইনে লিখুন। কোন সার্ভিস Stateless, কোনটি Stateful — তা চিহ্নিত করুন।

ধাপ ৩ — ADR লিখুন। নিচের ফরম্যাটে `docs/adr/ADR-001-kubernetes-platform.md` তৈরি করুন।

```
# ADR-001 — প্রোডাকশন প্ল্যাটফর্ম হিসেবে Kubernetes ব্যবহার
Status: Accepted | Date: 2026-11-09 | Author: CHANGE-ME-NAME

## Context
TechNova-র MERN অ্যাপ এখন একটি VM-এ চলে। ডেপ্লয় হাতে করা হয়,
স্কেল করতে হলে নতুন VM বানাতে হয়, ব্যর্থ হলে পুনরুদ্ধার ধীর।

## Decision
প্রোডাকশন প্ল্যাটফর্ম হিসেবে Kubernetes (লোকালে kind, প্রোডাকশনে
managed Kubernetes) ব্যবহার করা হবে। অ্যাপ Docker ইমেজ হিসেবে
প্যাকেজ হবে, ডেপ্লয়মেন্ট GitOps দিয়ে হবে।

## Alternatives Considered
- VM + Ansible শুধু: সহজ, কিন্তু স্কেল ও রোলব্যাক ধীর।
- Serverless শুধু: ছোট কাজে ভালো, কিন্তু Stateful অংশ ও খরচ
 অনুমান কঠিন।
```

```
## Consequences
ভালো: Declarative ডেপ্লয়, স্বয়ংক্রিয় পুনরুদ্ধার, পরিবেশের একরূপতা।
খারাপ: শেখার খরচ বেশি; ছোট দলে অপারেশনাল জটিলতা বাড়ে।
ঝুঁকি: ভুল কনফিগারেশনে পুরো ক্লাস্টার প্রভাবিত হতে পারে — তাই
Policy ও Staging পরীক্ষা বাধ্যতামূলক।
```

ধাপ ৪ — দ্বিতীয় ADR লিখুন। ডেটাবেস নিয়ে ADR-002 লিখুন: PostgreSQL কি ক্লাস্টারের ভেতরে StatefulSet হিসেবে চলবে, নাকি Managed Database সার্ভিসে থাকবে? শেখার পরিবেশে লোকাল StatefulSet, প্রোডাকশন সিদ্ধান্তে Managed সার্ভিসের পক্ষে যুক্তি দিন — এবং তার খরচ-ঝুঁকি লিখুন।

ধাপ ৫ — SPOF পর্যালোচনা করুন। আপনার ডায়াগ্রামের প্রতিটি উপাদানের পাশে লিখুন: এটি ভাঙলে কী হয়, কতক্ষণে সারবে, ডেটা হারাবে কি না। অন্তত দুটি SPOF দূর করার পরিকল্পনা লিখুন।

১.৬ প্রত্যাশিত আউটপুট

রিপোজিটরিতে `docs/architecture.md`, `docs/adr/ADR-001-*.md` ও `docs/adr/ADR-002-*.md` থাকবে; ডায়াগ্রামে Gateway, তিনটি সার্ভিস, ডেটাবেস ও Observability স্পষ্ট থাকবে।

১.৭ যাচাই

```
ls docs/adr/
cat docs/adr/ADR-001-kubernetes-platform.md
# প্রতিটি ADR-এ Context, Decision, Alternatives, Consequences — চারটি অংশই আছে কি না দেখুন
# একজন সহকর্মীকে ডায়াগ্রাম দেখান — তিনি কি এক মিনিটে প্রবাহ বুঝতে পারেন?
```

১.৮ সমস্যা সমাধান

সমস্যা	কারণ	করণীয়
আর্কিটেকচার খুব বড় হয়ে যাচ্ছে	শুরুতে সবকিছু ঢোকানো	প্রথম সংস্করণে শুধু মূল পথ (Happy Path) রাখুন; বাকিটা “ভবিষ্যৎ” হিসেবে চিহ্নিত করুন।
ADR-এ সিদ্ধান্ত অস্পষ্ট	কারণ লেখা হয়নি	“কেন নয়” অংশটি আবার লিখুন — বিকল্প বাতিলের কারণই সিদ্ধান্তের প্রমাণ।

১.৯ নিরাপত্তা পর্যালোচনা

আর্কিটেকচারে ডেটাবেস যেন সরাসরি ইন্টারনেটে না খোলে — শুধু সার্ভিসের ভেতর থেকে পৌঁছানো যায়। গোপন তথ্য ADR-এ লিখবেন না।

১.১০ Cleanup

এই Lab-এ মুছে ফেলার মতো কোনো রিসোর্স নেই; শুধু খসড়া ফাইল Git-এ Commit করুন।

১.১১ ডকুমেন্টেশন ডেলিভারেবল

- `docs/architecture.md` — পূর্ণ আর্কিটেকচার বর্ণনা ও ডায়াগ্রাম।
- `docs/adr/ADR-001` ও `ADR-002` — দুটি সিদ্ধান্ত-নথি।
- `docs/lab-01-notes.md` — SPOF তালিকা ও পরবর্তী প্রশ্ন।

১.১২ প্রতিফলনমূলক প্রশ্ন

1. আপনার আর্কিটেকচারে সবচেয়ে দুর্বল জায়গা কোনটি, এবং তা মেনে নেওয়ার যুক্তি কী?
2. কোন সিদ্ধান্তটি পরে বদলানো সবচেয়ে কঠিন হবে? সেটি আগে থেকে কীভাবে নরম করা যায়?
3. বাজেট অর্ধেক হলে কোন উপাদানটি প্রথমে বাদ দিতেন — আর কেন?

> **REAL WORLD:** প্রোডাকশনে সবচেয়ে দামি ভুলটি প্রায়শই প্রযুক্তির নয় — সিদ্ধান্তটি লেখা না থাকার। ছয় মাস পর কেউ জিজ্ঞেস করে “আমরা এটা কেন এভাবে করেছিলাম?” — ADR থাকলে উত্তরটি এক মিনিটে পাওয়া যায়, না থাকলে আবার তর্ক শুরু হয়।

Lab ২ • Distributed-Service Design

Lab 2 — MERN Application Distributed-Service Design

২.১ শেখার উদ্দেশ্য

এই Lab শেষে আপনি পারবেন — একটি Monolith MERN অ্যাপকে যুক্তিসঙ্গতভাবে কয়েকটি সার্ভিসে ভাগ করতে; প্রতিটি সার্ভিসের **API Contract**, ডেটা মালিকানা ও ব্যর্থতার সীমা (Failure Boundary) নির্ধারণ করতে; এবং কখন ভাগ করা উচিত নয় — তা ব্যাখ্যা করতে।

- Domain ভিত্তিতে সার্ভিস সীমা (Bounded Context) আঁকা।
- সার্ভিস-প্রতি API, ডেটাবেস ও নির্ভরতা তালিকাভুক্ত করা।
- Distributed Monolith-এর ফাঁদ চিনে এড়ানো।

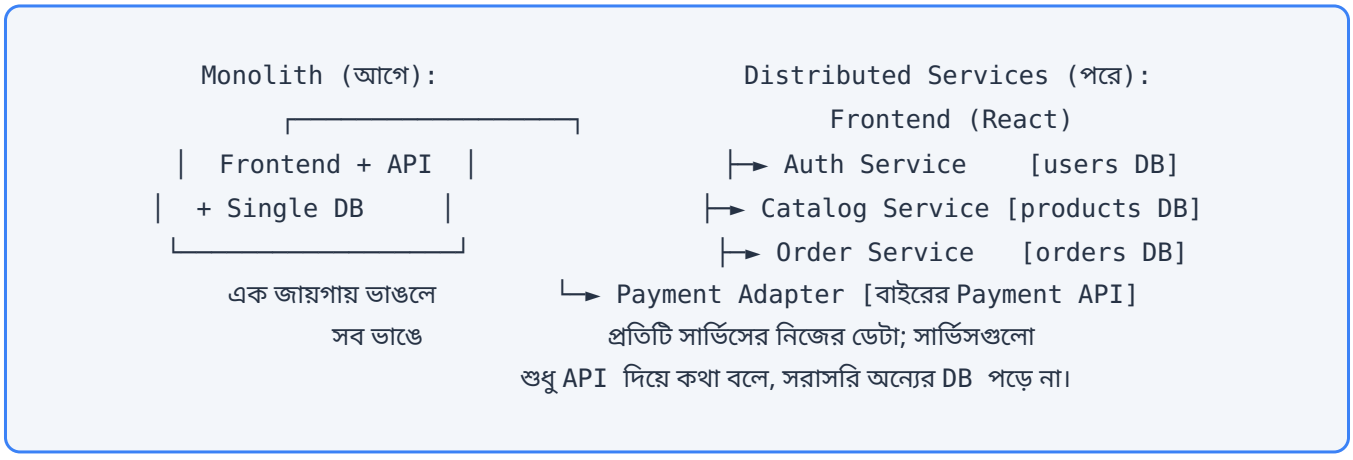
২.২ পূর্বশর্ত

Lab 1-এর আর্কিটেকচার ও ADR; MERN-এর Express API ও MongoDB / PostgreSQL-এর প্রাথমিক ধারণা।

২.৩ প্রয়োজনীয় অবকাঠামো

লোকাল এডিটর ও Git; কোনো চলমান সার্ভিস প্রয়োজন নেই। সময় ২–৩ ঘণ্টা; খরচ-ঝুঁকি **Low**।

২.৪ আর্কিটেকচার ডায়াগ্রাম



২.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — Domain তালিকা করুন। TechNova-র ব্যবসায়িক কাজগুলো লিখুন: ব্যবহারকারী নিবন্ধন, পণ্য তালিকা, অর্ডার, পেমেন্ট, নোটিফিকেশন। প্রতিটির পাশে লিখুন — এর ডেটা কে বদলায়, কে পড়ে।

ধাপ ২ — সার্ভিসে ভাগ করুন। অন্তত চারটি সার্ভিস নির্ধারণ করুন। প্রতিটির জন্য একটি টেবিল বানান — নাম, দায়িত্ব, API Endpoint, নিজের ডেটা, নির্ভরশীল সার্ভিস।

```
# services/catalog-service — উদাহরণ চুক্তি (Contract)
GET  /api/v1/products          # পণ্যের তালিকা
GET  /api/v1/products/:id      # একটি পণ্য
POST /api/v1/products          # নতুন পণ্য (শুধু admin)
# নিয়ম: Order Service পণ্যের দাম জানতে এই API ডাকে;
# সরাসরি Catalog-এর ডেটাবেস পড়ে না।
```

ধাপ ৩ — ব্যর্থতার সীমা আঁকুন। Catalog Service বন্ধ হলে কী হয়? Order তৈরি বন্ধ হবে, কিন্তু লগইন চলবে — এই ধরনের প্রভাব-তালিকা লিখুন। কোন সার্ভিসটি বন্ধ হলে পুরো অ্যাপ অচল হয়, তা চিহ্নিত করুন।

ধাপ ৪ — ভাগ না করার সিদ্ধান্ত লিখুন। ADR-003 লিখুন: প্রথম সংস্করণে কেন Modular Monolith থেকে শুরু করে ধীরে ধীরে আলাদা করা হবে — অথবা কেন সরাসরি সার্ভিসে ভাগ করা হবে। দুই পক্ষের যুক্তি লিখুন।

২.৬ প্রত্যাশিত আউটপুট

`docs/services.md`-এ চারটি সার্ভিসের চুক্তি, ডেটা মালিকানা ও ব্যর্থতার প্রভাব-তালিকা থাকবে।

২.৭ যাচাই

```
# প্রতিটি সার্ভিসের জন্য জিজ্ঞেস করুন:
# ১) এর নিজের ডেটাবেস আছে কি? ২) অন্য সার্ভিসের DB সরাসরি পড়ে কি?
# ৩) এটি বন্ধ হলে কোন ফিচার থামে, কোনটি চলে?
# কোনো সার্ভিসের উত্তর "হ্যাঁ" হলে নকশা আবার দেখুন।
```

২.৮ সমস্যা সমাধান

সমস্যা	কারণ	করণীয়
সার্ভিস অনেক বেশি হয়ে যাচ্ছে	প্রতিটি ফাংশনকে সার্ভিস ভাবা	Domain দিয়ে ভাগ করুন, ফাংশন দিয়ে নয়; শুরুতে ৩–৫টি সার্ভিসই যথেষ্ট।
সার্ভিসগুলো পরস্পরের DB পড়ছে	চুক্তি না থাকা	API Contract লিখুন; সরাসরি DB পড়া নিষিদ্ধ নিয়ম হিসেবে লিখে রাখুন।

২.৯ নিরাপত্তা পর্যালোচনা

সার্ভিস-টু-সার্ভিস কলে Authentication থাকবে — অভ্যন্তরীণ বলেই বিনা যাচাইয়ে বিশ্বাস করা যাবে না। সংবেদনশীল ডেটা (পাসওয়ার্ড হ্যাশ, পেমেন্ট তথ্য) শুধু মালিক সার্ভিসেই থাকবে।

২.১০ Cleanup

কোনো রিসোর্স নেই; নকশা ফাইল Commit করুন।

২.১১ ডকুমেন্টেশন ডেলিভারেবল

- `docs/services.md` — সার্ভিস তালিকা, API Contract, ডেটা মালিকানা।
- `docs/adr/ADR-003-service-boundaries.md`।
- `docs/lab-02-notes.md`।

২.১২ প্রতিফলনমূলক প্রশ্ন

1. কোন দুটি সার্ভিস আলাদা করলে সবচেয়ে বেশি জটিলতা বাড়বে — আর সেই জটিলতা কি মূল্যের যোগ্য?
2. একটি অনুরোধ (Request) কতগুলো সার্ভিস ঘুরে আসে? সেই সংখ্যা কমানো যায় কীভাবে?
3. আপনার দলে মাত্র দুজন ডেভেলপার থাকলে এই নকশা কি বদলাতেন? কীভাবে?

Lab ৩ • API Gateway & Traffic Routing

Lab 3 — API Gateway and Traffic Routing

৩.১ শেখার উদ্দেশ্য

এই Lab শেষে আপনি পারবেন — Kubernetes-এ **Ingress / Gateway API** দিয়ে বাইরের ট্রাফিক ভেতরের সার্ভিসে রুট করতে; হোস্ট ও পাথ-ভিত্তিক Routing, Rate Limit ও CORS নীতি প্রয়োগ করতে; এবং একটি সার্ভিস বন্ধ থাকলে Gateway কী আচরণ করে, তা পরীক্ষা করতে।

- Gateway API-র `Gateway` ও `HTTPRoute` রিসোর্স লেখা।
- `api.technova.example` ও `app.technova.example`-এ আলাদা রুট করা।
- ভুল রুটের সমস্যা নির্ণয় করা।

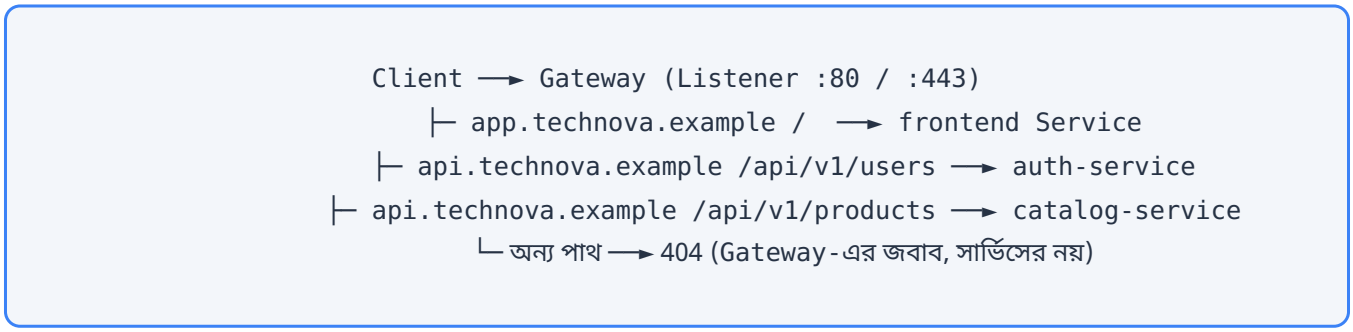
৩.২ পূর্বশর্ত

Lab 1–2, Docker ও kubectl-এর প্রাথমিক ব্যবহার, kind বা Minikube চালানোর অভিজ্ঞতা।

৩.৩ প্রয়োজনীয় অবকাঠামো

লোকাল kind ক্লাস্টার (একটি Control Plane, দুটি Worker — kind-এর ডিফল্টই যথেষ্ট), একটি ডেমো `http-echo` সার্ভিস। সময় ৩ ঘণ্টা; খরচ-ঝুঁকি **Low** (সম্পূর্ণ লোকাল)।

৩.৪ আর্কিটেকচার ডায়াগ্রাম



৩.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — ক্লাস্টার তৈরি করুন।

```
kind create cluster --name technova-dev
kubectl cluster-info
kubectl get nodes
```

ধাপ ২ — ডেমো সার্ভিস ডেপ্লয় করুন। দুটি ছোট Deployment ও Service বানান — `frontend` ও `catalog-service` — যারা আলাদা বার্তা দেয়, যাতে রুট বোঝা যায়।

ধাপ ৩ — Gateway ও HTTPRoute লিখুন।

```
# gateway.yaml — Gateway API (লোকালে kind-এর Gateway Controller প্রয়োজন;
# না থাকলে Ingress NGINX দিয়ে একই রুট Ingress রিসোর্সে লিখুন — ধারণা একই)
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: technova-gateway
  namespace: technova-app
spec:
  gatewayClassName: technova-gateway-class
  listeners:
    - name: http
      protocol: HTTP
      port: 80
---
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: technova-routes
  namespace: technova-app
spec:
  parentRefs:
    - name: technova-gateway
  hostnames:
    - "api.technova.example"
  rules:
    - matches:
        - path:
            type: PathPrefix
            value: /api/v1/products
      backendRefs:
        - name: catalog-service
          port: 80
```

```
kubectl apply -f gateway.yaml
kubectl get gateway -n technova-app
kubectl get httproute -n technova-app
```

ধাপ ৪ — লোকালে হোস্ট ম্যাপ করুন ও পরীক্ষা করুন। `/etc/hosts`-এ `127.0.0.1 api.technova.example` যোগ করে (শেখার মেশিনে) `curl` দিয়ে রুট পরীক্ষা করুন।

```
curl -H "Host: api.technova.example" http://127.0.0.1/api/v1/products
# প্রত্যাশা: catalog-service-এর জবাব; ভুল পাথে 404 আসবে
```

৩.৬ প্রত্যাশিত আউটপুট

সঠিক হোস্ট ও পাথে সঠিক সার্ভিসের জবাব; ভুল পাথে Gateway-এর 404; Gateway-এর Status-এ `Programmed: True`।

৩.৭ যাচাই

```
kubectl describe gateway technova-gateway -n technova-app
kubectl describe httproute technova-routes -n technova-app
kubectl logs -n technova-app -l app=catalog-service --tail=20
```

৩.৮ সমস্যা সমাধান

লক্ষণ	সম্ভাব্য কারণ	করণীয়
সব রুটে 404	Gateway Controller ইনস্টল নেই বা Class মেলেনি	Controller-এর Pod ও GatewayClass যাচাই করুন; Ingress দিয়ে বিকল্প পরীক্ষা করুন।
একটি রুট কাজ করে, অন্যটি নয়	Path মিলছে না / Service-এর Port ভুল	`kubectl get svc`-এ Port ও Selector মেলান।
curl-এ Connection Refused	Port-Forward করা হয়নি	লোকালে Gateway Service-এ Port-Forward করুন।

৩.৯ নিরাপত্তা পর্যালোচনা

Gateway-ই বাইরের দরজা — তাই TLS সেখানেই শেষ হবে (প্রোডাকশনে); সার্ভিসগুলো ClusterIP হিসেবে ভেতরে থাকবে, LoadBalancer নয়। Rate Limit ও বড় Request-এর সীমা Gateway-এ লিখুন।

৩.১০ Cleanup

```
kubectl delete -f gateway.yaml
kind delete cluster --name technova-dev
# /etc/hosts থেকে পরীক্ষামূলক লাইনটি মুছে দিন
```

৩.১১ ডকুমেন্টেশন ডেলিভারেবল

- `k8s/gateway/gateway.yaml` ও `httproute.yaml`।
- `docs/lab-03-notes.md` — রুট-তালিকা ও পরীক্ষার ফল।

৩.১২ প্রতিফলনমূলক প্রশ্ন

1. Ingress ও Gateway API-র মধ্যে কখন কোনটি বেছে নেবেন?
2. Gateway বন্ধ হলে কী হয় — সেই ঝুঁকি কীভাবে কমাবেন?
3. একটি নতুন সার্ভিস যোগ করতে কী কী বদলাতে হয়, আর কী বদলাতে হয় না?

TROUBLESHOOTING: লোকাল kind-এ আসল LoadBalancer IP পাওয়া যায় না — এটি স্বাভাবিক। শেখার জন্য Port-Forward বা `kubectl port-forward` ব্যবহার করুন; “External IP Pending” দেখে ঘাবড়াবেন না।

Lab 8 • Service-to-Service Communication

Lab 4 — Service-to-Service Communication

৪.১ শেখার উদ্দেশ্য

এই Lab শেষে আপনি পারবেন — সার্ভিসগুলোর মধ্যে **Synchronous (REST)** ও **Asynchronous (Message Queue / Event)** যোগাযোগের পার্থক্য ব্যাখ্যা ও প্রয়োগ করতে; Timeout, Retry ও Circuit Breaker-এর প্রয়োজন বুঝতে; এবং একটি সার্ভিস ধীর হলে পুরো চেইন যেন থেমে না যায় — সেই নকশা করতে।

- Order Service থেকে Catalog Service-এ REST কল করা।
- একটি Event (যেমন `order.created`) দিয়ে Notification আলাদা করা।
- Retry-এর বিপদ (Duplicate Order) চিনে Idempotency Key ব্যবহার করা।

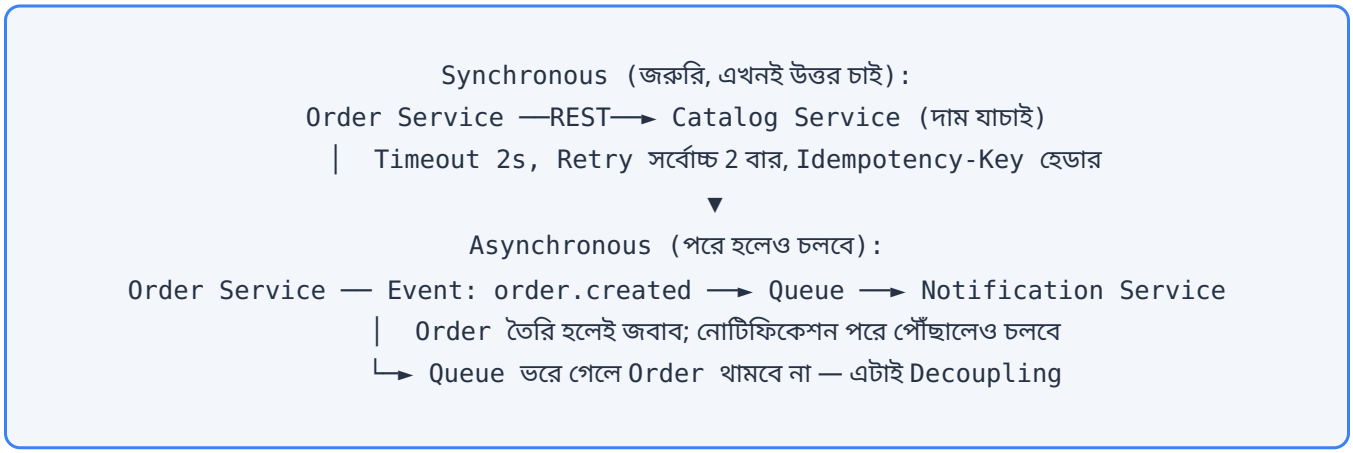
৪.২ পূর্বশর্ত

Lab 3-এর Gateway ও সার্ভিস ধারণা; Express-এ HTTP কল ও JSON-এর অভিজ্ঞতা।

৪.৩ প্রয়োজনীয় অবকাঠামো

লোকাল kind ক্লাস্টার; দুটি ডেমো সার্ভিস (Node.js Express) ও — সম্ভব হলে — লোকাল Redis বা NATS; না থাকলে In-Memory Event Log দিয়ে ধারণা দেখান। সময় ৩–৪ ঘণ্টা; খরচ-ঝুঁকি **Low**।

৪.৪ আর্কিটেকচার ডায়াগ্রাম



৪.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — Synchronous কল লিখুন। Order Service-এ Catalog-কে ডাকার কোডে Timeout বসান।

```
// order-service/src/catalog.js — সংক্ষেপ (ধারণা)
const response = await fetch("http://catalog-service/api/v1/products/" + productId, {
  signal: AbortSignal.timeout(2000), // 2 সেকেন্ডের বেশি অপেক্ষা নয়
  headers: { "Idempotency-Key": orderId, "X-Request-Id": requestId },
```

```
});
if (!response.ok) throw new Error("Catalog unavailable");
```

ধাপ ২ — Retry-এর নিয়ম লিখুন। শুধু অস্থায়ী ভুলে (যেমন 502/503, Timeout) Retry করুন; 400-ধরনের ভুলে নয়। Retry-এর মাঝে অপেক্ষা বাড়ান (Exponential Backoff) — ১ সেকেন্ড, তারপর ২ সেকেন্ড।

ধাপ ৩ — Event আলাদা করুন। Order তৈরি হওয়ার পর সরাসরি Notification Service-কে না ডেকে একটি Event প্রকাশ করুন; Notification Service সেই Event শুনে কাজ করবে। লোকালে এই Event-এর লগ ফাইলে লিখে দেখান।

```
# events/order.created.json — উদাহরণ Event
{
  "event": "order.created",
  "orderId": "ord-CHANGE-ME-001",
  "userId": "user-CHANGE-ME-001",
  "total": 1250,
  "occurredAt": "2026-11-09T00:00:00Z"
}
```

ধাপ ৪ — ব্যর্থতা পরীক্ষা করুন। Catalog Service-এর Pod মুছে দিন (`kubectl delete pod`) ও Order তৈরির চেষ্টা করুন — কী জবাব আসে, কতক্ষণে আসে, লগে কী লেখা হয় — নোট করুন।

৪.৬ প্রত্যাশিত আউটপুট

Catalog বন্ধ থাকলে Order Service দ্রুত, স্পষ্ট ভুল-জবাব দেয় (অনন্ত অপেক্ষা নয়); Order তৈরি হলেও Notification পরে পৌঁছায়; Duplicate Retry-তে একই Order দুবার তৈরি হয় না।

৪.৭ যাচাই

```
kubectl get pods -n technova-app
kubectl logs -n technova-app -l app=order-service --tail=30
# Catalog Pod মুছে আবার Order চেষ্টা করুন; Timeout 2 সেকেন্ডের মধ্যে জবাব আসে কি না দেখুন
```

৪.৮ সমস্যা সমাধান

লক্ষণ	সম্ভাব্য কারণ	করণীয়
অনুরোধ অনন্ত ঝুলে থাকে	Timeout বসানো হয়নি	সব আউটগোয়িং কলে Timeout বাধ্যতামূলক করুন।
একই অর্ডার দুবার তৈরি	Retry-তে Idempotency নেই	Idempotency-Key দিয়ে দ্বিতীয়বার একই Key এলে আগের ফল ফেরত দিন।
সার্ভিস DNS নামে পৌঁছায় না	Service নাম / নেমস্পেস ভুল	একই নেমস্পেসে ছোট নাম, ভিন্ন নেমস্পেসে `svc.namespace.svc.cluster.local` ব্যবহার করুন।

৪.৯ নিরাপত্তা পর্যালোচনা

অভ্যন্তরীণ কলেও Service Account ও — প্রোডাকশনে — mTLS বা টোকেন যাচাই থাকবে। লগে পূর্ণ Request Body বা টোকেন লিখবেন না; শুধু Request ID লিখুন।

৪.১০ Cleanup

```
kubectl delete namespace technova-app
kind delete cluster --name technova-dev
```

৪.১১ ডকুমেন্টেশন ডেলিভারেবল

- `docs/communication.md` — কোন যোগাযোগ Synchronous, কোনটি Asynchronous — তার যুক্তিসহ তালিকা।
- `docs/lab-04-notes.md` — ব্যর্থতা-পরীক্ষার ফল।

৪.১২ প্রতিফলনমূলক প্রশ্ন

1. পেমেন্টের মতো কাজে Synchronous না Asynchronous — কোনটি নিরাপদ, এবং কেন?
2. Retry কখন সাহায্য করে, আর কখন সিস্টেমকে আরও খারাপ করে?
3. একটি সার্ভিস ধীর হলে তার “ধীরতা” কীভাবে পুরো সিস্টেমে ছড়ায় — Cascade Failure কী?

Lab ৫ • Load Testing & Resilience

Lab 5 — Load Testing and Resilience

৫.১ শেখার উদ্দেশ্য

এই Lab শেষে আপনি পারবেন — **k6** বা সমতুল্য টুল দিয়ে লোকাল সার্ভিসে Load Test চালাতে; **Latency, Throughput, Error Rate** পড়ে বোতল-ঘাড় (Bottleneck) চিহ্নিত করতে; এবং **Horizontal Pod Autoscaler (HPA)** দিয়ে চাপ বাড়লে Pod বাড়ানো পরীক্ষা করতে।

- Baseline পরিমাপ নেওয়া — চাপ ছাড়া সার্ভিস কেমন চলে।
- ধাপে ধাপে চাপ বাড়িয়ে ভাঙার বিন্দু (Breaking Point) খোঁজা।
- HPA কনফিগার করে Autoscaling যাচাই করা।

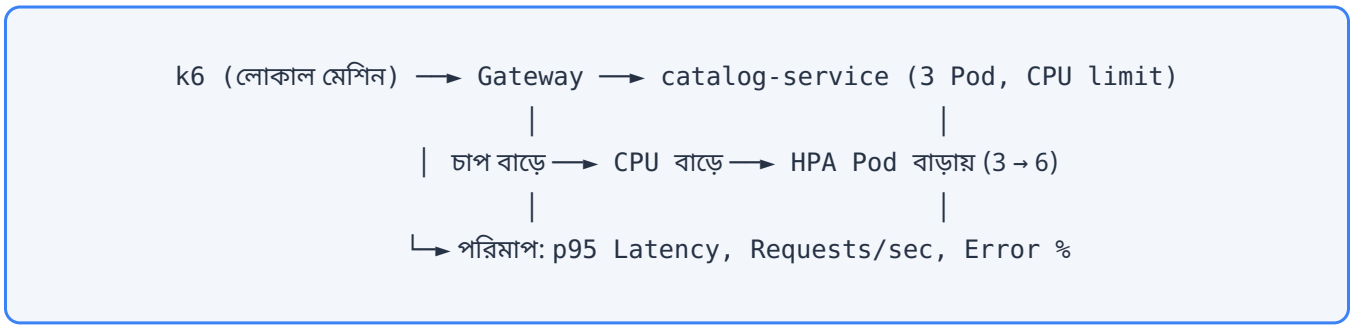
৫.২ পূর্বশর্ত

Lab 3–4, kubectl, একটি চলমান ডেমো API। প্রোডাকশন বা অন্যের সার্ভারে কখনো Load Test চালাবেন না — শুধু নিজের লোকাল পরিবেশে।

৫.৩ প্রয়োজনীয় অবকাঠামো

লোকাল kind ক্লাস্টার, k6 (লোকাল বাইনারি), একটি CPU-Limit দেওয়া Deployment। সময় ৩ ঘণ্টা; খরচ-ঝুঁকি **Low** — সম্পূর্ণ লোকাল; **সতর্কতা:** ক্লাউডে Load Test চালালে খরচ ও অ্যাকাউন্ট-সীমা হতে পারে, তাই এই Lab-এ ক্লাউডে করবেন না।

৫.৪ আর্কিটেকচার ডায়াগ্রাম



৫.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — Baseline নিন। প্রথমে অল্প চাপে পরীক্ষা চালান ও ফল সংরক্ষণ করুন।

```
# load-test.js — k6 স্ক্রিপ্ট (সংক্ষেপ)
import http from "k6/http";
import { check, sleep } from "k6";
export const options = { vus: 10, duration: "1m" };
export default function () {
```

```
  const res = http.get("http://127.0.0.1/api/v1/products");
  check(res, { "status is 200": (r) => r.status === 200 });
  sleep(1);
}

k6 run load-test.js
# ফল লিখে রাখুন: p95 latency, requests/sec, error %
```

ধাপ ২ — Resource Limit বসান ও HPA যোগ করুন।

```
# Deployment-এ resources অংশ
resources:
  requests:
    cpu: "100m"
    memory: "128Mi"
  limits:
    cpu: "500m"
    memory: "256Mi"
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: catalog-hpa
  namespace: technova-app
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: catalog-service
  minReplicas: 2
  maxReplicas: 6
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 60
```

ধাপ ৩ — চাপ বাড়ান ও পর্যবেক্ষণ করুন। Virtual User ১০ থেকে ৫০ করুন; `kubectl get hpa -w` ও `kubectl top pods` -এ Pod সংখ্যা বাড়ছে কি না দেখুন।

ধাপ ৪ — Chaos-ধরনের পরীক্ষা করুন। চলমান অবস্থায় একটি Pod মুছে দিন — অনুরোধ ব্যর্থ হয় কি না, কতক্ষণে নতুন Pod আসে — নোট করুন।

৫.৬ প্রত্যাশিত আউটপুট

Baseline-এর তুলনায় চাপ বাড়লে Latency বাড়ে; HPA নির্ধারিত সীমায় Pod বাড়ায়; একটি Pod মুছলেও সার্ভিস পুরোপুরি বন্ধ হয় না।

৫.৭ যাচাই

```
kubectl get hpa -n technova-app
kubectl get pods -n technova-app -w
kubectl top pods -n technova-app
# k6-এর শেষ লাইন: p(95) latency ও error % আগের রানের সাথে তুলনা করুন
```

৫.৮ সমস্যা সমাধান

লক্ষণ	সম্ভাব্য কারণ	করণীয়
HPA Pod বাড়ায় না	Metrics Server ইনস্টল নেই / Request বসানো হয়নি	kind-এ Metrics Server ইনস্টল করুন; Deployment-এ CPU Request আছে কি না দেখুন।
Latency হঠাৎ অনেক বাড়ে	CPU Limit-এ আটকে যাওয়া (Throttling)	Limit বাড়ানোর আগে Profile দেখুন; Limit-ই সমস্যা কি না নিশ্চিত হোন।
লোকাল মেশিন ধীর হয়ে যায়	k6 ও ক্লাস্টার একই মেশিনে	Virtual User কমান; বড় পরীক্ষা লোকালে নয়।

৫.৯ নিরাপত্তা পর্যালোচনা

Load Test শুধু নিজের পরিবেশে; পরীক্ষার ডেটায় আসল ব্যবহারকারীর তথ্য ব্যবহার করবেন না। Rate Limit থাকলে পরীক্ষার আগেই তা জানুন — নিজের Gateway-কে নিজেই আক্রমণ করবেন না।

৫.১০ Cleanup

```
kubectl delete hpa catalog-hpa -n technova-app
kubectl delete namespace technova-app
kind delete cluster --name technova-dev
```

৫.১১ ডকুমেন্টেশন ডেলিভারেবল

- `load-test.js` ও ফলাফলের টেবিল (Baseline বনাম চাপের রান)।
- `docs/lab-05-notes.md` — Bottleneck ও HPA-র আচরণ।

৫.১২ প্রতিফলনমূলক প্রশ্ন

1. p95 Latency গড় Latency-র চেয়ে কেন বেশি গুরুত্বপূর্ণ?
2. Autoscaling কখন সমস্যা লুকিয়ে ফেলে — যেমন ধীর ডেটাবেস কোয়েরি?
3. আপনার সার্ভিসের ভাঙার বিন্দু কোথায় — CPU, মেমোরি, নাকি ডেটাবেস সংযোগ?

> **WARNING:** Load Test-এর ফল লোকাল মেশিনের সীমা প্রতিফলিত করে, প্রোডাকশনের নয়। লোকাল ফল দিয়ে প্রোডাকশন ক্ষমতা ঘোষণা করবেন না — এটি শুধু আচরণ বোঝার অনুশীলন।

Lab ৬ • Advanced Kubernetes Cluster Setup

Lab 6 — Advanced Kubernetes Cluster Setup

৬.১ শেখার উদ্দেশ্য

এই Lab শেষে আপনি পারবেন — **kind**-এ একটি বহু-নোড ক্লাস্টার কনফিগারেশন ফাইল দিয়ে তৈরি করতে; **Node Label ও Taint** দিয়ে কাজ ভাগ করতে; **Resource Quota ও LimitRange** দিয়ে একটি নেমস্পেসের সীমা বাঁধতে; এবং **kubeconfig**-এর প্রসঙ্গ (Context) নিরাপদে পরিচালনা করতে।

- কনফিগারেশন ফাইল থেকে পুনরাবৃত্তিযোগ্য ক্লাস্টার তৈরি করা।
- Control Plane ও Worker নোডের ভূমিকা ব্যাখ্যা করা।
- একটি নেমস্পেসে Quota প্রয়োগ করে তা ভাঙার চেষ্টা করা।

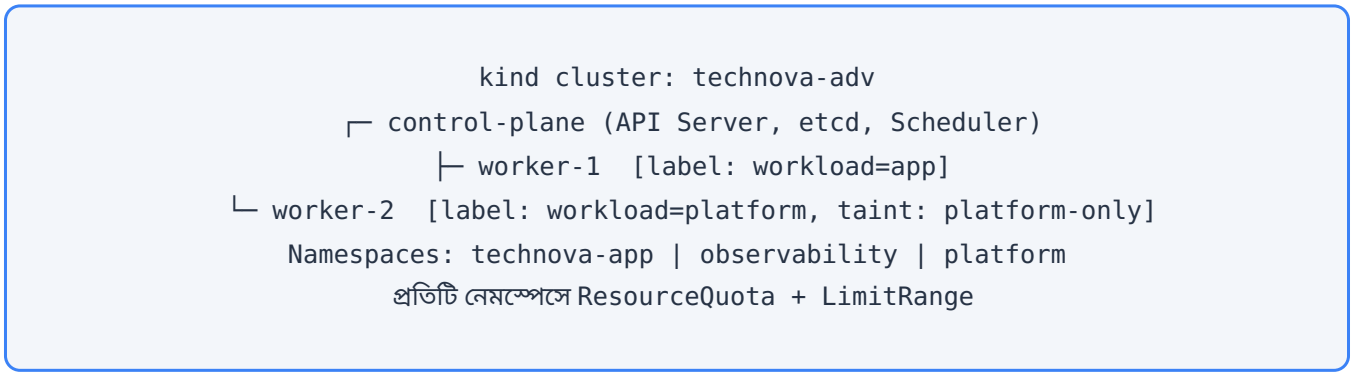
৬.২ পূর্বশর্ত

Lab 3–5, Docker, kubectl, YAML-এর অভিজ্ঞতা।

৬.৩ প্রয়োজনীয় অবকাঠামো

লোকাল kind; অন্তত ৪ GB ফাঁকা RAM। সময় ৩ ঘণ্টা; খরচ-ঝুঁকি **Low**।

৬.৪ আর্কিটেকচার ডায়াগ্রাম



৬.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — ক্লাস্টার কনফিগারেশন লিখুন।

```
# kind-config.yaml
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
  - role: control-plane
  - role: worker
    labels:
```

```
      workload: app
  - role: worker
    labels:
      workload: platform
```

```
kind create cluster --name technova-adv --config kind-config.yaml
kubectl get nodes --show-labels
```

ধাপ ২ — Taint ও Toleration পরীক্ষা করুন। একটি নোডে Taint দিন; Toleration ছাড়া Pod সেখানে যায় না — তা যাচাই করুন।

```
kubectl taint nodes technova-adv-worker2 workload=platform:NoSchedule
kubectl get nodes
# Toleration ছাড়া Pod Pending থাকে — এটি প্রত্যাশিত আচরণ
```

ধাপ ৩ — Quota বসান।

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: technova-app-quota
  namespace: technova-app
spec:
  hard:
    requests.cpu: "2"
    requests.memory: 2Gi
    pods: "10"
```

```
kubectl apply -f quota.yaml
kubectl describe quota -n technova-app
# সীমার বেশি Pod বানানোর চেষ্টা করুন — Kubernetes তা আটকাবে
```

ধাপ ৪ — kubeconfig যাচাই করুন। `kubectl config get-contexts` দিয়ে দেখুন কোন ক্লাস্টারে আছেন; ভুল Context-এ Apply করাই সবচেয়ে সাধারণ প্রোডাকশন দুর্ঘটনা।

৬.৬ প্রত্যাশিত আউটপুট

তিনটি নোড Ready; Label ও Taint অনুযায়ী Pod বণ্টন; Quota ছাড়িয়ে Pod তৈরি ব্যর্থ হয় — স্পষ্ট বার্তাসহ।

৬.৭ যাচাই

```
kubectl get nodes
kubectl describe node technova-adv-worker2 | grep -i taint
```

```
kubectl get resourcequota -n technova-app
kubectl config current-context
```

৬.৮ সমস্যা সমাধান

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Pod Pending	Taint / Resource অভাব / Node Selector ভুল	`kubectl describe pod` -এর Events পড়ুন।
kind ক্লাস্টার তৈরি হয় না	Docker চলছে না / Port দখলে	Docker চালু করুন; আগের ক্লাস্টার মুছে আবার চেষ্টা করুন।
ভুল ক্লাস্টারে Apply	Context বদলানো হয়নি	প্রতিটি Apply-এর আগে `kubectl config current-context` দেখুন।

৬.৯ নিরাপত্তা পর্যালোচনা

kubeconfig ফাইলে ক্লাস্টারের শংসাপত্র থাকে — এটি কখনো Git-এ তুলবেন না, অন্যের সাথে শেয়ার করবেন না। প্রোডাকশন kubeconfig আলাদা রাখুন; লোকাল ও প্রোডাকশন Context একসাথে মেলাবেন না।

৬.১০ Cleanup

```
kind delete cluster --name technova-adv
# Taint পরীক্ষার জন্য আলাদা করে কিছু মুছতে হয় না — ক্লাস্টার মুছলেই সব যায়
```

৬.১১ ডকুমেন্টেশন ডেলিভারেবল

- `kind-config.yaml`, `quota.yaml` ।
- `docs/lab-06-notes.md` — নোড পরিকল্পনা ও Quota সিদ্ধান্ত।

৬.১২ প্রতিফলনমূলক প্রশ্ন

1. Quota না থাকলে একটি নেমস্পেস কীভাবে পুরো ক্লাস্টারকে থামিয়ে দিতে পারে?
2. কোন কাজ Control Plane নোডে চলা উচিত নয় — এবং কেন?
3. লোকাল kind ও প্রোডাকশন Managed Kubernetes-এর সবচেয়ে বড় পার্থক্য কী?

Lab ৭ • Multi-Cluster Kubernetes

Lab 7 — Multi-Cluster Kubernetes Environment

৭.১ শেখার উদ্দেশ্য

এই Lab শেষে আপনি পারবেন — দুটি আলাদা kind ক্লাস্টার (**dev** ও **staging**) তৈরি ও পরিচালনা করতে; **Context** বদলে প্রতিটিতে একই অ্যাপের ভিন্ন সংস্করণ ডেপ্লয় করতে; এবং ক্লাস্টার-ভিত্তিক পার্থক্য (পরিবেশ চলক, Replica সংখ্যা) Kustomize Overlay দিয়ে প্রকাশ করতে।

- একাধিক kubeconfig Context নিরাপদে ব্যবহার করা।
- একই কোড, ভিন্ন কনফিগারেশন — এই নীতি প্রয়োগ করা।
- ক্লাস্টার-পার্থক্যের কারণে সৃষ্ট ভুল নির্ণয় করা।

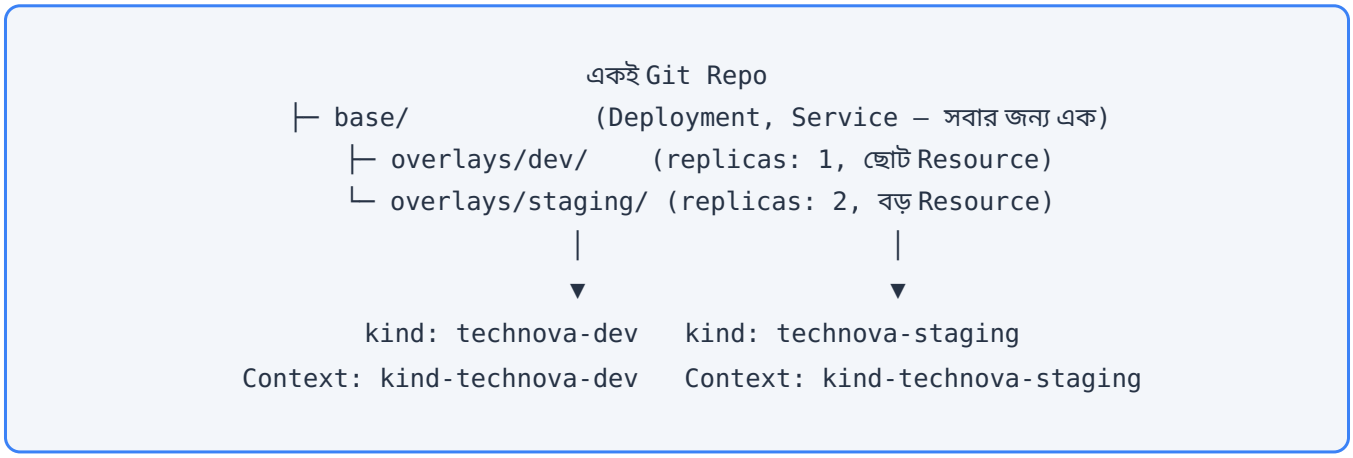
৭.২ পূর্বশর্ত

Lab 6, Kustomize-এর প্রাথমিক ধারণা।

৭.৩ প্রয়োজনীয় অবকাঠামো

লোকাল kind-এ দুটি ক্লাস্টার; অন্তত ৮ GB RAM সুপারিশ করা হয়, কম RAM-এ একটি করে ক্লাস্টার চালিয়ে ধাপগুলো করুন। সময় ৩–৪ ঘণ্টা; খরচ-ঝুঁকি **Low** (লোকাল)। **সতর্কতা:** আসল ক্লাউডে দুটি ক্লাস্টার মানে দ্বিগুণ খরচ — এই Lab-এ ক্লাউডে করবেন না।

৭.৪ আর্কিটেকচার ডায়াগ্রাম



৭.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — দুটি ক্লাস্টার তৈরি করুন।

```
kind create cluster --name technova-dev
kind create cluster --name technova-staging
```

```
kubectl config get-contexts
# দুটি Context-ই দেখা যাবে: kind-technova-dev ও kind-technova-staging
```

ধাপ ২ — Context বদলে ডেপ্লয় করুন।

```
kubectl config use-context kind-technova-dev
kubectl create namespace technova-app
kubectl apply -k overlays/dev/

kubectl config use-context kind-technova-staging
kubectl create namespace technova-app
kubectl apply -k overlays/staging/

# প্রতিটি ক্লাস্টারে আলাদা করে যাচাই করুন
kubectl config use-context kind-technova-dev
kubectl get pods -n technova-app
kubectl config use-context kind-technova-staging
kubectl get pods -n technova-app
```

ধাপ ৩ — পার্থক্য নথিভুক্ত করুন। কোন ক্লাস্টারে কত Replica, কোন Image Tag, কোন পরিবেশ চলক — একটি টেবিলে লিখুন। প্রোডাকশনে এই টেবিলই "কোন পরিবেশে কী চলে"-র সত্যের উৎস।

ধাপ ৪ — ভুল Context-এর অনুশীলন করুন। ইচ্ছা করে ভুল Context-এ একটি পরীক্ষামূলক ConfigMap বানান, তারপর খুঁজে বের করে মুছুন — এই অভ্যাসই প্রোডাকশন দুর্ঘটনা কমায়।

৭.৬ প্রত্যাশিত আউটপুট

dev-এ ১টি Replica, staging-এ ২টি Replica; একই Base থেকে দুটি ভিন্ন পরিবেশ; Context বদলালেই ভিন্ন ক্লাস্টারের তথ্য দেখায়।

৭.৭ যাচাই

```
kubectl config get-contexts
kubectl --context kind-technova-dev get pods -n technova-app
kubectl --context kind-technova-staging get pods -n technova-app
# --context ফ্ল্যাগ ব্যবহার করলে use-context বদলাতে হয় না — নিরাপদ অভ্যাস
```

৭.৮ সমস্যা সমাধান

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Pod এক ক্লাস্টারে, খুঁজছেন আরেকটিতে	Context ভুল	প্রতিটি কমান্ডে `--context` দিন অথবা `current-context` যাচাই করুন।

দ্বিতীয় ক্লাস্টার ধীর	RAM অভাব	একটি ক্লাস্টার বন্ধ করে ধাপগুলো করুন; দুটি একসাথে চালানো বাধ্যতামূলক নয়।

৭.৯ নিরাপত্তা পর্যালোচনা

dev-এর শংসাপত্র দিয়ে যেন staging-এ লেখা না যায় — প্রোডাকশনে প্রতিটি ক্লাস্টারের আলাদা শংসাপত্র ও RBAC থাকবে। এই Lab-এ Context-এর নাম প্রম্পটে দেখানোর অভ্যাস করুন।

৭.১০ Cleanup

```
kind delete cluster --name technova-dev
kind delete cluster --name technova-staging
kubectl config get-contexts
```

৭.১১ ডকুমেন্টেশন ডেলিভারেবল

- `k8s/base/` ও `k8s/overlays/dev|staging/`।
- `docs/lab-07-notes.md` — ক্লাস্টার-তুলনা টেবিল।

৭.১২ প্রতিফলনমূলক প্রশ্ন

1. একটি অ্যাপের জন্য কখন আলাদা ক্লাস্টার দরকার, আর কখন একই ক্লাস্টারে আলাদা নেমস্পেসই যথেষ্ট?
2. দুটি ক্লাস্টারের কনফিগারেশন আলাদা হয়ে গেলে (Drift) কীভাবে ধরবেন?
3. প্রোডাকশন ক্লাস্টারে ভুল করে dev-এর কনফিগারেশন গেলে কী হবে?

Lab ৮ • Gateway API & Advanced Networking

Lab 8 — Gateway API and Advanced Networking

৮.১ শেখার উদ্দেশ্য

এই Lab শেষে আপনি পারবেন — **Header-ভিত্তিক Routing**, **Traffic Weighting** (যেমন ৯০% পুরোনো সংস্করণে, ১০% নতুনে) ও **Timeout / Retry নীতি** Gateway স্তরে প্রয়োগ করতে; এবং **NetworkPolicy** দিয়ে কোন Pod কোন Pod-এর সাথে কথা বলতে পারবে — তা সীমিত করতে।

- Canary-ধরনের ট্রাফিক ভাগ করা।
- NetworkPolicy লিখে তা পরীক্ষা করা।
- নেটওয়ার্ক নীতির ভুলে সার্ভিস বন্ধ হওয়ার সমস্যা নির্ণয় করা।

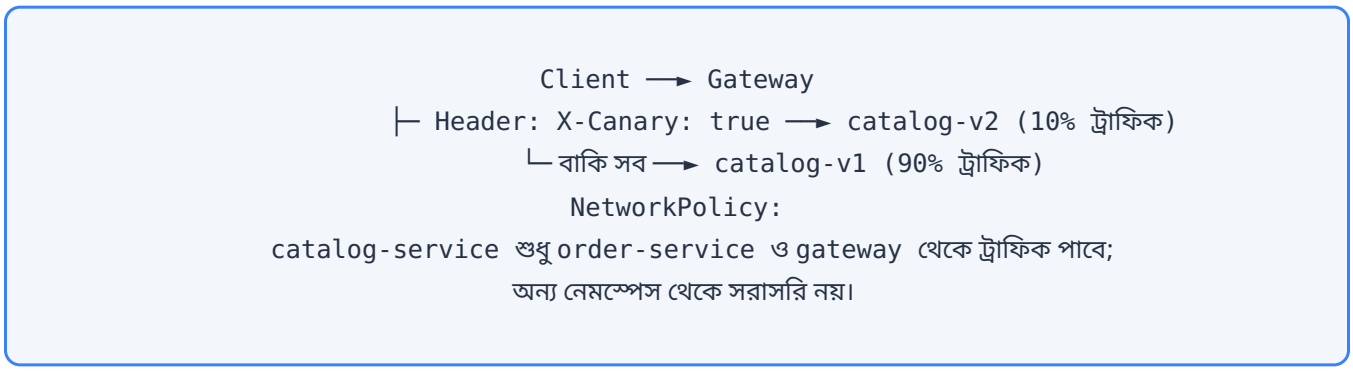
৮.২ পূর্বশর্ত

Lab 3, Lab 7; Gateway API ও NetworkPolicy-র ধারণা।

৮.৩ প্রয়োজনীয় অবকাঠামো

লোকাল kind ক্লাস্টার; Gateway Controller অথবা Ingress NGINX; দুটি সংস্করণের ডেমো সার্ভিস (v1 ও v2)। সময় ৩–৪ ঘণ্টা; খরচ-ঝুঁকি **Low**।

৮.৪ আর্কিটেকচার ডায়াগ্রাম



৮.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — দুটি সংস্করণ ডেপ্লয় করুন। একই Service-এর দুটি Deployment (v1 ও v2) বানান, যারা আলাদা বার্তা দেয়।

ধাপ ২ — Weight-ভিত্তিক রুট লিখুন।

```
# HTTPRoute-এ backendRefs-এ weight ব্যবহার
rules:
  - backendRefs:
      - name: catalog-v1
```

```
        port: 80
        weight: 90
      - name: catalog-v2
        port: 80
        weight: 10
```

ধাপ ৩ — Header-ভিত্তিক রুট যোগ করুন। `X-Canary: true` হেডার থাকলে সব ট্রাফিক v2-তে যাবে — curl দিয়ে পরীক্ষা করুন।

```
curl -H "Host: api.technova.example" -H "X-Canary: true" http://127.0.0.1/api/v1/produc
# প্রত্যাশা: v2-এর জবাব; হেডার ছাড়া বেশিরভাগ সময় v1-এর জবাব
```

ধাপ ৪ — NetworkPolicy লিখুন ও পরীক্ষা করুন।

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: catalog-allow-only-order
  namespace: technova-app
spec:
  podSelector:
    matchLabels:
      app: catalog-service
  policyTypes:
    - Ingress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: order-service
      ports:
        - port: 80
```

```
kubectl apply -f networkpolicy.yaml
# একটি অস্থায়ী Pod থেকে catalog-এ curl করুন — আটকানো উচিত;
# order-service Pod থেকে করলে পৌঁছানো উচিত
```

৮.৬ প্রত্যাশিত আউটপুট

Weight অনুযায়ী ট্রাফিক ভাগ হয়; Header দিলে v2-তে যায়; NetworkPolicy-র বাইরের Pod থেকে সংযোগ ব্যর্থ হয়।

৮.৭ যাচাই

```
kubectl get httproute -n technova-app
kubectl get networkpolicy -n technova-app
kubectl describe networkpolicy catalog-allow-only-order -n technova-app
```

৮.৮ সমস্যা সমাধান

লক্ষণ	সম্ভাব্য কারণ	করণীয়
সব ট্রাফিক v1-এ যায়	Controller Weight সমর্থন করে না / Route ক্যাশে	Controller-এর ডকুমেন্টেশন দেখুন; Header-ভিত্তিক রুট দিয়ে বিকল্প পরীক্ষা করুন।
NetworkPolicy-র পর সব বন্ধ	নীতি খুব কঠোর; DNS বা Gateway বাদ পড়েছে	নীতি মুছে আবার ধাপে ধাপে যোগ করুন; Gateway-এর Label যাচাই করুন।
kind-এ NetworkPolicy কাজ করে না	CNI NetworkPolicy সমর্থন করে না	এটি লোকাল সীমাবদ্ধতা — নীতি ফাইলটি যাচাই করে নথিতে লিখে রাখুন।

> **WARNING:** NetworkPolicy ভুল লিখলে প্রোডাকশন সার্ভিস হঠাৎ বিচ্ছিন্ন হয়ে যায় — এবং লগে কোনো ভুল দেখা যায় না, শুধু সংযোগ Timeout হয়। তাই NetworkPolicy সবসময় Staging-এ পরীক্ষা করে, ধাপে ধাপে, প্রোডাকশনে যাবে।

৮.৯ নিরাপত্তা পর্যালোচনা

Default-Deny নীতি দিয়ে শুরু করুন — সবকিছু বন্ধ, তারপর প্রয়োজনীয় পথ খুলুন। ডেটাবেস Pod-এর জন্য Ingress শুধু অ্যাপ সার্ভিস থেকে অনুমোদন করুন।

৮.১০ Cleanup

```
kubectl delete -f networkpolicy.yaml
kubectl delete namespace technova-app
kind delete cluster --name technova-dev
```

৮.১১ ডকুমেন্টেশন ডেলিভারেবল

- `k8s/networking/httproute-canary.yaml`, `networkpolicy.yaml`।
- `docs/lab-08-notes.md` — ট্রাফিক-ভাগের পরীক্ষার ফল।

৮.১২ প্রতিফলনমূলক প্রশ্ন

1. Gateway স্তরে Canary আর Deployment স্তরে Canary-র পার্থক্য কী?
2. NetworkPolicy না থাকলে একটি আক্রান্ত Pod কী কী করতে পারে?

3. Header-ভিত্তিক রুটিং পরীক্ষার জন্য ভালো, প্রোডাকশনের জন্য কেন ঝুঁকিপূর্ণ হতে পারে?

Lab ৯ • Kubernetes Policy & Security

Lab 9 — Kubernetes Policy and Security Enforcement

৯.১ শেখার উদ্দেশ্য

এই Lab শেষে আপনি পারবেন — **Pod Security Standards** (Restricted) প্রয়োগ করতে; **RBAC**-এ একটি সীমিত ServiceAccount বানাতে; এবং **Kyverno / OPA Gatekeeper-ধরনের Policy**-এর ধারণা দিয়ে "কোন ইমেজ চলবে, কোনটি চলবে না" — এই নিয়ম লিখে পরীক্ষা করতে।

- Privileged Pod আটকানো।
- শুধু অনুমোদিত Registry-র ইমেজ অনুমোদন করা।
- নীতি ভাঙার চেষ্টা করে তার বার্তা পড়া।

৯.২ পূর্বশর্ত

Lab 6–8, RBAC-এর প্রাথমিক ধারণা।

৯.৩ প্রয়োজনীয় অবকাঠামো

লোকাল kind ক্লাস্টার। Policy Engine (Kyverno) লোকালে ইনস্টল করা যায়; ভারী হলে শুধু Pod Security Admission ও RBAC দিয়ে Lab সম্পূর্ণ করুন — ধারণা একই। সময় ৩–৪ ঘণ্টা; খরচ-ঝুঁকি **Low**।

৯.৪ আর্কিটেকচার ডায়াগ্রাম



৯.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — নেমস্পেসে Pod Security Label দিন।

```
kubectl label namespace technova-app \
  pod-security.kubernetes.io/enforce=restricted \
  pod-security.kubernetes.io/audit=restricted \
```

```
  pod-security.kubernetes.io/warn=restricted
kubectl get namespace technova-app --show-labels
```

ধাপ ২ — Restricted Pod লিখুন।

```
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 10001
    seccompProfile:
      type: RuntimeDefault
  containers:
    - name: catalog-service
      image: technova/catalog-service:CHANGE-ME-TAG
      securityContext:
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
        capabilities:
          drop: ["ALL"]
```

ধাপ ৩ — নীতি ভাঙার চেষ্টা করুন। ইচ্ছা করে root হিসেবে চলা একটি Pod বানান — Kubernetes তা প্রত্যাখ্যান করবে। বার্তাটি নোট করুন।

ধাপ ৪ — সীমিত RBAC বানান। একটি ServiceAccount-কে শুধু Pod পড়ার অনুমতি দিন; সেই Account দিয়ে Secret পড়ার চেষ্টা করুন — ব্যর্থ হওয়া উচিত।

```
kubectl create serviceaccount reader -n technova-app
kubectl create role pod-reader -n technova-app \
  --verb=get,list --resource=pods
kubectl create rolebinding reader-binding -n technova-app \
  --role=pod-reader --serviceaccount=technova-app:reader
kubectl auth can-i get secrets --as=system:serviceaccount:technova-app:reader -n techno
# প্রত্যাশা: no
```

৯.৬ প্রত্যাশিত আউটপুট

Restricted Pod চলে; Privileged Pod প্রত্যাখ্যাত হয়; Reader Account Secret পড়তে পারে না।

৯.৭ যাচাই

```
kubectl get pods -n technova-app
kubectl auth can-i get pods --as=system:serviceaccount:technova-app:reader -n technova-
kubectl auth can-i get secrets --as=system:serviceaccount:technova-app:reader -n techno
```

৯.৮ সমস্যা সমাধান

লক্ষণ	সম্ভাব্য কারণ	করণীয়
সব Pod প্রত্যাখ্যাত হচ্ছে	পুরোনো ইমেজ root-এ চলে	ইমেজের Dockerfile-এ non-root User বসান; `warn` মোডে প্রথমে পরীক্ষা করুন।
Policy Engine ভারী লাগছে	লোকাল RAM কম	Kyverno বাদ দিয়ে Pod Security Admission দিয়েই Lab সম্পূর্ণ করুন।

৯.৯ নিরাপত্তা পর্যালোচনা

এই Lab-এর মূল শিক্ষা: নিরাপত্তা নীতি কাগজে নয়, Admission-এ প্রয়োগ করতে হয়। `latest` ট্যাগের ইমেজ, root User, Privileged Pod — তিনটিই প্রোডাকশনে নিষিদ্ধ তালিকায় রাখুন।

৯.১০ Cleanup

```
kubectl delete serviceaccount reader -n technova-app
kubectl delete namespace technova-app
kind delete cluster --name technova-dev
```

৯.১১ ডকুমেন্টেশন ডেলিভারেবল

- `k8s/security/pod-security.yaml`, `rbac-reader.yaml`।
- `docs/lab-09-notes.md` — প্রত্যাখ্যানের বার্তা ও নীতির তালিকা।

৯.১২ প্রতিফলনমূলক প্রশ্ন

1. নীতি ভাঙলে Pod তৈরি না হওয়া ভালো, নাকি তৈরি হয়ে সতর্কবার্তা দেওয়া ভালো — কখন কোনটি?
2. একটি জরুরি পরিস্থিতিতে নীতি বাইপাস করতে হলে কী প্রক্রিয়া থাকা উচিত?
3. আপনার নিজের অ্যাপের Dockerfile কি non-root User ব্যবহার করে?

Lab ১০ • Stateful Workloads & Storage

Lab 10 — Stateful Workloads and Persistent Storage

১০.১ শেখার উদ্দেশ্য

এই Lab শেষে আপনি পারবেন — **StatefulSet** দিয়ে PostgreSQL-ধরনের Stateful কাজ চালাতে; **PersistentVolumeClaim (PVC)** দিয়ে Pod মুছলেও ডেটা টিকে থাকে — তা প্রমাণ করতে; এবং **Backup ও Restore**-এর প্রাথমিক অনুশীলন করতে।

- Deployment বনাম StatefulSet-এর পার্থক্য ব্যাখ্যা করা।
- PVC, StorageClass ও Volume-এর সম্পর্ক বোঝা।
- Pod মুছে আবার বানিয়ে ডেটা ফিরে পাওয়া।

১০.২ পূর্বশর্ত

Lab 6, Stateful ধারণা, PostgreSQL-এর প্রাথমিক জ্ঞান।

১০.৩ প্রয়োজনীয় অবকাঠামো

লোকাল kind ক্লাস্টার (kind-এ ডিফল্ট StorageClass থাকে); PostgreSQL ইমেজ। সময় ৩ ঘণ্টা; খরচ-ঝুঁকি **Low**। **সতর্কতা:** প্রোডাকশনে ডেটাবেস সাধারণত Managed সার্ভিসে রাখা হয়; এই Lab শেখার জন্য ক্লাস্টারের ভেতরে ডেটাবেস চালায়।

১০.৪ আর্কিটেকচার ডায়াগ্রাম



১০.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — StatefulSet লিখুন।

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
  namespace: technova-app
spec:
```

```
  serviceName: postgres
  replicas: 1
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: postgres:16
          env:
            - name: POSTGRES_USER
              value: technova
            - name: POSTGRES_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: postgres-secret
                  key: password
            - name: POSTGRES_DB
              value: technova
          ports:
            - containerPort: 5432
          volumeMounts:
            - name: data
              mountPath: /var/lib/postgresql/data
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        accessModes: ["ReadWriteOnce"]
        resources:
          requests:
            storage: 1Gi

kubectl create secret generic postgres-secret -n technova-app \
  --from-literal=password=CHANGE-ME-DB-PASSWORD
kubectl apply -f postgres-statefulset.yaml
```

```
kubectl get pods -n technova-app -w
kubectl get pvc -n technova-app
```

ধাপ ২ — ডেটা লিখে Pod মুছুন। একটি টেবিল বানিয়ে একটি রো ঢোকান, তারপর Pod মুছে আবার পরীক্ষা করুন।

```
kubectl exec -n technova-app postgres-0 -- psql -U technova -d technova \
  -c "CREATE TABLE lab_check (id serial primary key, note text); INSERT INTO lab_check
kubectl delete pod -n technova-app postgres-0
kubectl exec -n technova-app postgres-0 -- psql -U technova -d technova \
  -c "SELECT * FROM lab_check;"
# প্রত্যাশা: রোটি এখনও আছে — PVC  ডেটা ধরে রেখেছে
```

ধাপ ৩ — Backup নিন।

```
kubectl exec -n technova-app postgres-0 -- pg_dump -U technova technova \
  > /tmp/technova-backup.sql
ls -lh /tmp/technova-backup.sql
# এই ফাইলটি নিরাপদ জায়গায় রাখুন; প্রোডাকশনে Backup  স্বয়ংক্রিয় ও পরীক্ষিত হতে হবে
```

১০.৬ প্রত্যাশিত আউটপুট

Pod পুনরায় তৈরি হওয়ার পরও ডেটা অক্ষত; PVC Status `Bound`; Backup ফাইলে টেবিলের সংজ্ঞা আছে।

১০.৭ যাচাই

```
kubectl get statefulset -n technova-app
kubectl get pvc -n technova-app
kubectl describe pvc data-postgres-0 -n technova-app
```

১০.৮ সমস্যা সমাধান

লক্ষণ	সম্ভাব্য কারণ	করণীয়
PVC Pending	StorageClass নেই / Storage শেষ	`kubectl get storageclass` দেখুন; kind-এ ডিফল্ট ক্লাস থাকে।
Pod CrashLoopBackOff	পাসওয়ার্ড Secret মেলেনি / ডেটা ডিরেক্টরি অনুমতি	Secret-এর নাম ও Key মেলান; লগ দেখুন।
ডেটা হারিয়ে গেল	PVC মুছে ফেলা হয়েছে	PVC মুছবেন না — Pod মুছুন; StatefulSet মুছলেও PVC থাকে।

১০.৯ নিরাপত্তা পর্যালোচনা

ডেটাবেসের পাসওয়ার্ড Secret-এ রাখুন, YAML-এ নয়। ডেটাবেস Service ClusterIP হবে — বাইরে থেকে পৌঁছানো যাবে না। প্রোডাকশনে ডেটাবেসের ডিস্ক এনক্রিপ্টেড থাকবে।

১০.১০ Cleanup

```
kubectl delete statefulset postgres -n technova-app
kubectl delete pvc data-postgres-0 -n technova-app
kubectl delete secret postgres-secret -n technova-app
# সতর্কতা: PVC মুছলে ডেটা চলে যাবে — Lab শেষেই মুছুন, আগে নয়
```

১০.১১ ডকুমেন্টেশন ডেলিভারেবল

- `k8s/stateful/postgres-statefulset.yaml`।
- `docs/lab-10-notes.md` — Backup ও Restore পরীক্ষার ফল।

১০.১২ প্রতিফলনমূলক প্রশ্ন

1. Pod মুছলে ডেটা টিকে থাকা ভালো, নাকি বিপজ্জনক — কখন কোনটি?
2. Backup নেওয়া থাকলেই কি যথেষ্ট? Restore পরীক্ষা না করলে কী ঝুঁকি থাকে?
3. একটি Replica দিয়ে কি High Availability হয়? কেন নয়?

> **CONCEPT:** Stateful কাজের মূল প্রশ্নটি Storage নয় — পরিচয় (Identity) ও ক্রম (Ordering)। StatefulSet প্রতিটি Pod-কে স্থির নাম দেয় (`postgres-0`), আর PVC সেই নামের সাথে ডেটা বেঁধে রাখে। Deployment-এ Pod-এর নাম বদলে যায় বলেই সেখানে Stateful ডেটা রাখা বিপজ্জনক।

Lab ১১ • GitOps Repository Architecture

Lab 11 — GitOps Repository Architecture

১১.১ শেখার উদ্দেশ্য

এই Lab শেষে আপনি পারবেন — **GitOps**-এর জন্য একটি পরিষ্কার রিপোজিটরি কাঠামো ডিজাইন করতে; **App-of-Apps** ধারণা ব্যাখ্যা করতে; এবং পরিবেশভেদে (dev/staging/prod) একই অ্যাপের ভিন্ন কনফিগারেশন Git-এ প্রকাশ করতে।

- Monorepo বনাম আলাদা Repo-র সিদ্ধান্ত নেওয়া।
- Base + Overlay কাঠামো লেখা।
- Git-কে Source of Truth হিসেবে প্রতিষ্ঠা করা।

১১.২ পূর্বশর্ত

Lab 7, Git Branch ও Pull Request, Kustomize-এর ধারণা।

১১.৩ প্রয়োজনীয় অবকাঠামো

GitHub-এ একটি পরীক্ষামূলক রিপোজিটরি; কোনো ক্লাস্টার প্রয়োজন নেই। সময় ২–৩ ঘণ্টা; খরচ-ঝুঁকি **Low**।

১১.৪ আর্কিটেকচার ডায়াগ্রাম

```
                    gitops-repo/
                       ├─ apps/
   │   ├─ catalog-service/base/            (Deployment, Service)
            │   ├─ catalog-service/overlays/dev/
          │   ├─ catalog-service/overlays/staging/
           │   └─ catalog-service/overlays/prod/
                       ├─ clusters/
    │   ├─ dev/       (App-of-Apps: কোন অ্যাপ, কোন সংস্করণ)
                     │   ├─ staging/
                      │   └─ prod/
     └─ platform/     (Argo CD, Observability, Policy)
   Git-এ Commit হলেই — পরের Lab-এ Argo CD তা ক্লাস্টারে মেলাবে।
```

১১.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — কাঠামো তৈরি করুন। উপরের ডায়াগ্রাম অনুযায়ী ডিরেক্টরি বানান; প্রতিটি Overlay-তে শুধু পার্থক্যটুকু রাখুন (Replica, Image Tag, Resource)।

ধাপ ২ — নিয়ম লিখুন। `README.md`-এ লিখুন: সরাসরি `main`-এ Commit নয়; প্রতিটি পরিবর্তন Pull Request দিয়ে; Image Tag কখনো `latest` নয়; Secret কখনো Git-এ নয়।

ধাপ ৩ — App-of-Apps ফাইল লিখুন।

```
# clusters/dev/apps.yaml — ধারণা
# প্রতিটি অ্যাপ একটি Argo CD Application হিসেবে তালিকাভুক্ত
# - catalog-service: overlays/dev
# - order-service:   overlays/dev
# - observability:   platform/observability
# এই একটি ফাইল বদলালেই পুরো ক্লাস্টারের অ্যাপ-তালিকা বদলায়।
```

ধাপ ৪ — Drift-এর গল্প লিখুন। কেউ যদি ক্লাস্টারে হাতে কিছু বদলায়, GitOps তা কীভাবে ধরবে — এক প্যারায় লিখুন। এই বোঝাপড়াই Lab 12-এর ভিত্তি।

১১.৬ প্রত্যাশিত আউটপুট

একটি পরিষ্কার GitOps Repo; Base ও Overlay আলাদা; README-তে নিয়ম লেখা; `main` ব্রাঞ্চে শুধু পর্যালোচিত Commit।

১১.৭ যাচাই

```
find gitops-repo -type f | sort
git log --oneline -10
# প্রতিটি Overlay-তে শুধু পার্থক্য আছে কি না দেখুন — Base-এর কপি থাকলে কাঠামো ভুল
```

১১.৮ সমস্যা সমাধান

সমস্যা	কারণ	করণীয়
Overlay-তে পুরো YAML কপি করা হয়েছে	Base-এর ধারণা পরিষ্কার নয়	Overlay-তে শুধু Patch রাখুন; Base-ই মূল ফাইল।
Secret ভুল করে Git-এ উঠেছে	প্লেসহোল্ডার ব্যবহার হয়নি	তৎক্ষণাৎ Secret বদলান, ইতিহাস থেকে মুছুন, `.gitignore` ও Pre-Commit Hook বসান।

১১.৯ নিরাপত্তা পর্যালোচনা

GitOps Repo-তে কখনো আসল Secret, kubeconfig বা ক্লাউড কী রাখবেন না। Sealed Secret বা External Secrets-এর মতো পদ্ধতি পরের Lab-গুলোতে আসবে; এই Lab-এ শুধু প্লেসহোল্ডার।

১১.১০ Cleanup

কোনো ক্লাস্টার রিসোর্স নেই; Repo-টি সংরক্ষণ করুন — এটি Lab 12–14-এ ব্যবহৃত হবে।

১১.১১ ডকুমেন্টেশন ডেলিভারেবল

- GitOps Repo-র কাঠামো ও `README.md`।
- `docs/lab-11-notes.md` — Monorepo বনাম আলাদা Repo সিদ্ধান্ত ও ADR।

১১.১২ প্রতিফলনমূলক প্রশ্ন

1. Git-ই যদি Source of Truth হয়, তাহলে ক্লাস্টারে হাতে বদলানো কেন বিপজ্জনক?
2. একটি ছোট দলে Monorepo কখন সুবিধাজনক, কখন বোঝা?
3. Image Tag আপডেট কি মানুষ করবে, নাকি CI করবে — এবং কেন?

Lab ১২ • Argo CD Deployment

Lab 12 — Argo CD Application Deployment

১২.১ শেখার উদ্দেশ্য

এই Lab শেষে আপনি পারবেন — লোকাল ক্লাস্টারে **Argo CD** ইনস্টল করতে; একটি **Application** বানিয়ে Git Repo থেকে অ্যাপ Sync করতে; **Sync Status ও Health** পড়তে; এবং ভুল Sync হলে **Rollback** করতে।

- Argo CD-র UI ও CLI ব্যবহার করা।
- Auto-Sync বনাম Manual Sync-এর পার্থক্য বোঝা।
- Git-এ বদল করলেই ক্লাস্টারে বদল আসে — তা প্রমাণ করা।

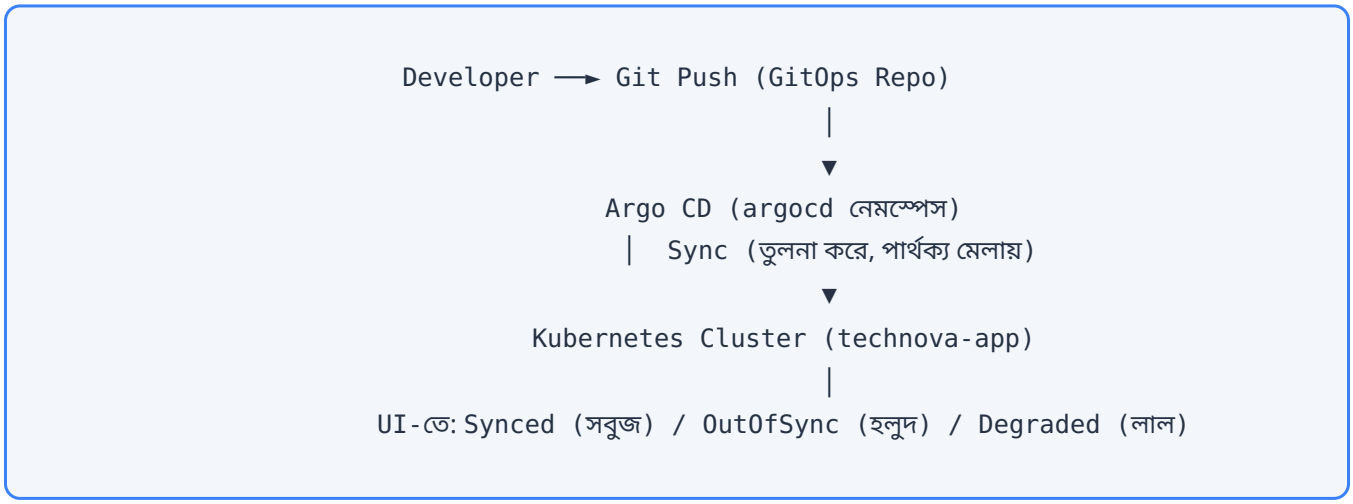
১২.২ পূর্বশর্ত

Lab 11-এর GitOps Repo, Lab 6-এর ক্লাস্টার ধারণা।

১২.৩ প্রয়োজনীয় অবকাঠামো

লোকাল kind ক্লাস্টার; Argo CD (Helm বা Manifest দিয়ে); GitHub Repo-র URL। সময় ৩–৪ ঘণ্টা; খরচ-ঝুঁকি **Low** (লোকাল)।

১২.৪ আর্কিটেকচার ডায়াগ্রাম



১২.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — Argo CD ইনস্টল করুন।

```
kubectl create namespace argocd
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/ma
kubectl get pods -n argocd -w
# সব Pod Running হওয়া পর্যন্ত অপেক্ষা করুন
```

ধাপ ২ — UI-তে ঢুকুন ও প্রাথমিক পাসওয়ার্ড নিন।

```
kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}"
kubectl port-forward svc/argocd-server -n argocd 8080:443
# ব্রাউজারে https://localhost:8080 — admin ব্যবহারকারী, উপরের পাসওয়ার্ড
# প্রথম লগইনের পর পাসওয়ার্ড বদলান; এই পাসওয়ার্ড কোথাও লিখে রাখবেন না।
```

ধাপ ৩ — Application বানান।

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: catalog-service-dev
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/CHANGE-ME-ORG/technova-gitops.git
    targetRevision: main
    path: apps/catalog-service/overlays/dev
  destination:
    server: https://kubernetes.default.svc
    namespace: technova-app
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
```

```
kubectl apply -f argocd-app.yaml
kubectl get application -n argocd
argocd app get catalog-service-dev
# অথবা UI-তে Sync Status দেখুন
```

ধাপ ৪ — Git বদল করে Sync পরীক্ষা করুন। Overlay-তে Replica ১ থেকে ২ করুন, Commit ও Push করুন; Argo CD তা ধরে ক্লাস্টারে মেলায় কি না দেখুন। তারপর ক্লাস্টারে হাতে একটি Pod মুছে দেখুন — Self-Heal তা ফিরিয়ে আনে কি না।

১২.৬ প্রত্যাশিত আউটপুট

Application Status `Synced` ও `Healthy`; Git-এ Replica বদলালে ক্লাস্টারে Replica বদলায়; হাতে মুছা Pod ফিরে আসে।

১২.৭ যাচাই

```
kubectl get application -n argocd
kubectl get pods -n technova-app
kubectl get application catalog-service-dev -n argocd -o jsonpath='{.status.sync.status
kubectl get application catalog-service-dev -n argocd -o jsonpath='{.status.health.stat
```

১২.৮ সমস্যা সমাধান

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Application OutOfSync থাকে	Repo-র Path ভুল / Branch ভুল	UI-তে Diff দেখুন; Path ও targetRevision মেলান।
Sync ব্যর্থ	YAML ভুল / Quota / Image পাওয়া যায়নি	Argo CD-র Sync লগ ও Kubernetes Event দেখুন।
প্রাথমিক পাসওয়ার্ড পাওয়া যায় না	Secret আগেই মুছে গেছে	প্রথম লগইনের পর Secret মুছে যায় — পাসওয়ার্ড বদলে নিন।

১২.৯ নিরাপত্তা পর্যালোচনা

Argo CD-র Admin পাসওয়ার্ড প্রথমেই বদলান। প্রোডাকশনে Argo CD-র UI ইন্টারনেটে খুলবেন না; RBAC দিয়ে কে Sync করতে পারে, তা সীমিত করুন। Auto-Sync প্রোডাকশনে সাবধানে ব্যবহার করুন — Lab 17-এ এর নিরাপদ রূপ আসবে।

১২.১০ Cleanup

```
kubectl delete application catalog-service-dev -n argocd
kubectl delete namespace argocd
kind delete cluster --name technova-dev
```

১২.১১ ডকুমেন্টেশন ডেলিভারেবল

- `gitops/argocd-app.yaml`।
- `docs/lab-12-notes.md` — Sync পরীক্ষার ফল ও Rollback পদ্ধতি।

১২.১২ প্রতিফলনমূলক প্রশ্ন

1. Auto-Sync কখন আশীর্বাদ, কখন বিপদ?
2. Git-এ ভুল Commit গেলে Argo CD কী করবে — আর তা আটকানোর উপায় কী?
3. Argo CD নিজে ভেঙে গেলে ক্লাস্টারের অ্যাপগুলো কি চলতে থাকবে?

REAL WORLD: GitOps-এর সবচেয়ে বড় সুবিধা Audit নয় — আত্মবিশ্বাস। “প্রোডাকশনে কী চলছে?” — এই প্রশ্নের উত্তর আর কারো স্মৃতিতে নয়, Git-এর একটি Commit Hash-এ থাকে।

Lab ১৩ • Flux CD Reconciliation

Lab 13 — Flux CD Reconciliation Workflow

১৩.১ শেখার উদ্দেশ্য

এই Lab শেষে আপনি পারবেন — **Flux CD**-র **GitRepository, Kustomization ও HelmRelease** ধারণা ব্যাখ্যা করতে; Flux দিয়ে Git থেকে ক্লাস্টারে **Reconciliation** চালাতে; এবং Argo CD-র সাথে Flux-এর পার্থক্য নিজের ভাষায় বলতে।

- Flux Bootstrap-এর ধারণা বোঝা।
- Reconciliation Interval ও Suspend-এর ব্যবহার।
- Flux-এর লগ থেকে Sync-এর ব্যর্থতা পড়া।

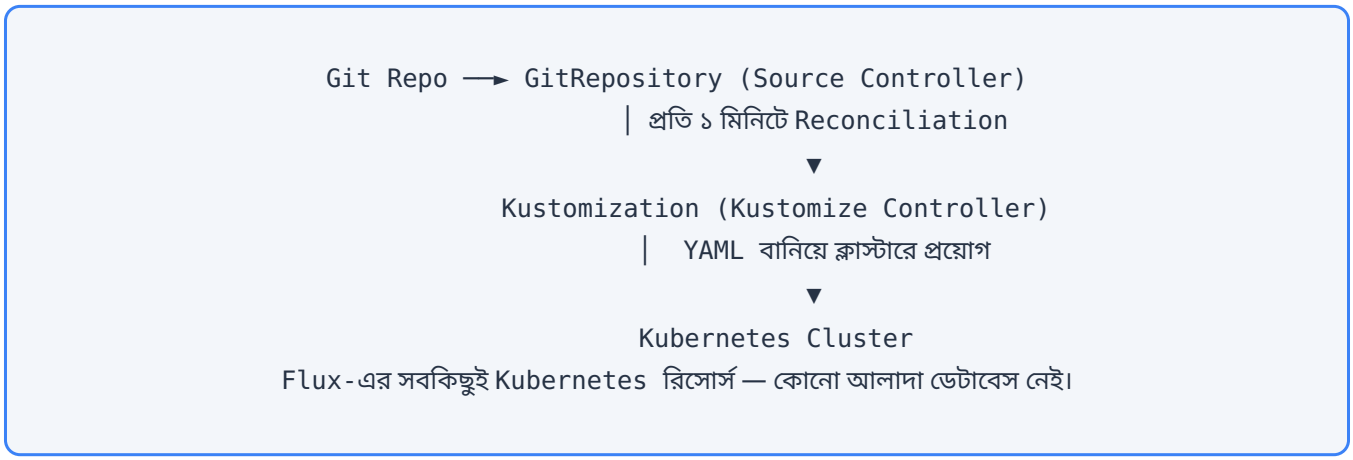
১৩.২ পূর্বশর্ত

Lab 11–12, GitOps ধারণা।

১৩.৩ প্রয়োজনীয় অবকাঠামো

লোকাল kind ক্লাস্টার; Flux CLI; GitHub Repo (পরীক্ষামূলক)। সময় ৩ ঘণ্টা; খরচ-ঝুঁকি **Low**। **নোট:** Flux Bootstrap-এ GitHub Token লাগে — Token-টি পরিবেশ চলকে রাখুন, ফাইলে নয়, এবং শুধু Repo-স্কোপের সীমিত Token ব্যবহার করুন।

১৩.৪ আর্কিটেকচার ডায়াগ্রাম



১৩.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — Flux ইনস্টল যাচাই করুন।

```
flux check --pre
# এই কমান্ডটি শুধু পূর্বশর্ত যাচাই করে; কিছু ইনস্টল করে না
```

ধাপ ২ — GitRepository ও Kustomization লিখুন (Bootstrap ছাড়া, শেখার জন্য)।

```
apiVersion: source.toolkit.fluxcd.io/v1
kind: GitRepository
metadata:
  name: technova-gitops
  namespace: flux-system
spec:
  interval: 1m
  url: https://github.com/CHANGE-ME-ORG/technova-gitops.git
  ref:
    branch: main
---
apiVersion: kustomize.toolkit.fluxcd.io/v1
kind: Kustomization
metadata:
  name: catalog-service-dev
  namespace: flux-system
spec:
  interval: 5m
  path: ./apps/catalog-service/overlays/dev
  prune: true
  sourceRef:
    kind: GitRepository
    name: technova-gitops
```

ধাপ ৩ — Reconciliation পর্যবেক্ষণ করুন।

```
flux get sources git
flux get kustomizations
kubectl -n flux-system logs deploy/source-controller --tail=30
kubectl -n flux-system logs deploy/kustomize-controller --tail=30
```

ধাপ ৪ — Suspend ও Resume পরীক্ষা করুন। একটি Kustomization Suspend করুন, Git-এ বদল করুন — ক্লাস্টারে বদল আসে না; Resume করলে আসে। এই পরীক্ষাটিই জরুরি সময়ে “ডেপ্লয় থামাও” বোতামটির প্রমাণ।

```
flux suspend kustomization catalog-service-dev
flux resume kustomization catalog-service-dev
flux reconcile kustomization catalog-service-dev --with-source
```

১৩.৬ প্রত্যাশিত আউটপুট

`flux get kustomizations`-এ Status `Applied`; Git বদল নির্ধারিত Interval-এ ক্লাস্টারে আসে; Suspend অবস্থায় আসে না।

১৩.৭ যাচাই

```
flux get all
kubectl get gitrepository -n flux-system
kubectl get kustomization -n flux-system
```

১৩.৮ সমস্যা সমাধান

লক্ষণ	সম্ভাব্য কারণ	করণীয়
GitRepository Ready নয়	URL / Branch / Token ভুল	`flux get sources git`-এর Message পড়ুন; প্রাইভেট Repo হলে Secret যাচাই করুন।
Kustomization ব্যর্থ	Kustomize Path ভুল	Path-এ `./` আছে কি না ও `kustomization.yaml` আছে কি না দেখুন।

১৩.৯ নিরাপত্তা পর্যালোচনা

Flux-এর Git Token-এর স্কোপ সীমিত রাখুন। Flux নিজে ক্লাস্টারের ভেতরে চলে — তার ServiceAccount-এর অনুমতি অতিরিক্ত বড় রাখবেন না।

১৩.১০ Cleanup

```
flux uninstall --silent
kind delete cluster --name technova-dev
```

১৩.১১ ডকুমেন্টেশন ডেলিভারেবল

- `gitops/flux-git-repository.yaml`, `flux-kustomization.yaml`।
- `docs/lab-13-notes.md` — Argo CD বনাম Flux তুলনা।

১৩.১২ প্রতিফলনমূলক প্রশ্ন

1. Argo CD-র UI থাকা কখন সুবিধা, আর Flux-এর “সবকিছুই Kubernetes রিসোর্স” হওয়া কখন সুবিধা?
2. Reconciliation Interval ১ মিনিট বনাম ১০ মিনিট — কখন কোনটি বেছে নেবেন?
3. একই ক্লাস্টারে Argo CD ও Flux দুটোই চালালে কী সমস্যা হতে পারে?

Lab ১৪ • Helm & Kustomize Release

Lab 14 — Helm and Kustomize Release Management

১৪.১ শেখার উদ্দেশ্য

এই Lab শেষে আপনি পারবেন — একটি অ্যাপের জন্য **Helm Chart** বানাতে; **Values** দিয়ে পরিবেশভেদে কনফিগারেশন বদলাতে; **Helm Rollback** করতে; এবং কখন Helm, কখন Kustomize ব্যবহার করবেন — তা সিদ্ধান্ত নিতে।

- Chart-এর কাঠামো (Chart.yaml, values.yaml, templates/) বোঝা।
- `helm template` দিয়ে রেন্ডার করা YAML পরীক্ষা করা।
- একটি Release আপগ্রেড করে আগের সংস্করণে ফেরা।

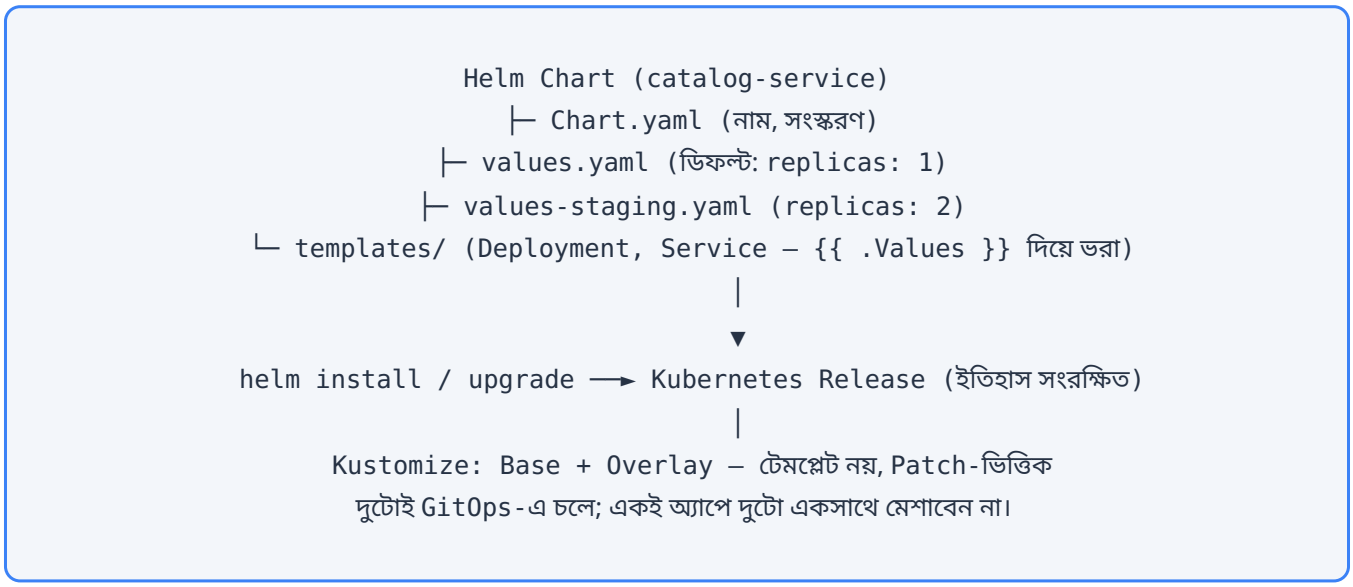
১৪.২ পূর্বশর্ত

Lab 7, Lab 11; Helm-এর প্রাথমিক ধারণা।

১৪.৩ প্রয়োজনীয় অবকাঠামো

লোকাল kind ক্লাস্টার, Helm। সময় ৩ ঘণ্টা; খরচ-ঝুঁকি **Low**।

১৪.৪ আর্কিটেকচার ডায়াগ্রাম



১৪.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — Chart বানান।

```
helm create catalog-service
ls catalog-service/
cat catalog-service/values.yaml | head -30
```

ধাপ ২ — Values সহজ করুন। ডেমো Chart-এর অপ্রয়োজনীয় অংশ মুছে শুধু Deployment ও Service রাখুন; `values.yaml`-এ Image, Replica ও Port রাখুন।

ধাপ ৩ — Template রেন্ডার করে পরীক্ষা করুন (ইনস্টল না করে)।

```
helm template catalog-service ./catalog-service --values values-staging.yaml
helm lint ./catalog-service
```

ধাপ ৪ — ইনস্টল, আপগ্রেড ও রোলব্যাক করুন।

```
helm install catalog-dev ./catalog-service -n technova-app --create-namespace
helm list -n technova-app
helm upgrade catalog-dev ./catalog-service -n technova-app --set replicaCount=2
helm history catalog-dev -n technova-app
helm rollback catalog-dev 1 -n technova-app
helm list -n technova-app
```

ধাপ ৫ — Kustomize দিয়ে একই কাজের তুলনা করুন। একই অ্যাপ Kustomize Overlay দিয়ে ডেপ্লয় করুন; দুই পদ্ধতির YAML-এর পার্থক্য এক প্যারায় লিখুন।

১৪.৬ প্রত্যাশিত আউটপুট

Chart Lint পাস করে; Install-এ Pod চলে; Upgrade-এ Replica বাড়ে; Rollback-এ আগের অবস্থায় ফেরে; History-তে প্রতিটি Revision দেখা যায়।

১৪.৭ যাচাই

```
helm list -n technova-app
helm status catalog-dev -n technova-app
helm history catalog-dev -n technova-app
kubectl get pods -n technova-app
```

১৪.৮ সমস্যা সমাধান

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Template-এ ভুল YAML	ইনডেন্টেশন / টেমপ্লেট সিনট্যাক্স	`helm template` দিয়ে রেন্ডার করে দেখুন; ইনস্টল নয়।
Upgrade ব্যর্থ, Release আটকে আছে	আগের Release Failed অবস্থায়	`helm history` দেখে Rollback করুন; প্রয়োজনে `--atomic` ব্যবহার করুন।

১৪.৯ নিরাপত্তা পর্যালোচনা

Values ফাইলে Secret রাখবেন না — Secret আলাদা পদ্ধতিতে দিন। Chart-এর সংস্করণ (Version) বদল না করে একই সংস্করণে ভিন্ন কোড প্রকাশ করবেন না।

১৪.১০ Cleanup

```
helm uninstall catalog-dev -n technova-app
kind delete cluster --name technova-dev
```

১৪.১১ ডকুমেন্টেশন ডেলিভারেবল

- `charts/catalog-service/` — সম্পূর্ণ Chart।
- `docs/lab-14-notes.md` — Helm বনাম Kustomize সিদ্ধান্ত-নোট।

১৪.১২ প্রতিফলনমূলক প্রশ্ন

1. একটি Chart যখন অনেক পরিবেশে ব্যবহার হয়, তখন Values কীভাবে নিয়ন্ত্রণে রাখবেন?
2. Rollback কি ডেটাবেসের পরিবর্তনও ফিরিয়ে আনে? কেন নয়?
3. Helm Chart-এর সংস্করণ ও অ্যাপের সংস্করণ — দুটো আলাদা কেন রাখা হয়?

Lab ১৫ • Backstage Developer Portal

Lab 15 — Backstage Developer Portal

১৫.১ শেখার উদ্দেশ্য

এই Lab শেষে আপনি পারবেন — লোকালে **Backstage** চালিয়ে **Software Catalog**-এ একটি সার্ভিস নিবন্ধন করতে; **catalog-info.yaml** লিখতে; এবং **TechDocs**-এর ধারণা দিয়ে ডকুমেন্টেশনকে কোডের পাশে রাখতে।

- Catalog-এ Component, System ও Owner-এর সম্পর্ক বোঝা।
- একটি সার্ভিসের মালিক, লাইফসাইকেল ও নির্ভরতা নথিভুক্ত করা।
- Portal ছাড়া ডেভেলপার অভিজ্ঞতা কেমন হয় — তা ব্যাখ্যা করা।

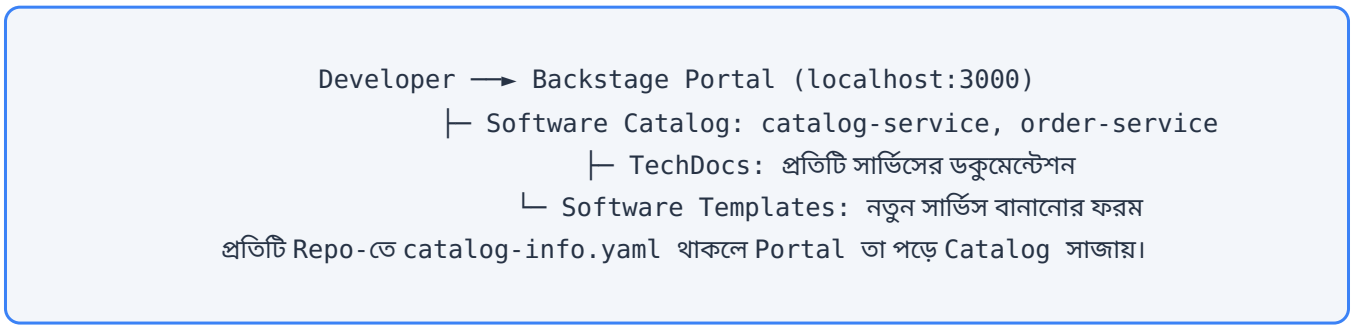
১৫.২ পূর্বশর্ত

Lab 2, Lab 11; Node.js ও YAML-এর অভিজ্ঞতা। Backstage ভারী — লোকাল মেশিনে ধীর হতে পারে; ধৈর্য রাখুন।

১৫.৩ প্রয়োজনীয় অবকাঠামো

লোকাল মেশিনে Backstage (npx দিয়ে তৈরি অ্যাপ) অথবা Docker-এ চালানো Backstage; Git Repo। সময় ৪ ঘণ্টা; খরচ-ঝুঁকি **Low** (লোকাল)। **সতর্কতা:** Backstage প্রোডাকশনে চালাতে ডেটাবেস ও Authentication লাগে — এই Lab-এ শুধু লোকাল ডেমো।

১৫.৪ আর্কিটেকচার ডায়াগ্রাম



১৫.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — Backstage অ্যাপ তৈরি করুন (প্রথমবারে সময় লাগবে)।

```
npx @backstage/create-app@latest
# অ্যাপের নাম: technova-portal
# ডিফল্ট সেটআপেই চালান; প্রথমে কোনো প্লাগইন যোগ করবেন না
cd technova-portal
yarn install
yarn dev
# ব্রাউজারে http://localhost:3000 খুলুন
```

ধাপ ২ — catalog-info.yaml লিখুন।

```
# catalog-service/catalog-info.yaml
apiVersion: backstage.io/v1alpha1
kind: Component
metadata:
  name: catalog-service
  description: TechNova-র পণ্য তালিকা সার্ভিস
  annotations:
    github.com/project-slug: CHANGE-ME-ORG/catalog-service
spec:
  type: service
  lifecycle: production
  owner: team-technova-platform
  system: technova-commerce
  providesApis:
    - catalog-api
```

ধাপ ৩ — Catalog-এ নিবন্ধন করুন। Portal-এর Catalog পেজ থেকে “Register Existing Component” দিয়ে Repo-র URL দিন; Component দেখা যায় কি না যাচাই করুন।

ধাপ ৪ — TechDocs যোগ করুন। সার্ভিসের পাশে `docs/index.md` রাখুন — কী করে, কীভাবে চালাতে হয়, কার সাথে যোগাযোগ করতে হয়। Portal-এ Docs ট্যাব দেখা যায় কি না পরীক্ষা করুন।

১৫.৬ প্রত্যাশিত আউটপুট

Portal-এ catalog-service দেখা যায়; Owner, Lifecycle ও System দেখায়; TechDocs পেজ খোলে।

১৫.৭ যাচাই

```
# Portal-এ Catalog → catalog-service খুলুন
# Owner: team-technova-platform, Lifecycle: production — দুটোই দেখা যায় কি?
# ভুল YAML থাকলে Portal-এ Error দেখাবে — সেই বার্তাটি নোট করুন
```

১৫.৮ সমস্যা সমাধান

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Backstage চালু হতে অনেক সময় লাগে	প্রথমবার Dependency ডাউনলোড	অপেক্ষা করুন; মেশিনে অন্তত ৮ GB RAM রাখুন।
Component নিবন্ধন হয় না	YAML-এ ভুল / Owner Group নেই	YAML যাচাই করুন; Owner হিসেবে `team-technova-platform`-এর Group ফাইল যোগ করুন।

১৫.৯ নিরাপত্তা পর্যালোচনা

Portal-এ Authentication ছাড়া প্রোডাকশনে চালাবেন না — কে কোন সার্ভিস দেখতে পারে, তা নিয়ন্ত্রণ করতে হয়। Catalog-এ Secret বা অভ্যন্তরীণ URL লিখবেন না।

১৫.১০ Cleanup

```
# Backstage Dev সার্ভার বন্ধ করুন (Ctrl+C)
# technova-portal ডিরেক্টরি সংরক্ষণ করুন — Lab 16-এ এটি আবার লাগবে
# কোনো ক্লাউড রিসোর্স তৈরি হয়নি, তাই মুছতে কিছু নেই
```

১৫.১১ ডকুমেন্টেশন ডেলিভারেবল

- `catalog-service/catalog-info.yaml`।
- `docs/lab-15-notes.md` — Catalog-এর সার্ভিস-তালিকা।

১৫.১২ প্রতিফলনমূলক প্রশ্ন

1. Catalog না থাকলে নতুন ডেভেলপার প্রথম দিনে কী কী খুঁজে পান না?
2. Owner না লেখা থাকলে একটি সার্ভিস ভাঙলে কাকে ডাকবেন?
3. Portal কি শুধু সুবিধা, নাকি নিরাপত্তারও অংশ — কীভাবে?

Lab ১৬ • Self-Service Template

Lab 16 — Self-Service Application Deployment Template

১৬.১ শেখার উদ্দেশ্য

এই Lab শেষে আপনি পারবেন — Backstage-এ একটি **Software Template** লিখতে যা দিয়ে ডেভেলপার ফরম পূরণ করেই নতুন সার্ভিসের Repo, CI ও Kubernetes Manifest পান; এবং **Golden Path** বলতে কী বোঝায় — তা ব্যাখ্যা করতে।

- Template-এর ধাপ (Fetch, Publish, Register) বোঝা।
- নতুন সার্ভিসে স্বয়ংক্রিয়ভাবে মানসম্মত কাঠামো দেওয়া।
- Template-এর ভুল পরীক্ষা করে ঠিক করা।

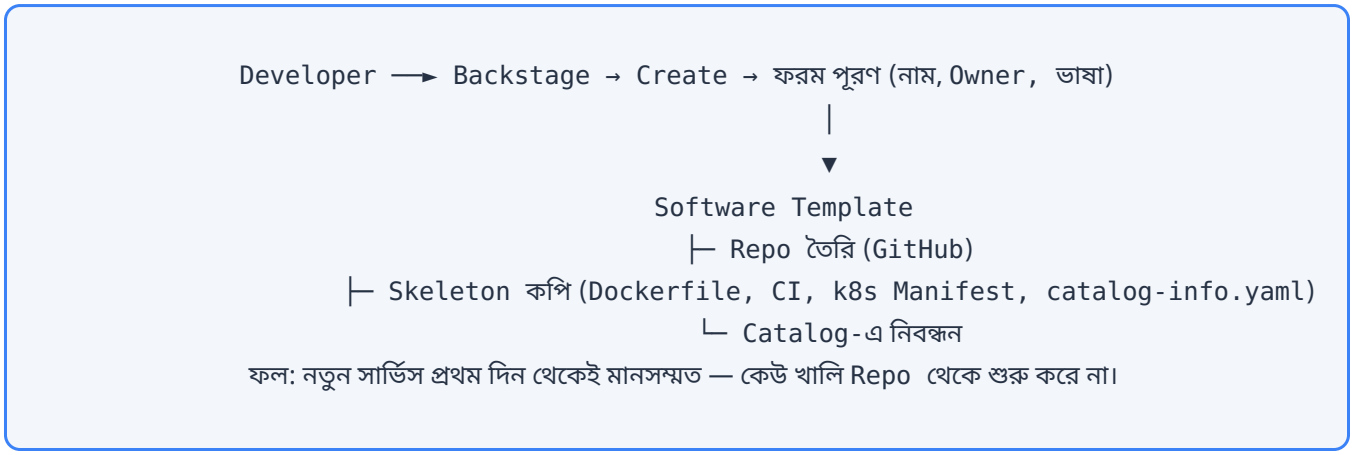
১৬.২ পূর্বশর্ত

Lab 15, GitHub Repo তৈরি করার অভিজ্ঞতা।

১৬.৩ প্রয়োজনীয় অবকাঠামো

Lab 15-এর Backstage অ্যাপ; একটি Template Skeleton Repo। সময় ৪ ঘণ্টা; খরচ-ঝুঁকি **Low**।

১৬.৪ আর্কিটেকচার ডায়াগ্রাম



১৬.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — Skeleton Repo বানান। একটি ছোট Node.js সার্ভিসের কাঠামো তৈরি করুন: `Dockerfile`, `package.json`, `k8s/deployment.yaml`, `catalog-info.yaml`, `README.md` — প্রতিটিতে সার্ভিসের নামের জায়গায় `${{ values.name }}`-ধরনের প্লেসহোল্ডার রাখুন।

ধাপ ২ — Template ফাইল লিখুন।

```
# templates/node-service/template.yaml (সংক্ষেপ)
apiVersion: scaffolder.backstage.io/v1beta3
kind: Template
metadata:
  name: node-service-template
  title: TechNova Node.js Service
  description: মানসম্মত Node.js সার্ভিস তৈরি করুন
spec:
  owner: team-technova-platform
  type: service
  parameters:
    - title: Service Details
      properties:
        name:
          title: Service Name
          type: string
        owner:
          title: Owner Team
          type: string
  steps:
    - id: fetch-base
      name: Fetch Base
      action: fetch:template
      input:
        url: ./skeleton
        values:
          name: ${{ parameters.name }}
    - id: publish
      name: Publish
      action: publish:github
      input:
        allowedHosts: ["github.com"]
        description: "TechNova service: ${{ parameters.name }}"
        repoUrl: "github.com?repo=${{ parameters.name }}&owner=CHANGE-ME-ORG"
    - id: register
      name: Register
      action: catalog:register
      input:
        repoContentsUrl: ${{ steps['publish'].output.repoContentsUrl }}
        catalogInfoPath: "/catalog-info.yaml"
```

ধাপ ৩ — Backstage-এ Template যোগ করুন ও পরীক্ষা করুন। `app-config.yaml`-এ Template-এর অবস্থান যোগ করে Backstage পুনরায় চালান; Create পেজে Template দেখা যায় কি না দেখুন।

ধাপ ৪ — শুকনো পরীক্ষা (Dry Run) করুন। প্রথমে আসল Repo তৈরি না করে Template-এর ধাপগুলো পড়ে দেখুন — কোন ধাপে কী ইনপুট লাগে, কী আউটপুট আসে। GitHub Token না থাকলে Publish ধাপটি ম্যানুয়ালি অনুকরণ করে নোট লিখুন।

১৬.৬ প্রত্যাশিত আউটপুট

Create পেজে Template দেখা যায়; ফরম পূরণ করলে Skeleton-সহ Repo তৈরি হয়; নতুন Repo-তে Catalog ফাইল ও Kubernetes Manifest থাকে।

১৬.৭ যাচাই

```
# নতুন Repo-তে দেখুন:
# - Dockerfile আছে কি? catalog-info.yaml আছে কি?
# - k8s/deployment.yaml-এ সার্ভিসের নাম ঠিক আছে কি?
# - README-তে কীভাবে চালাতে হয় লেখা আছে কি?
```

১৬.৮ সমস্যা সমাধান

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Template Create পেজে দেখা যায় না	app-config-এ অবস্থান যোগ হয়নি	Config ফাইল ও Template-এর Path মেলান; Backstage পুনরায় চালান।
Publish ব্যর্থ	GitHub Token নেই / অনুমতি নেই	Token-এর স্কোপ যাচাই করুন; শেখার জন্য Publish ধাপ বাদ দিয়ে Skeleton পরীক্ষা করুন।

১৬.৯ নিরাপত্তা পর্যালোচনা

Template থেকে তৈরি প্রতিটি Repo-তে একই নিরাপত্তা মান থাকবে — non-root User, Secret-এর প্লেসহোল্ডার, Policy-সম্মত Manifest। Template-ই সেই মানের গ্যারান্টি; প্রতিটি Repo-তে আলাদা করে মনে করিয়ে দেওয়া নয়।

১৬.১০ Cleanup

পরীক্ষামূলক Repo মুছে দিন (যদি তৈরি করে থাকেন); Backstage অ্যাপ সংরক্ষণ করুন।

১৬.১১ ডকুমেন্টেশন ডেলিভারেবল

- `templates/node-service/template.yaml` ও Skeleton।
- `docs/lab-16-notes.md` — Golden Path-এর নিয়ম-তালিকা।

১৬.১২ প্রতিফলনমূলক প্রশ্ন

1. Golden Path কখন সাহায্য করে, আর কখন ডেভেলপারের স্বাধীনতা কেড়ে নেয়?
2. Template-এ ভুল থাকলে সেই ভুল কতগুলো সার্ভিসে ছড়াবে?
3. নতুন সার্ভিস তৈরি করতে কত সময় লাগা উচিত — আর কেন?

> **CONCEPT:** Self-Service মানে “যা খুশি করো” নয় — মানে “নিরাপদ পথটি সবচেয়ে সহজ পথ”। ডেভেলপার ফরম পূরণ করলেই সঠিক কাঠামো পান; ভুল কাঠামো বানাতে তাঁকে ইচ্ছা করে পথ ছাড়তে হয়।

Lab ১৭ • Progressive Delivery & Canary

Lab 17 — Progressive Delivery and Canary Release

১৭.১ শেখার উদ্দেশ্য

এই Lab শেষে আপনি পারবেন — **Argo Rollouts** বা সমতুল্য পদ্ধতিতে **Canary Release** করতে; **Analysis** (Error Rate, Latency) দিয়ে স্বয়ংক্রিয়ভাবে এগোনো বা ফেরা (Promote / Rollback) ঠিক করতে; এবং **Blue-Green** ও **Canary**-র পার্থক্য ব্যাখ্যা করতে।

- ধাপে ধাপে ট্রাফিক বাড়ানো (১০% → ৫০% → ১০০%)।
- ভুল সংস্করণে স্বয়ংক্রিয় Rollback পরীক্ষা করা।
- Canary-র সিদ্ধান্ত-মানদণ্ড লেখা।

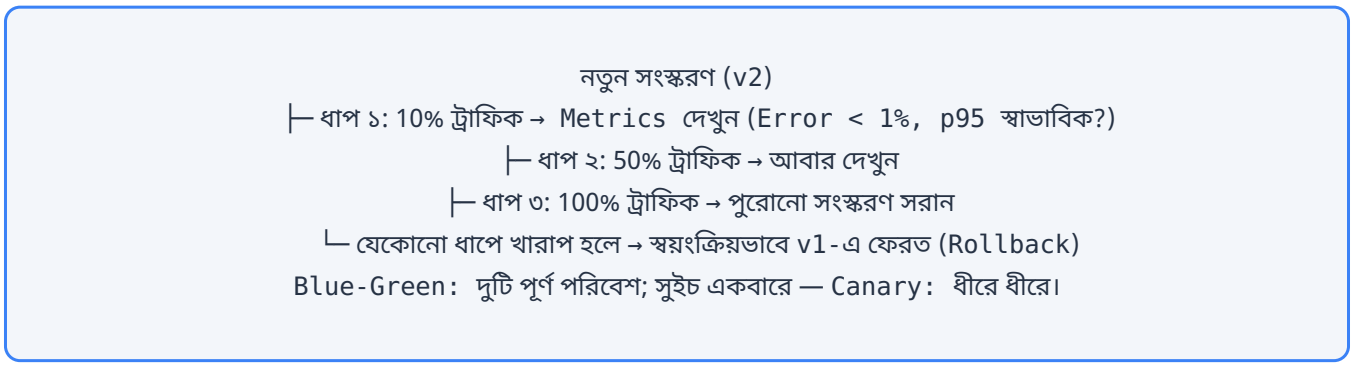
১৭.২ পূর্বশর্ত

Lab 8, Lab 12; Argo CD-র অভিজ্ঞতা।

১৭.৩ প্রয়োজনীয় অবকাঠামো

লোকাল kind ক্লাস্টার; Argo Rollouts (অথবা Gateway Weight দিয়ে ম্যানুয়াল Canary — ধারণা একই); দুটি সংস্করণের ডেমো সার্ভিস। সময় ৪ ঘণ্টা; খরচ-ঝুঁকি **Low**।

১৭.৪ আর্কিটেকচার ডায়াগ্রাম



১৭.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — Argo Rollouts ইনস্টল করুন (অথবা Gateway Weight পদ্ধতি ব্যবহার করুন)।

```
kubectl create namespace argo-rollouts
kubectl apply -n argo-rollouts -f https://github.com/argoproj/argo-rollouts/releases/do
kubectl get pods -n argo-rollouts
```

ধাপ ২ — Rollout রিসোর্স লিখুন।

```
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: catalog-rollout
  namespace: technova-app
spec:
  replicas: 4
  strategy:
    canary:
      steps:
        - setWeight: 10
        - pause: { duration: 2m }
        - setWeight: 50
        - pause: { duration: 2m }
        - setWeight: 100
  selector:
    matchLabels:
      app: catalog-service
  template:
    metadata:
      labels:
        app: catalog-service
    spec:
      containers:
        - name: catalog-service
          image: technova/catalog-service:CHANGE-ME-TAG-V2
```

ধাপ ৩ — Canary চালান ও পর্যবেক্ষণ করুন।

```
kubectl argo rollouts get rollout catalog-rollout -n technova-app --watch
# অথবা kubectl get rollout -n technova-app -w
# প্রতিটি ধাপে Weight ও Pod সংখ্যা নোট করুন
```

ধাপ ৪ — খারাপ সংস্করণ দিয়ে Rollback পরীক্ষা করুন। ইচ্ছা করে একটি ভাঙা ইমেজ (ভুল Port-এ শোনে এমন সংস্করণ) দিয়ে Rollout চালান; স্বয়ংক্রিয়ভাবে বা ম্যানুয়ালি আগের সংস্করণে ফিরুন।

```
kubectl argo rollouts abort rollout catalog-rollout -n technova-app
kubectl argo rollouts promote rollout catalog-rollout -n technova-app
# Abort করলে ট্রাফিক আবার পুরোনো সংস্করণে ফেরে
```

১৭.৬ প্রত্যাশিত আউটপুট

Canary ধাপে ধাপে এগোয়; প্রতিটি ধাপে Pause থাকে; খারাপ সংস্করণে Rollback হয়; শেষে সব ট্রাফিক নতুন সংস্করণে যায়।

১৭.৭ যাচাই

```
kubectl get rollout -n technova-app
kubectl describe rollout catalog-rollout -n technova-app
kubectl get pods -n technova-app -l app=catalog-service
```

১৭.৮ সমস্যা সমাধান

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Canary আটকে আছে	Pause ধাপে অপেক্ষা করছে	এটি স্বাভাবিক — Promote কমান্ড দিয়ে এগোন; Pause-এর উদ্দেশ্য মানুষের পরীক্ষা।
নতুন Pod তৈরি হয় না	Image Tag ভুল	Image নাম ও Tag যাচাই করুন।

১৭.৯ নিরাপত্তা পর্যালোচনা

Canary-তে নতুন সংস্করণও একই Policy ও Secret নিয়ম মানে — নতুন সংস্করণ বলে নীতি শিথিল হয় না। Rollback-এর সিদ্ধান্ত কে নেবে — মানুষ না স্বয়ংক্রিয় Analysis — তা আগে থেকে লিখে রাখুন।

১৭.১০ Cleanup

```
kubectl delete rollout catalog-rollout -n technova-app
kubectl delete namespace argo-rollouts
kind delete cluster --name technova-dev
```

১৭.১১ ডকুমেন্টেশন ডেলিভারেবল

- `k8s/rollouts/catalog-rollout.yaml` ।
- `docs/lab-17-notes.md` — Canary ধাপ, মানদণ্ড ও Rollback পরীক্ষার ফল।

১৭.১২ প্রতিফলনমূলক প্রশ্ন

1. Canary-তে ১০% ব্যবহারকারী ভুল পেলে সেটি কি “গ্রহণযোগ্য” — আর কে ঠিক করবে?
2. ডেটাবেসের Schema বদলালে Canary কি কাজ করে? কেন নয়?
3. কোন পরিস্থিতিতে Blue-Green Canary-র চেয়ে ভালো?

Lab ১৮ • Terraform, Ansible & Crossplane

Lab 18 — Terraform, Ansible and Crossplane Automation

১৮.১ শেখার উদ্দেশ্য

এই Lab শেষে আপনি পারবেন — **Terraform** দিয়ে লোকাল / পরীক্ষামূলক অবকাঠামো বানাতে; **Ansible** দিয়ে সার্ভার কনফিগার করতে; এবং **Crossplane**-এর ধারণা ব্যাখ্যা করতে — Kubernetes API দিয়ে ক্লাউড রিসোর্স চাওয়া। তিনটি টুলের দায়িত্বের সীমা আলাদা করতে পারবেন।

- Terraform-এর Plan → Apply → Destroy চক্র।
- Ansible Playbook-এর Idempotence যাচাই।
- কোন কাজ কোন টুলের — এই সিদ্ধান্ত-তালিকা লেখা।

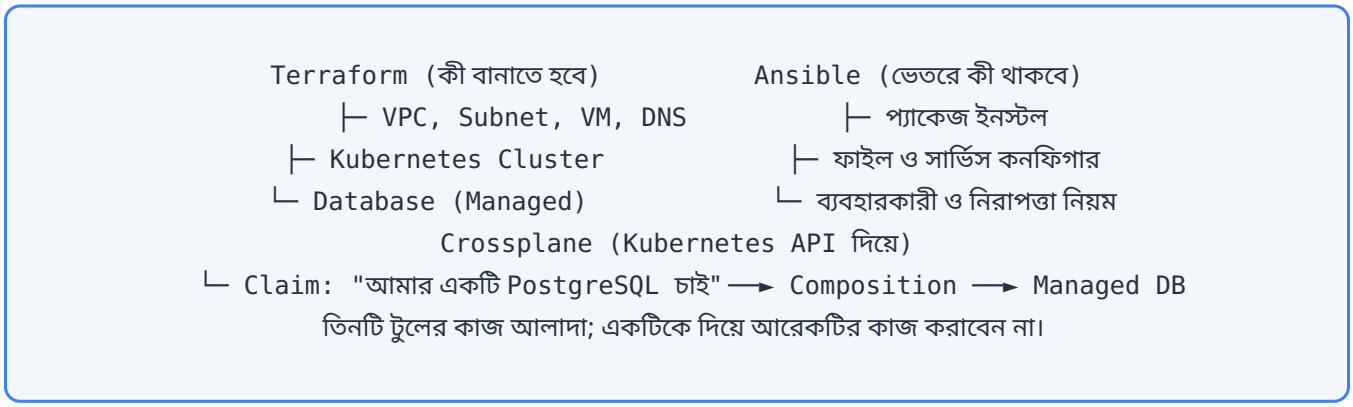
১৮.২ পূর্বশর্ত

Volume 8 (Terraform) ও Volume 9 (Ansible)-এর মৌলিক জ্ঞান।

১৮.৩ প্রয়োজনীয় অবকাঠামো

লোকাল মেশিন; Terraform; Ansible; Docker। সময় ৪ ঘণ্টা; খরচ-ঝুঁকি **Low**। **সতর্কতা:** এই Lab-এ Terraform দিয়ে আসল ক্লাউড রিসোর্স বানাবেন না — শুধু লোকাল Docker Provider বা পরীক্ষামূলক Null Resource দিয়ে ধারণা দেখান। আসল ক্লাউডে Apply করলে খরচ হতে পারে।

১৮.৪ আর্কিটেকচার ডায়াগ্রাম



১৮.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — Terraform দিয়ে লোকাল Docker কনটেইনার বানান।

```
# main.tf — Docker Provider (শেখার জন্য; প্রোডাকশনে Cloud Provider)
terraform {
  required_providers {
```

```
    docker = {
      source = "kreuzwerker/docker"
  }
 }
}
resource "docker_image" "technova" {
  name = "nginx:1.27"
}
resource "docker_container" "technova" {
  name  = "technova-lab18"
  image = docker_image.technova.image_id
  ports {
    internal = 80
    external = 18080
 }
}

terraform init
terraform plan
terraform apply
curl http://127.0.0.1:18080
terraform destroy
```

ধাপ ২ — Ansible দিয়ে কনফিগারেশন লিখুন।

```
# playbook.yaml — লোকাল মেশিনে পরীক্ষা (connection: local)
- name: TechNova base setup
  hosts: localhost
  connection: local
  tasks:
    - name: Ensure app directory exists
      ansible.builtin.file:
        path: /tmp/technova-app
        state: directory
        mode: "0755"
    - name: Write welcome file
      ansible.builtin.copy:
        dest: /tmp/technova-app/README.txt
        content: "TechNova — configured by Ansible\n"
        mode: "0644"
```

```
ansible-playbook playbook.yaml
ansible-playbook playbook.yaml
# দ্বিতীয়বার চালালে কোনো পরিবর্তন হয় না — এটাই Idempotence; "changed=0" দেখুন
```

ধাপ ৩ — Crossplane-এর ধারণা লিখুন। একটি Composition-এর খসড়া YAML লিখুন (ইনস্টল না করেও চলবে) — ডেভেলপার কী Claim করেন, প্ল্যাটফর্ম দল কী Composition লেখে।

```
# claim.yaml — ধারণা (Crossplane ইনস্টল ছাড়াও পড়া যায়)
apiVersion: technova.example/v1alpha1
kind: PostgreSQLClaim
metadata:
  name: technova-db
  namespace: technova-app
spec:
  storageGB: 10
  version: "16"
# প্ল্যাটফর্ম দল এই Claim-কে Managed DB-তে রূপান্তর করে —
# ডেভেলপারকে ক্লাউড কনসোল খুলতে হয় না।
```

১৮.৬ প্রত্যাশিত আউটপুট

Terraform Plan-এ পরিবর্তন দেখা যায়, Apply-তে কনটেইনার চলে, Destroy-তে মুছে যায়; Ansible দ্বিতীয়বারে কোনো পরিবর্তন করে না; Crossplane Claim-এর ধারণা নথিতে লেখা থাকে।

১৮.৭ যাচাই

```
terraform show
docker ps | grep technova-lab18
cat /tmp/technova-app/README.txt
terraform plan
# Plan-এ "No changes" আসা উচিত — Apply-এর পর আবার Plan করলে
```

১৮.৮ সমস্যা সমাধান

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Terraform Apply ব্যর্থ	Provider / Docker চলছে না	Docker চালু করুন; `terraform init` আবার চালান।
Ansible দ্বিতীয়বারেও Changed দেখায়	Task Idempotent নয়	Shell কমান্ডের বদলে Built-in Module ব্যবহার করুন।

১৮.৯ নিরাপত্তা পর্যালোচনা

Terraform State-এ Secret থাকতে পারে — State ফাইল Git-এ তুলবেন না; Remote Backend ও এনক্রিপশন ব্যবহার করুন। Ansible Vault ছাড়া Playbook-এ পাসওয়ার্ড লিখবেন না।

১৮.১০ Cleanup

```
terraform destroy
rm -rf /tmp/technova-app
# Crossplane এই Lab-এ ইনস্টল করা হয়নি; ধারণা-ফাইলটি সংরক্ষণ করুন
```

১৮.১১ ডকুমেন্টেশন ডেলিভারেবল

- `terraform/main.tf`, `ansible/playbook.yaml`, `crossplane/claim.yaml`।
- `docs/lab-18-notes.md` — কোন কাজ কোন টুলের — সিদ্ধান্ত-তালিকা।

১৮.১২ প্রতিফলনমূলক প্রশ্ন

1. Terraform দিয়ে সার্ভারের ভেতরের প্যাকেজ ইনস্টল করা উচিত কি? কেন নয়?
2. Ansible দিয়ে VPC বানানো উচিত কি? কেন নয়?
3. Crossplane কখন Terraform-এর বিকল্প, আর কখন শুধু একটি স্তর?

Lab ১৯ • Multi-Cloud Platform Governance

Lab 19 — Multi-Cloud Platform Governance

১৯.১ শেখার উদ্দেশ্য

এই Lab শেষে আপনি পারবেন — একাধিক ক্লাউড / পরিবেশে একই নীতি (Policy) প্রয়োগের পরিকল্পনা লিখতে; **Tagging, Naming ও Cost Allocation**-এর মান নির্ধারণ করতে; এবং **Vendor Lock-in**-এর ঝুঁকি চিহ্নিত করে তা কমানোর উপায় লিখতে।

- পরিবেশভেদে একই নিরাপত্তা নীতি লেখা।
- Tag-ভিত্তিক খরচ-বণ্টনের কাঠামো বানানো।
- একটি ক্লাউড-নির্দিষ্ট সার্ভিসের বিকল্প লেখা।

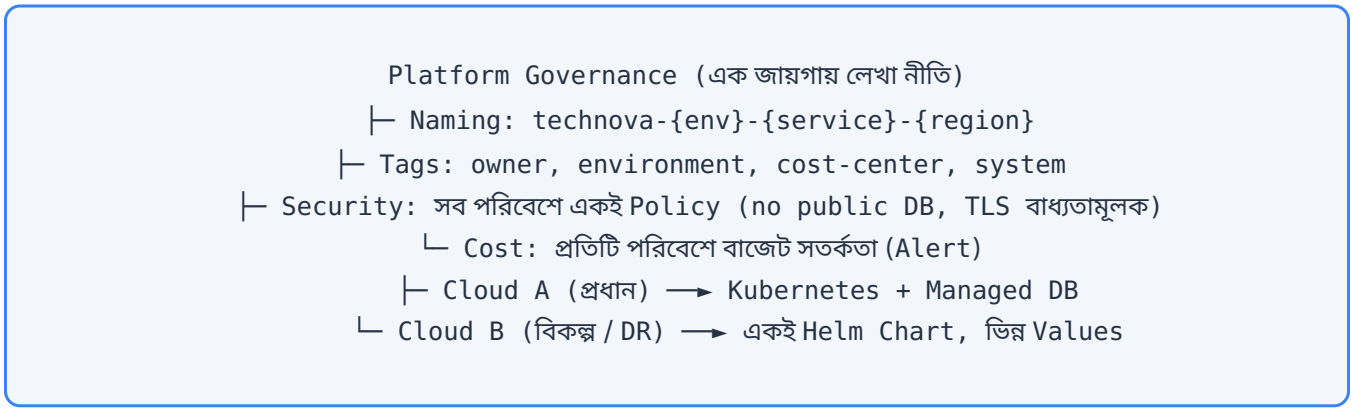
১৯.২ পূর্বশর্ত

Lab 9, Lab 18; Terraform Workspace / Variable-এর ধারণা।

১৯.৩ প্রয়োজনীয় অবকাঠামো

শুধু নথি ও Terraform Variable ফাইল — আসল মাল্টি-ক্লাউড অ্যাকাউন্ট প্রয়োজন নেই। সময় ২–৩ ঘণ্টা; খরচ-ঝুঁকি **Low**। **সতর্কতা:** এই Lab-এ কোনো আসল ক্লাউড অ্যাকাউন্ট খুলবেন না বা রিসোর্স বানাবেন না।

১৯.৪ আর্কিটেকচার ডায়াগ্রাম



১৯.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — Naming ও Tagging মান লিখুন। একটি টেবিল বানান: প্রতিটি রিসোর্সের নাম কেমন হবে, কোন Tag বাধ্যতামূলক, Tag না থাকলে কী হবে।

Tag	উদাহরণ	উদ্দেশ্য
environment	dev / staging / prod	খরচ ও ঝুঁকি আলাদা করা
owner	team-technova-platform	দায়িত্ব নির্ধারণ

cost-center	technova-commerce	খরচ-বণ্টন
system	technova-commerce	নির্ভরতা বোঝা

ধাপ ২ — Terraform Variable দিয়ে পরিবেশভেদ করুন।

```
# terraform/variables.tf (সংক্ষেপ)
variable "environment" {
  type        = string
  description = "dev, staging, or prod"
}
variable "region" {
  type        = string
  description = "Cloud region — উদাহরণ: CHANGE-ME-REGION"
}
variable "tags" {
  type = map(string)
  default = {
    owner       = "team-technova-platform"
    system      = "technova-commerce"
    cost-center = "technova-commerce"
 }
}
# মূল কোড একটাই; বদলায় শুধু .tfvars ফাইল — এটাই Multi-Cloud Governance-এর ভিত্তি
```

ধাপ ৩ — Lock-in পর্যালোচনা করুন। TechNova-র প্রতিটি ক্লাউড-নির্দিষ্ট সার্ভিসের পাশে লিখুন: এর বিকল্প কী, বদলাতে কতদিন লাগবে, ডেটা কীভাবে বের করা যাবে। অন্তত একটি সার্ভিসের জন্য Exit Plan লিখুন।

ধাপ ৪ — Governance চেকলিস্ট বানান। নতুন পরিবেশ চালু করার আগে যে ১০টি প্রশ্নের উত্তর লাগবে — তার তালিকা লিখুন (নিরাপত্তা, খরচ, ব্যাকআপ, মালিক, সতর্কতা)।

১৯.৬ প্রত্যাশিত আউটপুট

Naming/Tagging মান, Terraform Variable কাঠামো, Lock-in পর্যালোচনা ও Governance চেকলিস্ট — চারটি নথি।

১৯.৭ যাচাই

```
terraform fmt -check
terraform validate
# একটি কাল্পনিক রিসোর্সের নাম আপনার Naming মান অনুযায়ী লিখুন —
# অন্য কেউ কি নাম দেখেই পরিবেশ ও মালিক বুঝতে পারে?
```

১৯.৮ সমস্যা সমাধান

সমস্যা	কারণ	করণীয়
Tag-এর নাম একেক জায়গায় একেক রকম	মান লেখা নেই	একটি `tags.tf` -এ সব Tag কেন্দ্রীয়ভাবে নির্ধারণ করুন।
পরিবেশভেদে কোড আলাদা হয়ে যাচ্ছে	Variable-এর বদলে কপি-পেস্ট	মূল কোড একটাই রাখুন; পার্থক্য শুধু Variable-এ।

১৯.৯ নিরাপত্তা পর্যালোচনা

প্রতিটি পরিবেশে একই নিরাপত্তা নীতি — dev-এ শিথিল, prod-এ কঠোর — এই ধারণাটি ভুল। নীতি একই থাকবে; বদলাবে শুধু আকার ও খরচ। একটি পরিবেশের শংসাপত্র দিয়ে আরেক পরিবেশে ঢোকা যাবে না।

১৯.১০ Cleanup

কোনো ক্লাউড রিসোর্স তৈরি হয়নি; নথি Commit করুন।

১৯.১১ ডকুমেন্টেশন ডেলিভারেবল

- `docs/governance.md` — Naming, Tagging, Policy ও চেকলিস্ট।
- `terraform/variables.tf` ।
- `docs/lab-19-notes.md` — Lock-in পর্যালোচনা।

১৯.১২ প্রতিফলনমূলক প্রশ্ন

1. Multi-Cloud কি সবসময় ভালো? কখন এটি শুধু জটিলতা বাড়ায়?
2. একটি ছোট দলের জন্য Multi-Cloud-এর সবচেয়ে বড় খরচ কী — টাকা, নাকি সময়?
3. Tag না থাকলে খরচ-বণ্টন কীভাবে ভুল হয়?

Lab ২০ • Enterprise Observability

Lab 20 — Enterprise Observability Integration

২০.১ শেখার উদ্দেশ্য

এই Lab শেষে আপনি পারবেন — লোকাল ক্লাস্টারে **Prometheus + Grafana** চালিয়ে Metrics সংগ্রহ করতে; **Logs ও Traces**-এর সাথে Metrics-কে জুড়ে একটি ঘটনার পূর্ণ গল্প পড়তে; এবং **Alert**-এর নিয়ম লিখতে যা শুধু সমস্যা হলেই বাজে।

- kube-prometheus-stack-ধরনের প্যাকেজ দিয়ে Observability চালু করা।
- একটি সার্ভিসের Golden Signal (Latency, Traffic, Errors, Saturation) দেখা।
- Alert-এর জন্য Runbook-এর লিংক লেখা।

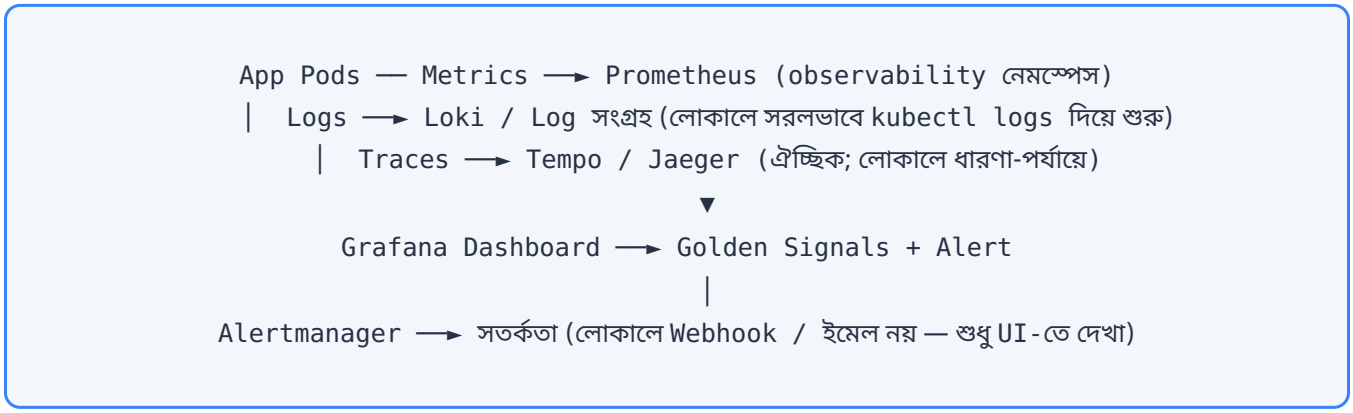
২০.২ পূর্বশর্ত

Lab 5, Lab 6; Prometheus ও Grafana-র ধারণা।

২০.৩ প্রয়োজনীয় অবকাঠামো

লোকাল kind ক্লাস্টার; Helm; অন্তত ৮ GB RAM। সময় ৪ ঘণ্টা; খরচ-ঝুঁকি **Low** (লোকাল)।

২০.৪ আর্কিটেকচার ডায়াগ্রাম



২০.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — Prometheus + Grafana ইনস্টল করুন।

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
helm install observability prometheus-community/kube-prometheus-stack \
  -n observability --create-namespace
kubectl get pods -n observability -w
# সব Pod Running হতে কয়েক মিনিট লাগতে পারে — লোকাল মেশিনে ধৈর্য রাখুন
```

ধাপ ২ — Grafana-তে ঢুকুন।

```
kubectl port-forward svc/observability-grafana -n observability 3000:80
# ব্রাউজারে http://localhost:3000 — admin / prom-operator
# প্রথম লগইনের পর পাসওয়ার্ড বদলান
```

ধাপ ৩ — Golden Signal Dashboard দেখুন। Kubernetes-এর ডিফল্ট Dashboard-এ Pod-এর CPU, মেমোরি, Restart দেখুন; তারপর একটি ছোট Dashboard বানান — catalog-service-এর Request সংখ্যা, Error হার, p95 Latency।

ধাপ ৪ — একটি Alert নিয়ম লিখুন।

```
# alert-rule.yaml — ধারণা (PrometheusRule)
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: catalog-service-alerts
  namespace: observability
spec:
  groups:
    - name: catalog-service
      rules:
        - alert: CatalogServiceHighErrorRate
          expr: |
            sum(rate(http_requests_total{job="catalog-service", status=~"5.."}[5m]))
     /
            sum(rate(http_requests_total{job="catalog-service"}[5m])) > 0.05
          for: 5m
          labels:
            severity: warning
          annotations:
            summary: "Catalog Service-এ Error হার ৫%-এর বেশি"
            runbook_url: "https://technova.example/docs/runbooks/catalog-service"
```

ধাপ ৫ — Metrics, Logs ও Trace-কে জুড়ুন। একটি ভুল অনুরোধের Request ID নিন; Metrics-এ সময় দেখুন, `kubectl logs`-এ সেই ID খুঁজুন — তিনটি উৎস একই গল্প বলে কি না, নোট করুন।

২০.৬ প্রত্যাশিত আউটপুট

Grafana-তে ক্লাস্টারের Dashboard দেখা যায়; একটি কাস্টম Dashboard-এ Catalog Service-এর Golden Signal দেখা যায়; Alert নিয়মটি Prometheus-এ লোড হয়।

২০.৭ যাচাই

```
kubectl get pods -n observability
kubectl get prometheusrules -n observability
kubectl port-forward svc/observability-kube-prometheus-prometheus -n observability 9090
# Prometheus UI-তে (http://localhost:9090) Alert-এর Status দেখুন
```

২০.৮ সমস্যা সমাধান

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Prometheus Pod Pending / OOM	RAM কম	Retention কমান; লোকালে শুধু প্রয়োজনীয় Dashboard রাখুন।
Grafana-তে ডেটা নেই	Datasource যুক্ত হয়নি	Helm Chart-এর ডিফল্ট Datasource যাচাই করুন; Pod পুনরায় চালু করুন।
Alert বাজছে না	Expr-এ Metric-এর নাম মেলেনি	Prometheus UI-তে Expr চালিয়ে দেখুন; Metric-এর Label মেলান।

২০.৯ নিরাপত্তা পর্যালোচনা

Observability-তে সংবেদনশীল ডেটা (টোকেন, পাসওয়ার্ড) লগে লিখবেন না। Grafana ও Prometheus প্রোডাকশনে ইন্টারনেটে খুলবেন না; Authentication বাধ্যতামূলক।

২০.১০ Cleanup

```
helm uninstall observability -n observability
kubectl delete namespace observability
kind delete cluster --name technova-dev
```

২০.১১ ডকুমেন্টেশন ডেলিভারেবল

- `observability/alert-rule.yaml` ও Dashboard-এর স্ক্রিনশট-বর্ণনা।
- `docs/lab-20-notes.md` — Golden Signal-এর ব্যাখ্যা।

২০.১২ প্রতিফলনমূলক প্রশ্ন

1. Metrics, Logs ও Traces-এর মধ্যে কোনটি কখন সবচেয়ে কাজে আসে?
2. Alert বেশি হলে কী হয় — Alert Fatigue কী?
3. Observability-র খরচ কখন নিজেই সমস্যা হয়ে দাঁড়ায়?

Lab ২১ • SLO & Reliability Dashboard

Lab 21 — SLO and Platform Reliability Dashboard

২১.১ শেখার উদ্দেশ্য

এই Lab শেষে আপনি পারবেন — একটি সার্ভিসের জন্য **SLI, SLO ও Error Budget** নির্ধারণ করতে; **Error Budget** শেষ হয়ে গেলে কী করবেন — সেই নীতি লিখতে; এবং Grafana-তে একটি Reliability Dashboard বানাতে।

- Availability ও Latency-ভিত্তিক SLI লেখা।
- SLO থেকে Error Budget হিসাব করা।
- Burn Rate-এর ধারণা ব্যাখ্যা করা।

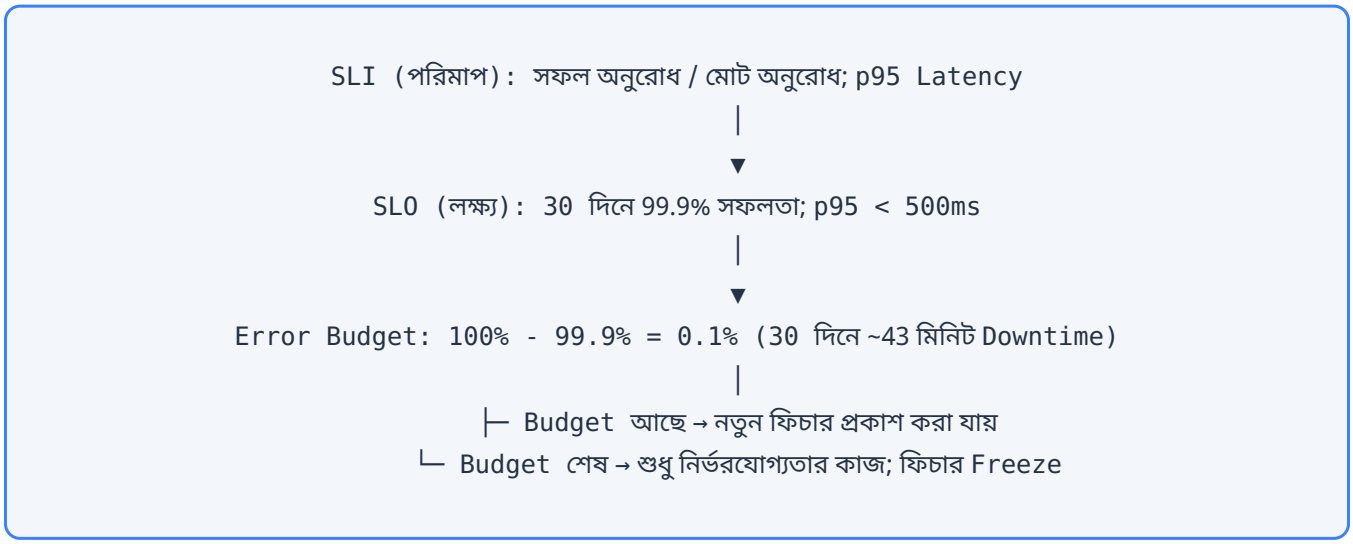
২১.২ পূর্বশর্ত

Lab 20, SRE ধারণা।

২১.৩ প্রয়োজনীয় অবকাঠামো

Lab 20-এর Observability সেটআপ; কাগজ-কলমের হিসাব। সময় ৩ ঘণ্টা; খরচ-ঝুঁকি **Low**।

২১.৪ আর্কিটেকচার ডায়াগ্রাম



২১.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — SLI নির্ধারণ করুন। catalog-service-এর জন্য দুটি SLI লিখুন: Availability (সফল অনুরোধের হার) ও Latency (p95)। কোন অনুরোধ “সফল” — তা স্পষ্ট লিখুন (যেমন HTTP 2xx ও 3xx; 5xx ব্যর্থ)।

ধাপ ২ — SLO ও Error Budget হিসাব করুন।

SLO	Error Budget (৩০ দিনে)	অনুমোদিত Downtime

99.9%	0.1%	প্রায় ৪৩ মিনিট
99.5%	0.5%	প্রায় ৩ ঘণ্টা ৩৬ মিনিট
99.0%	1.0%	প্রায় ৭ ঘণ্টা ১২ মিনিট

এই সংখ্যাগুলো হিসাবের উদাহরণ; আপনার SLO-র জন্য নিজে হিসাব করে লিখুন।

ধাপ ৩ — Dashboard বানান। Grafana-তে একটি প্যানেল যোগ করুন: গত ৩০ দিনের সফলতার হার, বর্তমান Error Budget কতটুকু বাকি, এবং Burn Rate। লোকালে ৩০ দিনের ডেটা নেই — তাই গত ১ ঘণ্টার ডেটা দিয়ে Dashboard-এর কাঠামো দেখান, এবং নোট লিখুন যে প্রোডাকশনে উইন্ডো ৩০ দিন হবে।

ধাপ ৪ — নীতি লিখুন। Error Budget শেষ হলে কী হবে — ফিচার Freeze, অন-কল বাড়ানো, নাকি SLO পুনর্বিবেচনা — এক পৃষ্ঠায় লিখুন। কে এই সিদ্ধান্ত নেবে, তাও লিখুন।

২১.৬ প্রত্যাশিত আউটপুট

`docs/slo.md`-এ SLI, SLO, Error Budget ও নীতি লেখা; Grafana-তে Reliability Dashboard-এর কাঠামো দেখা যায়।

২১.৭ যাচাই

```
# নিজেকে জিজ্ঞেস করুন:
# ১) SLO-টি কি ব্যবহারকারীর অভিজ্ঞতা প্রতিফলিত করে, নাকি শুধু সার্ভারের?
# ২) Error Budget শেষ হলে কে জানবে — Dashboard-এ কি তা দেখা যায়?
# ৩) SLO কি এত কঠিন যে কেউ তা মানতে পারবে না?
```

২১.৮ সমস্যা সমাধান

সমস্যা	কারণ	করণীয়
SLO সবসময় ভাঙছে	SLO অবাস্তব / SLI ভুল	SLO কমান অথবা SLI ঠিক করুন; SLO একটি লক্ষ্য, শাস্তি নয়।
Dashboard-এ ডেটা নেই	Metric-এর নাম মেলেনি	Prometheus-এ Query চালিয়ে Metric যাচাই করুন।

২১.৯ নিরাপত্তা পর্যালোচনা

SLO Dashboard-এ ব্যবহারকারীর ব্যক্তিগত তথ্য দেখাবেন না। Reliability ডেটা সবার জন্য খোলা রাখলে আক্রমণকারীও সিস্টেমের দুর্বল সময় জানতে পারে — Dashboard-এর প্রবেশাধিকার সীমিত রাখুন।

২১.১০ Cleanup

Dashboard-এর JSON রপ্তানি করে সংরক্ষণ করুন; Lab 20-এর মতো Cleanup করুন।

২১.১১ ডকুমেন্টেশন ডেলিভারেবল

- `docs/slo.md` — SLI, SLO, Error Budget ও নীতি।

- `observability/reliability-dashboard.json`।
- `docs/lab-21-notes.md`।

২১.১২ প্রতিফলনমূলক প্রশ্ন

1. ১০০% Availability-র SLO কেন ভুল লক্ষ্য?
2. Error Budget কি শুধু সংখ্যা, নাকি দলের সংস্কৃতির অংশ?
3. একটি সার্ভিসের SLO বদলাতে হলে কার অনুমোদন লাগবে — এবং কেন?

> **REAL WORLD:** SLO-র সবচেয়ে বড় সুবিধা প্রযুক্তিগত নয় — এটি দলকে একটি সাধারণ ভাষা দেয়। “সিস্টেমটি কি যথেষ্ট ভালো?” — এই তর্কের বদলে সবাই Error Budget-এর সংখ্যাটি দেখে সিদ্ধান্ত নেয়।

Lab ২২ • Disaster Recovery & Failover

Lab 22 — Disaster Recovery and Failover Simulation

২২.১ শেখার উদ্দেশ্য

এই Lab শেষে আপনি পারবেন — **RTO (Recovery Time Objective)** ও **RPO (Recovery Point Objective)** নির্ধারণ করতে; লোকালে একটি **Failover Simulation** চালাতে (প্রধান ডেটাবেস / ক্লাস্টার বন্ধ করে); এবং **Backup থেকে Restore** করে তা যাচাই করতে।

- DR Plan-এর ধাপ লেখা।
- Simulated বিপর্যয়ে কী হয়, তা পরিমাপ করা।
- Restore পরীক্ষা ছাড়া Backup-এর মূল্য নেই — তা প্রমাণ করা।

২২.২ পূর্বশর্ত

Lab 10, Lab 20; PostgreSQL Backup-এর অভিজ্ঞতা।

২২.৩ প্রয়োজনীয় অবকাঠামো

লোকাল kind ক্লাস্টার; Lab 10-এর PostgreSQL StatefulSet; Backup ফাইল। সময় ৪ ঘণ্টা; খরচ-ঝুঁকি **Low**। **সতর্কতা:** এই Lab শুধু লোকাল পরীক্ষামূলক ডেটায় করবেন — আসল ডেটা নয়।

২২.৪ আর্কিটেকচার ডায়াগ্রাম

স্বাভাবিক: App ⟶ Primary DB (postgres-0) ⟶ Backup (নিয়মিত)
বিপর্যয়: Primary বন্ধ ⟶ App ব্যর্থ হতে শুরু করে
পুনরুদ্ধার: Backup থেকে Restore ⟶ নতুন Primary ⟶ App আবার চলে
(সময় মাপুন: কতক্ষণে সারল? কতটুকু ডেটা হারাল?)
RTO: কতক্ষণে সারতে হবে | RPO: কতটুকু ডেটা হারানো চলবে

২২.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — RTO ও RPO লিখুন। TechNova-র জন্য লিখুন: অর্ডার ডেটার RPO ১ ঘণ্টা (১ ঘণ্টার বেশি ডেটা হারানো চলবে না), RTO ৪ ঘণ্টা (৪ ঘণ্টার মধ্যে সারাতে হবে)। এই সংখ্যাগুলো ব্যবসায়িক সিদ্ধান্ত — প্রযুক্তিগত নয় — তা নোটে লিখুন।

ধাপ ২ — Backup নিন ও যাচাই করুন।

```
kubectl exec -n technova-app postgres-0 -- pg_dump -U technova technova > /tmp/dr-backu
ls -lh /tmp/dr-backup.sql
grep -c "CREATE TABLE" /tmp/dr-backup.sql
# Backup ফাইলটি খালি নয় কি না — তা যাচাই করুন
```

ধাপ ৩ — বিপর্যয় অনুকরণ করুন। StatefulSet-এর Scale ০ করুন অথবা Pod মুছে PVC-সহ পুরো StatefulSet মুছুন (শুধু Lab-এর ডেটায়)। সময় নোট করুন।

```
date
kubectl scale statefulset postgres -n technova-app --replicas=0
kubectl get pods -n technova-app
# অ্যাপ থেকে ডেটাবেসে সংযোগের চেষ্টা করুন — ব্যর্থ হওয়া উচিত
```

ধাপ ৪ — Restore করুন ও সময় মাপুন।

```
kubectl scale statefulset postgres -n technova-app --replicas=1
kubectl exec -n technova-app postgres-0 -- psql -U technova -d technova -c "SELECT * FR
date
#  Restore-এর পর ডেটা ফিরেছে কি না, আর কতক্ষণ লাগল — নোট করুন
```

ধাপ ৫ — DR Runbook লিখুন। ধাপ ২–৪-এর প্রতিটি কমান্ড, যাচাই ও সিদ্ধান্ত-বিন্দু (কখন Failover ঘোষণা করবেন, কাকে জানাবেন) এক পৃষ্ঠায় লিখুন।

২২.৬ প্রত্যাশিত আউটপুট

বিপর্যয়ের সময় ও পুনরুদ্ধারের সময় মাপা আছে; Restore-এর পর ডেটা ফিরে এসেছে; DR Runbook লেখা আছে।

২২.৭ যাচাই

```
kubectl get pods -n technova-app
kubectl exec -n technova-app postgres-0 -- psql -U technova -d technova -c "\dt"
#  RTO-র সাথে আপনার মাপা সময় তুলনা করুন — RTO-র মধ্যে সারল কি?
```

২২.৮ সমস্যা সমাধান

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Restore-এর পর ডেটা নেই	Backup পুরোনো / ভুল ডেটাবেসে Restore	Backup-এর তারিখ যাচাই করুন; Restore-এর আগে `pg_dump` -এর বিষয়বস্তু দেখুন।
Pod ফিরে আসে না	PVC / StatefulSet মুছে গেছে	StatefulSet আবার Apply করুন; PVC আলাদা করে যাচাই করুন।

২২.৯ নিরাপত্তা পর্যালোচনা

Backup ফাইলে আসল ডেটা থাকতে পারে — এটি এনক্রিপ্ট করে রাখুন, Git-এ তুলবেন না। DR পরীক্ষার সময় প্রোডাকশন ডেটা ব্যবহার করবেন না।

২২.১০ Cleanup

```
kubectl delete statefulset postgres -n technova-app
kubectl delete pvc data-postgres-0 -n technova-app
rm /tmp/dr-backup.sql /tmp/technova-backup.sql
kind delete cluster --name technova-dev
```

২২.১১ ডকুমেন্টেশন ডেলিভারেবল

- `docs/dr-plan.md` — RTO, RPO, Failover ধাপ ও যোগাযোগ-তালিকা।
- `docs/runbooks/database-restore.md` ।
- `docs/lab-22-notes.md` — মাপা সময়।

২২.১২ প্রতিফলনমূলক প্রশ্ন

1. Backup নেওয়া থাকলেই কি DR প্রস্তুত? Restore পরীক্ষা কেন আলাদা?
2. RTO ও RPO কে ঠিক করবে — প্রকৌশলী, নাকি ব্যবসায়িক দল?
3. DR পরীক্ষা কতবার করা উচিত — আর কেন নিয়মিত করা হয় না?

> **WARNING:** যে Backup Restore করে দেখা হয়নি, সেটি Backup নয় — সেটি একটি আশা। প্রোডাকশনে DR Plan-এর সবচেয়ে সাধারণ ব্যর্থতা হলো “Backup আছে” ভেবে নেওয়া, কিন্তু Restore কখনো পরীক্ষা না করা।

Lab ২৩ • Platform Security & Cost Governance

Lab 23 — Platform Security and Cost Governance

২৩.১ শেখার উদ্দেশ্য

এই Lab শেষে আপনি পারবেন — একটি প্ল্যাটফর্মের **Security Review** চেকলিস্ট দিয়ে পরীক্ষা করতে; **Image Scanning, SBOM ও Secret Scanning**-এর ধারণা প্রয়োগ করতে; এবং **FinOps**-এর প্রাথমিক হিসাব (কোন নেমস্পেস কত Resource ব্যবহার করে) করতে।

- Trivy-ধরনের স্ক্যানার দিয়ে ইমেজ পরীক্ষা করা।
- অব্যবহৃত Resource চিহ্নিত করে খরচ কমানোর পরিকল্পনা লেখা।
- Security ও Cost-এর মধ্যে ভারসাম্য ব্যাখ্যা করা।

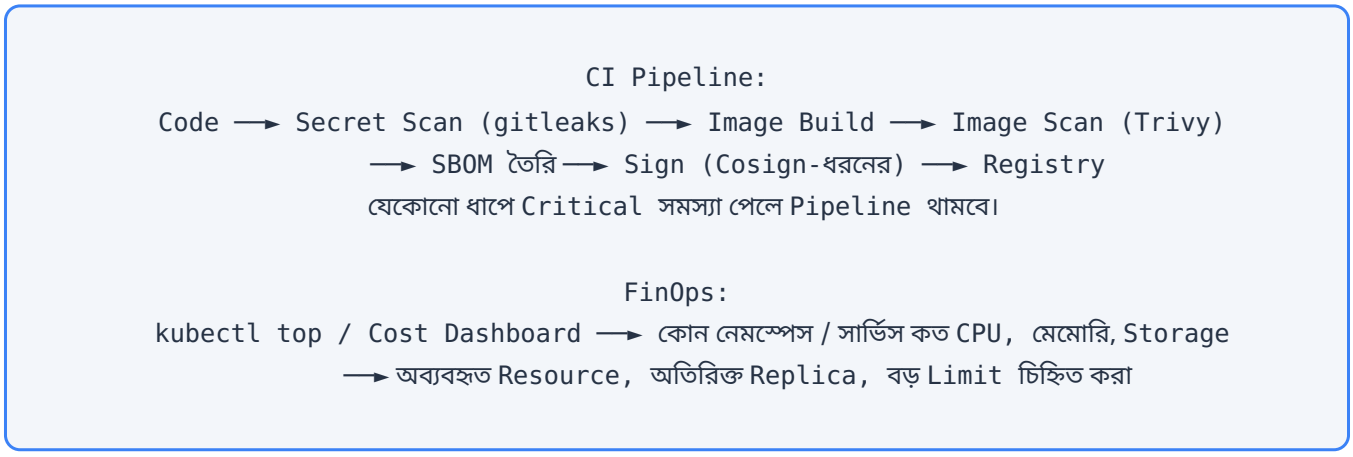
২৩.২ পূর্বশর্ত

Lab 9, Lab 19, Lab 20; Docker ইমেজের ধারণা।

২৩.৩ প্রয়োজনীয় অবকাঠামো

লোকাল মেশিন; Trivy (অথবা সমতুল্য স্ক্যানার); kind ক্লাস্টার। সময় ৩–৪ ঘণ্টা; খরচ-ঝুঁকি **Low**।

২৩.৪ আর্কিটেকচার ডায়াগ্রাম



২৩.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — ইমেজ স্ক্যান করুন।

```
trivy image technova/catalog-service:CHANGE-ME-TAG
# অথবা লোকাল ইমেজের নাম দিন
# ফলাফলে Critical ও High দুর্বলতার সংখ্যা নোট করুন
```

ধাপ ২ — SBOM তৈরি করুন।

```
trivy image --format cyclonedx --output sbom.json technova/catalog-service:CHANGE-ME-TA
ls -lh sbom.json
# SBOM হলো ইমেজের ভেতরে কী কী প্যাকেজ আছে — তার তালিকা; ঘটনার সময় এটি অমূল্য
```

ধাপ ৩ — Secret Scan করুন।

```
# Repo-তে গোপন তথ্য আছে কি না পরীক্ষা করুন
# gitleaks বা সমতুল্য টুল; অথবা grep দিয়ে প্রাথমিক পরীক্ষা:
grep -r "CHANGE-ME" --include="*.yaml" --include="*.tf" . | head -20
# আসল পাসওয়ার্ড / কী-ধরনের স্ট্রিং আছে কি না — ম্যানুয়ালি পরীক্ষা করুন
```

ধাপ ৪ — Cost পর্যালোচনা করুন।

```
kubectl top pods -A
kubectl top nodes
kubectl get pods -A --field-selector=status.phase=Running | wc -l
# কোন নেমস্পেস সবচেয়ে বেশি CPU / মেমোরি ব্যবহার করে — তা লিখুন
# কোন Pod-এর Request-এর তুলনায় ব্যবহার অনেক কম — তা চিহ্নিত করুন
```

ধাপ ৫ — Governance রিপোর্ট লিখুন। একটি পৃষ্ঠায় লিখুন: পাওয়া দুর্বলতা, তাদের তীব্রতা, ঠিক করার পরিকল্পনা; এবং খরচ কমানোর তিনটি উপায় (যেমন অব্যবহৃত Replica কমানো, Limit ঠিক করা, অপ্রয়োজনীয় নেমস্পেস মুছে ফেলা)।

২৩.৬ প্রত্যাশিত আউটপুট

Scan-এর ফলাফল, SBOM ফাইল, Secret Scan-এর ফল ও Cost পর্যালোচনা — চারটি প্রমাণ।

২৩.৭ যাচাই

```
trivy image --severity CRITICAL,HIGH technova/catalog-service:CHANGE-ME-TAG
cat sbom.json | head -40
kubectl top pods -n technova-app
```

২৩.৮ সমস্যা সমাধান

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Trivy ডেটাবেস ডাউনলোড হয় না	ইন্টারনেট / ক্যাশে সমস্যা	আবার চেষ্টা করুন; অফলাইনে পুরোনো ক্যাশে দিয়ে পরীক্ষা করুন।
অনেক দুর্বলতা দেখায়	Base ইমেজ পুরোনো	Base ইমেজ আপডেট করুন; সব দুর্বলতাই Critical নয় — তীব্রতা দেখুন।

২৩.৯ নিরাপত্তা পর্যালোচনা

এই Lab-এর মূল বার্তা: নিরাপত্তা একটি ধাপ নয় — প্রতিটি ধাপে একটি পরীক্ষা। Image Scan, Secret Scan, Policy — তিনটিই CI-তে স্বয়ংক্রিয় হতে হবে, মানুষের মনে রাখার উপর নির্ভরশীল নয়।

২৩.১০ Cleanup

```
rm -f sbom.json
kind delete cluster --name technova-dev
```

২৩.১১ ডকুমেন্টেশন ডেলিভারেবল

- `docs/security-review.md` — Scan ফল ও সংশোধন-পরিকল্পনা।
- `docs/cost-review.md` — Resource ব্যবহার ও সাশ্রয়ের পরিকল্পনা।
- `docs/lab-23-notes.md`।

২৩.১২ প্রতিফলনমূলক প্রশ্ন

1. একটি Critical দুর্বলতা পেলে কি ডেপ্লয় থামানো উচিত — সবসময়?
2. খরচ কমাতে গিয়ে নির্ভরযোগ্যতা কমে গেলে কে সিদ্ধান্ত নেবে?
3. SBOM ছাড়া একটি নিরাপত্তা ঘটনার তদন্ত কেমন হবে?

Lab ২৪ • Final Production Platform

Lab 24 — Final Production Platform Deployment

২৪.১ শেখার উদ্দেশ্য

এই Lab শেষে আপনি পারবেন — Lab 1–23-এর সবকিছু একসাথে জুড়ে একটি **সম্পূর্ণ Production Platform** লোকালে ডেপ্লয় করতে; **Production Readiness Review** চেকলিস্ট দিয়ে তা যাচাই করতে; এবং একটি **Runbook ও Handover নোট** লিখতে যা দিয়ে অন্য কেউ প্ল্যাটফর্মটি চালাতে পারে।

- GitOps, Gateway, Policy, Observability ও Backup — সব একসাথে চালানো।
- End-to-End পরীক্ষা: নতুন সার্ভিস তৈরি থেকে প্রোডাকশন ডেপ্লয় পর্যন্ত।
- Production Readiness-এর প্রমাণ সংগ্রহ করা।

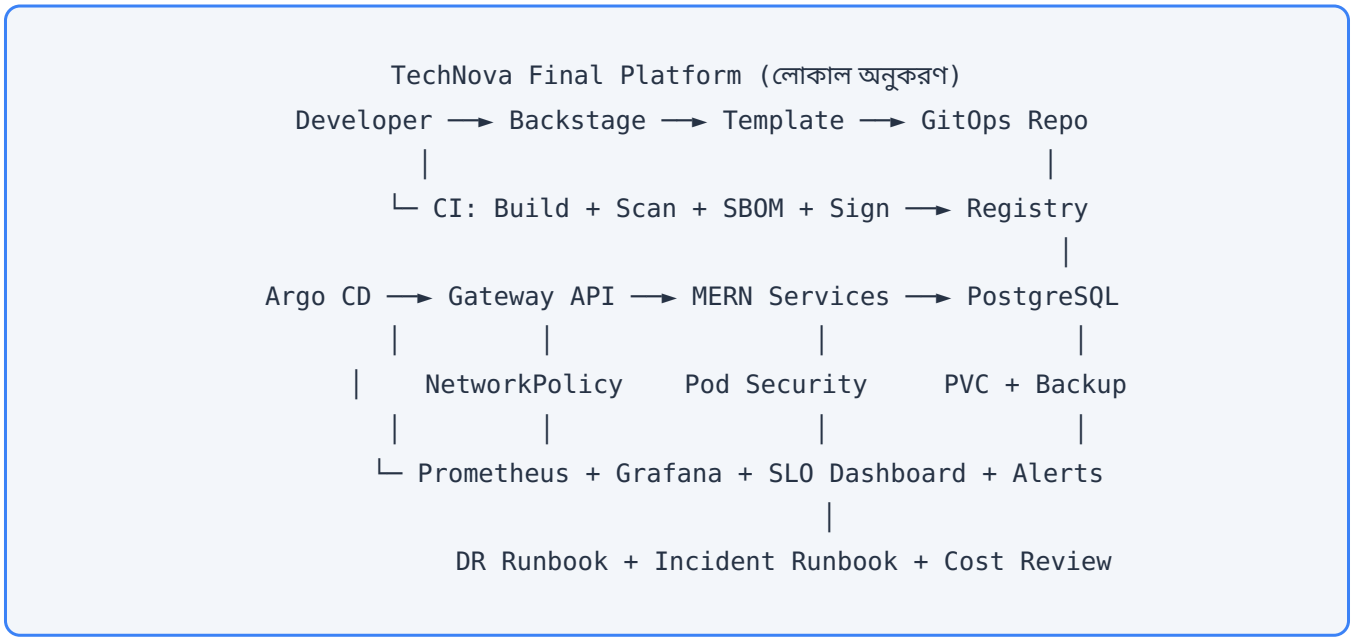
২৪.২ পূর্বশর্ত

Lab 1–23 সম্পূর্ণ; সব নথি ও কনফিগারেশন সংরক্ষিত।

২৪.৩ প্রয়োজনীয় অবকাঠামো

লোকাল kind ক্লাস্টার (বা Minikube); Argo CD; Prometheus + Grafana; PostgreSQL StatefulSet। সময় ৬–৮ ঘণ্টা (দুই দিনে ভাগ করে করুন); খরচ-ঝুঁকি **Low** (লোকাল)। **সতর্কতা:** এই Lab-এর "প্রোডাকশন" মানে লোকাল অনুকরণ — আসল প্রোডাকশনে ডেপ্লয় করার আগে Capstone-এর Production Readiness Review সম্পূর্ণ করুন। আসল ক্লাউডে ডেপ্লয় করলে খরচ হতে পারে।

২৪.৪ আর্কিটেকচার ডায়াগ্রাম



২৪.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — ক্লাস্টার ও ভিত্তি তৈরি করুন। Lab 6-এর kind কনফিগারেশন দিয়ে ক্লাস্টার বানান; নেমস্পেস, Quota ও Pod Security Label বসান।

```
kind create cluster --name technova-prod-sim --config kind-config.yaml
kubectl create namespace technova-app
kubectl create namespace observability
kubectl create namespace argocd
kubectl label namespace technova-app pod-security.kubernetes.io/enforce=restricted
```

ধাপ ২ — GitOps ও Gateway বসান। Lab 12-এর মতো Argo CD ইনস্টল করুন; Lab 3 ও 8-এর Gateway ও NetworkPolicy Apply করুন।

ধাপ ৩ — ডেটা স্তর ও অ্যাপ ডেপ্লয় করুন। Lab 10-এর PostgreSQL StatefulSet ও Lab 14-এর Helm Chart দিয়ে catalog-service ও order-service ডেপ্লয় করুন।

ধাপ ৪ — Observability ও SLO বসান। Lab 20-এর Prometheus + Grafana ও Lab 21-এর SLO Dashboard বসান; একটি Alert নিয়ম যাচাই করুন।

ধাপ ৫ — End-to-End পরীক্ষা করুন। একটি নতুন অনুরোধ Gateway দিয়ে Catalog Service পর্যন্ত যায় কি না; Order তৈরি হয় কি না; ডেটাবেসে ডেটা জমা হয় কি না — পুরো পথটি পরীক্ষা করুন। তারপর একটি Pod মুছে দেখুন সার্ভিস টিকে থাকে কি না।

```
kubectl get pods -A
kubectl get application -n argocd
kubectl get gateway -n technova-app
kubectl get pvc -n technova-app
kubectl get hpa -n technova-app
kubectl get networkpolicy -n technova-app
# প্রতিটি রিসোর্সের Status লিখে রাখুন — এটাই Readiness-এর প্রমাণ
```

ধাপ ৬ — Production Readiness Review করুন। নিচের চেকলিস্টের প্রতিটি আইটেমের পাশে প্রমাণ লিখুন — “হ্যাঁ” নয়, প্রমাণসহ।

বিভাগ	প্রশ্ন	প্রমাণ
ডেপ্লয়	Git থেকে ডেপ্লয় হয় কি? Rollback করা যায় কি?	Argo CD Sync Status; Rollback পরীক্ষার নোট
নিরাপত্তা	Policy প্রয়োগ হয় কি? Secret Git-এ নেই তো?	Pod Security Label; Secret Scan ফল
পর্যবেক্ষণ	Golden Signal দেখা যায় কি? Alert বাজে কি?	Grafana Dashboard; Alert নিয়ম
ডেটা	Backup নেওয়া হয়েছে কি? Restore পরীক্ষা হয়েছে কি?	Backup ফাইল; Restore নোট
খরচ	Resource Limit বসানো আছে কি?	Quota ও Limit তালিকা
নথি	Runbook আছে কি? Handover নোট আছে কি?	Runbook ফাইল

২৪.৬ প্রত্যাশিত আউটপুট

সব সার্ভিস Healthy; Gateway দিয়ে End-to-End অনুরোধ সফল; Readiness চেকলিস্টের প্রতিটি আইটেমে প্রমাণ লেখা আছে।

২৪.৭ যাচাই

```
kubectl get pods -A | grep -v Running
# কোনো Pod Running নয় — এমন লাইন থাকা উচিত নয়
kubectl get application -n argocd -o jsonpath='{.items[*].status.sync.status}'
kubectl get application -n argocd -o jsonpath='{.items[*].status.health.status}'
```

২৪.৮ সমস্যা সমাধান

লক্ষণ	সম্ভাব্য কারণ	করণীয়
একটি স্তর কাজ করে, অন্যটি নয়	Lab-গুলোর কনফিগারেশন মেলেনি	Lab 1–23-এর নোট খুলে পার্থক্য খুঁজুন; একবারে সব নয়, স্তরে স্তরে যাচাই করুন।
ক্লাস্টার ধীর / Pod Pending	RAM কম	অপ্রয়োজনীয় অ্যাপ বন্ধ করুন; Observability-র Retention কমান।

২৪.৯ নিরাপত্তা পর্যালোচনা

এই Lab-এর শেষে পুরো প্ল্যাটফর্মের একটি Security Review করুন: কোন Port বাইরে খোলা, কোন ServiceAccount-এর অতিরিক্ত অনুমতি, কোন Secret কোথায় আছে — তার তালিকা বানান।

২৪.১০ Cleanup

```
# প্রথমে Backup নিন, তারপর মুছুন
kubectl delete namespace technova-app observability argocd
kind delete cluster --name technova-prod-sim
# সব নথি ও কনফিগারেশন Git-এ Commit করুন — এটাই এই Lab-এর আসল ফল
```

২৪.১১ ডকুমেন্টেশন ডেলিভারেবল

- `docs/production-readiness.md` — চেকলিস্ট ও প্রমাণ।
- `docs/runbooks/` — ডেপ্লয়, পুনরুদ্ধার ও ঘটনা-পরিচালনার Runbook।
- `docs/handover.md` — নতুন প্রকৌশলীর জন্য Handover নোট।
- `docs/lab-24-notes.md`।

২৪.১২ প্রতিফলনমূলক প্রশ্ন

1. আপনার প্ল্যাটফর্মের সবচেয়ে দুর্বল স্তর কোনটি — আর তা শক্ত করতে পরবর্তী পদক্ষেপ কী?

2. এই প্ল্যাটফর্মটি আসল প্রোডাকশনে নিতে হলে প্রথম তিনটি কাজ কী হবে?

3. Lab 1-এর আর্কিটেকচারের সাথে Lab 24-এর বাস্তব প্ল্যাটফর্মের পার্থক্য কোথায় — আর কেন?

> **CONCEPT:** Production মানে “চলছে” নয় — মানে “ভাঙলে সারানো যায়, বদলালে পরীক্ষা করা যায়, আর কেউ চলে গেলেও চালানো যায়”। Lab 24-এর Readiness Review সেই তিনটি প্রশ্নের উত্তর খোঁজে।

Assignment ১ • Architecture & ADR

Assignment 1 — TechNova Production Architecture & ADR Set

সংযুক্ত অধ্যায়: Production Architecture Design (Lab 1)। **সময়:** ৩–৪ ঘণ্টা। **খরচ-ঝুঁকি:** Low।

১.১ কাজ

TechNova-র MERN অ্যাপ্লিকেশনের জন্য একটি সম্পূর্ণ Production Architecture নথি লিখুন: Non-Functional Requirement, উপাদান-তালিকা, ডেটা প্রবাহ, ব্যর্থতার বিশ্লেষণ ও অন্তত তিনটি ADR (Kubernetes, ডেটাবেস স্থাপনা, API Gateway)। প্রতিটি ADR-এ Context, Decision, Alternatives ও Consequences থাকতে হবে।

১.২ ডেলিভারেবল

- `docs/architecture.md` — ডায়াগ্রামসহ পূর্ণ নথি।
- `docs/adr/ADR-001` থেকে `ADR-003`।
- SPOF তালিকা ও প্রতিটির প্রতিকার-পরিকল্পনা।

১.৩ মূল্যায়ন মানদণ্ড

মানদণ্ড	ভালো কাজের লক্ষণ
সিদ্ধান্তের যুক্তি	প্রতিটি সিদ্ধান্তের “কেন নয়” অংশটি স্পষ্ট
ব্যর্থতা-চিন্তা	অন্তত তিনটি SPOF চিহ্নিত ও প্রতিকার লেখা
বাস্তবতা	সংখ্যা অনুমান হলে “অনুমান” লেখা; ভুয়া নিশ্চয়তা নেই

Assignment ২ • Distributed Services

Assignment 2 — MERN Service Boundaries & API Contracts

সংযুক্ত অধ্যায়: Distributed-Service Design (Lab 2)। **সময়:** ৩ ঘণ্টা। **খরচ-ঝুঁকি:** Low।

২.১ কাজ

TechNova-র Monolith-কে চারটি সার্ভিসে ভাগ করুন। প্রতিটি সার্ভিসের জন্য API Contract (Endpoint, Request/Response, Error), ডেটা মালিকানা, নির্ভরতা ও ব্যর্থতার প্রভাব লিখুন। একটি সার্ভিসের ব্যর্থতায় কোন ফিচার থামবে, কোনটি চলবে — তার ম্যাট্রিক্স বানান।

২.২ ডেলিভারেবল

- `docs/services.md` — সার্ভিস-তালিকা ও Contract।
- ব্যর্থতা-প্রভাব ম্যাট্রিক্স (টেবিল)।
- ADR: Modular Monolith বনাম সরাসরি Microservices।

২.৩ মূল্যায়ন মানদণ্ড

মানদণ্ড	ভালো কাজের লক্ষণ
সীমা	কোনো সার্ভিস অন্যের DB সরাসরি পড়ে না
চুক্তি	Error-সহ Contract সম্পূর্ণ
বাস্তবতা	সার্ভিস সংখ্যা যুক্তিসঙ্গত (৩–৫টি)

Assignment ৩ • Gateway & Networking

Assignment 3 — Gateway Routing & Network Policy Design

সংযুক্ত অধ্যায়: API Gateway, Gateway API, Advanced Networking (Lab 3, Lab 8)। **সময়:** ৪ ঘণ্টা। **খরচ-ঝুঁকি:** Low (লোকাল)।

৩.১ কাজ

লোকাল kind ক্লাস্টারে Gateway API (অথবা Ingress) দিয়ে `app.technova.example` ও `api.technova.example`-এর রুট বানান; Header-ভিত্তিক একটি রুট ও Weight-ভিত্তিক একটি রুট যোগ করুন; একটি NetworkPolicy লিখে তা পরীক্ষা করুন — অনুমোদিত Pod থেকে সংযোগ যায়, অননুমোদিত Pod থেকে যায় না।

৩.২ ডেলিভারেবল

- `k8s/gateway/` — Gateway, HTTPRoute ও NetworkPolicy YAML।
- পরীক্ষার লগ: curl-এর ফলাফল (সফল ও ব্যর্থ দুটোই)।
- `docs/lab-03-08-notes.md` — রুট-তালিকা।

৩.৩ মূল্যায়ন মানদণ্ড

মানদণ্ড	ভালো কাজের লক্ষণ
রুটিং	সঠিক হোস্ট/পাথে সঠিক সার্ভিস; ভুল পাথে 404
নীতি	NetworkPolicy পরীক্ষা করে প্রমাণিত
নিরাপত্তা	সার্ভিস ClusterIP; বাইরে শুধু Gateway

Assignment 8 • Resilience

Assignment 4 — Load Test Report & Resilience Plan

সংযুক্ত অধ্যায়: Load Testing & Resilience (Lab 5)। **সময়:** ৪ ঘণ্টা। **খরচ-ঝুঁকি:** Low (শুধু লোকাল)।

৪.১ কাজ

একটি লোকাল সার্ভিসে k6 দিয়ে Baseline ও চাপের দুটি রান চালান; p95 Latency, Throughput ও Error Rate-এর টেবিল বানান; HPA বসিয়ে Autoscaling পরীক্ষা করুন; একটি Pod মুছে Resilience পরীক্ষা করুন। ফলাফলের ভিত্তিতে Bottleneck ও পরবর্তী পদক্ষেপ লিখুন।

৪.২ ডেলিভারেবল

- `load-test.js` ও ফলাফলের টেবিল।
- HPA YAML ও Autoscaling-এর পর্যবেক্ষণ।
- `docs/resilience-report.md`।

৪.৩ মূল্যায়ন মানদণ্ড

মানদণ্ড	ভালো কাজের লক্ষণ
পরিমাপ	Baseline-এর সাথে তুলনা আছে; p95 ব্যবহার হয়েছে
সততা	লোকাল ফলকে প্রোডাকশন ক্ষমতা বলে দাবি করা হয়নি
পরিকল্পনা	Bottleneck-এর জন্য নির্দিষ্ট পরবর্তী পদক্ষেপ আছে

Assignment ৫ • Kubernetes Platform

Assignment 5 — Multi-Node Cluster & Multi-Cluster Setup

সংযুক্ত অধ্যায়: Advanced Kubernetes, Multi-Cluster (Lab 6, Lab 7)। **সময়:** ৪ ঘণ্টা। **খরচ-ঝুঁকি:** Low (লোকাল)।

৫.১ কাজ

kind কনফিগারেশন ফাইল দিয়ে বহু-নোড ক্লাস্টার বানান; Label, Taint ও ResourceQuota প্রয়োগ করুন; দুটি ক্লাস্টার (dev ও staging) তৈরি করে Kustomize Overlay দিয়ে একই অ্যাপের ভিন্ন সংস্করণ ডেপ্লয় করুন; Context-ভিত্তিক ভুল এড়ানোর চেকলিস্ট লিখুন।

৫.২ ডেলিভারেবল

- `kind-config.yaml`, `quota.yaml`।
- `k8s/base/` ও `k8s/overlays/dev|staging/`।
- ক্লাস্টার-তুলনা টেবিল।

৫.৩ মূল্যায়ন মানদণ্ড

মানদণ্ড	ভালো কাজের লক্ষণ
পুনরাবৃত্তি	ক্লাস্টার কনফিগারেশন ফাইল থেকে আবার বানানো যায়
সীমা	Quota ভাঙার চেষ্টা করে তা প্রমাণিত
নিরাপত্তা	kubeconfig Git-এ নেই; Context যাচাইয়ের অভ্যাস লেখা

Assignment ৬ • Stateful & Storage

Assignment 6 — StatefulSet, PVC & Backup-Restore Proof

সংযুক্ত অধ্যায়: Stateful Workloads (Lab 10)। **সময়:** ৩ ঘণ্টা। **খরচ-ঝুঁকি:** Low।

৬.১ কাজ

PostgreSQL StatefulSet ও PVC দিয়ে ডেটাবেস চালান; ডেটা লিখে Pod মুছে ডেটা টিকে থাকে — তা প্রমাণ করুন; `pg_dump` দিয়ে Backup নিয়ে তা থেকে Restore করুন; Backup ও Restore-এর সময় নোট করুন।

৬.২ ডেলিভারেবল

- `k8s/stateful/postgres-statefulset.yaml`।
- Backup ও Restore পরীক্ষার লগ।
- `docs/runbooks/database-restore.md`।

৬.৩ মূল্যায়ন মানদণ্ড

মানদণ্ড	ভালো কাজের লক্ষণ
স্থায়িত্ব	Pod পুনরায় তৈরি হওয়ার পর ডেটা অক্ষত
প্রমাণ	Restore পরীক্ষা করা হয়েছে, শুধু Backup নয়
নিরাপত্তা	পাসওয়ার্ড Secret-এ; ডেটাবেস বাইরে খোলা নয়

Assignment ৭ • GitOps

Assignment 7 — GitOps Repository & Argo CD Sync Proof

সংযুক্ত অধ্যায়: GitOps, Argo CD, Flux CD (Lab 11–13)। **সময়:** ৫ ঘণ্টা। **খরচ-ঝুঁকি:** Low (লোকাল)।

৭.১ কাজ

একটি GitOps Repo বানান (Base + Overlay, App-of-Apps); লোকালে Argo CD ইনস্টল করে একটি Application Sync করুন; Git-এ Replica বদলে Sync প্রমাণ করুন; ক্লাস্টারে হাতে বদল করে Self-Heal প্রমাণ করুন; Flux-এর GitRepository ও Kustomization ফাইল লিখে Argo CD বনাম Flux-এর তুলনা লিখুন।

৭.২ ডেলিভারেবল

- GitOps Repo-র কাঠামো ও README (নিয়মসহ)।
- `gitops/argocd-app.yaml` ও `gitops/flux-*.yaml`।
- Sync ও Self-Heal পরীক্ষার লগ; তুলনা-নোট।

৭.৩ মূল্যায়ন মানদণ্ড

মানদণ্ড	ভালো কাজের লক্ষণ
Source of Truth	Git-এ বদল ছাড়া ক্লাস্টারে বদল নেই
নিরাপত্তা	Secret Git-এ নেই; Admin পাসওয়ার্ড বদলানো হয়েছে
তুলনা	Argo CD বনাম Flux-এর পার্থক্য নিজের ভাষায় লেখা

Assignment ৮ • Helm & Kustomize

Assignment 8 — Helm Chart & Release Management

সংযুক্ত অধ্যায়: Helm & Kustomize (Lab 14)। **সময়:** ৩–৪ ঘণ্টা। **খরচ-ঝুঁকি:** Low।

৮.১ কাজ

একটি সার্ভিসের জন্য Helm Chart বানান (Chart.yaml, values.yaml, values-staging.yaml, templates/); `helm lint` ও `helm template` দিয়ে যাচাই করুন; Install → Upgrade → Rollback করুন ও History সংরক্ষণ করুন; একই অ্যাপ Kustomize দিয়ে ডেপ্লয় করে Helm বনাম Kustomize-এর সিদ্ধান্ত-নোট লিখুন।

৮.২ ডেলিভারেবল

- `charts/catalog-service/` — সম্পূর্ণ Chart।
- `helm history`-এর আউটপুট।
- `docs/lab-14-notes.md` — তুলনা।

৮.৩ মূল্যায়ন মানদণ্ড

মানদণ্ড	ভালো কাজের লক্ষণ
Chart	Lint পাস; Values দিয়ে পরিবেশভেদ হয়
Release	Rollback প্রমাণিত; History সংরক্ষিত
নিরাপত্তা	Values-এ Secret নেই

Assignment ৯ • Developer Platform

Assignment 9 — Backstage Catalog & Self-Service Template

সংযুক্ত অধ্যায়: Backstage, Self-Service Template (Lab 15–16)। **সময়:** ৫ ঘণ্টা। **খরচ-ঝুঁকি:** Low।

৯.১ কাজ

লোকালে Backstage চালিয়ে Catalog-এ অন্তত তিনটি সার্ভিস নিবন্ধন করুন (catalog-info.yaml-সহ); প্রতিটির Owner, Lifecycle ও System লিখুন; একটি Software Template লিখুন যা দিয়ে নতুন Node.js সার্ভিসের Skeleton Repo তৈরি হয় (Dockerfile, CI, k8s Manifest, catalog-info.yaml); Golden Path-এর নিয়ম-তালিকা লিখুন।

৯.২ ডেলিভারেবল

- তিনটি `catalog-info.yaml`।
- `templates/node-service/template.yaml` ও Skeleton।
- TechDocs-এর খসড়া (প্রতিটি সার্ভিসের `docs/index.md`)।

৯.৩ মূল্যায়ন মানদণ্ড

মানদণ্ড	ভালো কাজের লক্ষণ
Catalog	Owner ও Lifecycle ছাড়া কোনো সার্ভিস নেই
Template	নতুন সার্ভিসে মানসম্মত কাঠামো আসে
ডকুমেন্টেশন	TechDocs-এ “কীভাবে চালাতে হয়” লেখা আছে

Assignment ১০ • Progressive Delivery

Assignment 10 — Canary Release Plan & Rollback Proof

সংযুক্ত অধ্যায়: Progressive Delivery (Lab 17)। **সময়:** ৪ ঘণ্টা। **খরচ-ঝুঁকি:** Low।

১০.১ কাজ

catalog-service-এর জন্য একটি Canary পরিকল্পনা লিখুন: ধাপ (১০% → ৫০% → ১০০%), প্রতিটি ধাপের মানদণ্ড (Error Rate, p95 Latency), Pause-এর সময় ও Rollback-এর শর্ত। লোকালে Argo Rollouts বা Gateway Weight দিয়ে Canary চালান; একটি ভাঙা সংস্করণ দিয়ে Rollback প্রমাণ করুন।

১০.২ ডেলিভারেবল

- `k8s/rollouts/catalog-rollout.yaml`।
- Canary পরিকল্পনা ও মানদণ্ড (`docs/canary-plan.md`)।
- Rollback পরীক্ষার লগ।

১০.৩ মূল্যায়ন মানদণ্ড

মানদণ্ড	ভালো কাজের লক্ষণ
মানদণ্ড	সংখ্যাসহ স্পষ্ট — “ভালো লাগলে” নয়
Rollback	খারাপ সংস্করণে ফেরা প্রমাণিত
ডেটাবেস	Schema বদলের ঝুঁকি আলাদা করে লেখা আছে

Assignment ১১ • Automation

Assignment 11 — Terraform + Ansible Automation Pair

সংযুক্ত অধ্যায়: Terraform, Ansible, Crossplane (Lab 18)। **সময়:** ৪ ঘণ্টা। **খরচ-ঝুঁকি:** Low (শুধু লোকাল Docker Provider)।

১১.১ কাজ

Terraform দিয়ে লোকাল Docker কনটেইনার বানান (Plan → Apply → Destroy); Ansible Playbook দিয়ে একটি মেশিনের কনফিগারেশন করুন ও দ্বিতীয়বার চালিয়ে Idempotence প্রমাণ করুন; Crossplane Claim-এর খসড়া লিখুন; "কোন কাজ কোন টুলের" — এই সিদ্ধান্ত-তালিকা লিখুন। আসল ক্লাউডে Apply করবেন না।

১১.২ ডেলিভারেবল

- `terraform/main.tf` ও Plan/Apply/Destroy লগ।
- `ansible/playbook.yaml` ও Idempotence প্রমাণ।
- `crossplane/claim.yaml` ও সিদ্ধান্ত-তালিকা।

১১.৩ মূল্যায়ন মানদণ্ড

মানদণ্ড	ভালো কাজের লক্ষণ
সীমা	Terraform দিয়ে কনফিগারেশন, Ansible দিয়ে Provisioning — এমন ভুল নেই
নিরাপত্তা	State ফাইল Git-এ নেই; Playbook-এ পাসওয়ার্ড নেই
খরচ	আসল ক্লাউড রিসোর্স তৈরি হয়নি

Assignment ১২ • Governance

Assignment 12 — Multi-Cloud Governance Pack

সংযুক্ত অধ্যায়: Multi-Cloud Governance (Lab 19)। **সময়:** ৩ ঘণ্টা। **খরচ-ঝুঁকি:** Low।

১২.১ কাজ

TechNova-র জন্য একটি Governance Pack লিখুন: Naming মান, বাধ্যতামূলক Tag-তালিকা, পরিবেশভেদে Terraform Variable কাঠামো, Lock-in পর্যালোচনা (প্রতিটি ক্লাউড-নির্দিষ্ট সার্ভিসের বিকল্প ও Exit Plan) ও নতুন পরিবেশ চালু করার ১০-প্রশ্নের চেকলিস্ট।

১২.২ ডেলিভারেবল

- `docs/governance.md`।
- `terraform/variables.tf`।
- Lock-in পর্যালোচনা টেবিল।

১২.৩ মূল্যায়ন মানদণ্ড

মানদণ্ড	ভালো কাজের লক্ষণ
একরূপতা	নীতি সব পরিবেশে একই; বদলায় শুধু Variable
Exit Plan	অন্তত একটি সার্ভিসের বের হওয়ার পথ লেখা আছে
বাস্তবতা	Multi-Cloud-এর খরচ (সময়) স্বীকার করা হয়েছে

Assignment ১৩ • Observability & SLO

Assignment 13 — Observability Dashboard & SLO Document

সংযুক্ত অধ্যায়: Observability, SLO (Lab 20–21)। **সময়:** ৫ ঘণ্টা। **খরচ-ঝুঁকি:** Low (লোকাল)।

১৩.১ কাজ

লোকালে Prometheus + Grafana চালিয়ে Golden Signal Dashboard বানান; catalog-service-এর জন্য SLI, SLO ও Error Budget লিখুন (`docs/slo.md`); একটি Alert নিয়ম লিখুন যার সাথে Runbook-এর লিংক আছে; Metrics, Logs ও Trace দিয়ে একটি ভুল অনুরোধের গল্প পড়ে নোট লিখুন।

১৩.২ ডেলিভারেবল

- Grafana Dashboard (JSON রপ্তানি)।
- `docs/slo.md` — SLI, SLO, Error Budget ও নীতি।
- `observability/alert-rule.yaml` ।

১৩.৩ মূল্যায়ন মানদণ্ড

মানদণ্ড	ভালো কাজের লক্ষণ
SLO	ব্যবহারকারীর অভিজ্ঞতা প্রতিফলিত করে; ১০০% নয়
Alert	প্রতিটি Alert-এর Runbook লিংক আছে
Dashboard	Golden Signal চারটিই দেখা যায়

Assignment ১৪ • Disaster Recovery

Assignment 14 — Disaster Recovery Plan & Simulation Report

সংযুক্ত অধ্যায়: Disaster Recovery (Lab 22)। **সময়:** ৪ ঘণ্টা। **খরচ-ঝুঁকি:** Low (শুধু লোকাল পরীক্ষামূলক ডেটা)।

১৪.১ কাজ

TechNova-র জন্য DR Plan লিখুন (RTO, RPO, Failover ধাপ, যোগাযোগ-তালিকা); লোকালে ডেটাবেস বন্ধ করে Failover অনুকরণ করুন; Backup থেকে Restore করে সময় মাপুন; RTO/RPO-র সাথে মাপা সময় তুলনা করুন; DR Runbook লিখুন।

১৪.২ ডেলিভারেবল

- `docs/dr-plan.md`।
- `docs/runbooks/database-restore.md`।
- Simulation রিপোর্ট — মাপা সময়সহ।

১৪.৩ মূল্যায়ন মানদণ্ড

মানদণ্ড	ভালো কাজের লক্ষণ
প্রমাণ	Restore পরীক্ষা করা হয়েছে
সময়	RTO/RPO-র সাথে মাপা সময় তুলনা করা হয়েছে
নিরাপত্তা	Backup এনক্রিপ্ট করা; আসল ডেটা ব্যবহার হয়নি

Assignment ১৫ • Security & Cost

Assignment 15 — Security & Cost Governance Review

সংযুক্ত অধ্যায়: Platform Security & Cost Governance (Lab 23)। **সময়:** ৪ ঘণ্টা। **খরচ-ঝুঁকি:** Low।

১৫.১ কাজ

আপনার Lab 24-এর প্ল্যাটফর্মের উপর একটি Security ও Cost Review করুন: ইমেজ স্ক্যান (Trivy), SBOM তৈরি, Secret Scan, Policy পরীক্ষা (Pod Security, RBAC, NetworkPolicy) ও Resource ব্যবহারের পর্যালোচনা। Critical সমস্যার সংশোধন-পরিকল্পনা ও খরচ কমানোর তিনটি উপায় লিখুন।

১৫.২ ডেলিভারেবল

- `docs/security-review.md` — Scan ফল ও সংশোধন-পরিকল্পনা।
- `docs/cost-review.md` — Resource ব্যবহার ও সাশ্রয়ের পরিকল্পনা।
- `sbom.json` (নমুনা ইমেজের)।

১৫.৩ মূল্যায়ন মানদণ্ড

মানদণ্ড	ভালো কাজের লক্ষণ
প্রমাণ	Scan-এর আসল ফলাফল সংযুক্ত, অনুমান নয়
অগ্রাধিকার	Critical আগে, Low পরে — সাজানো আছে
ভারসাম্য	খরচ কমানোর প্রস্তাবে নির্ভরযোগ্যতার ঝুঁকি লেখা আছে

> **CONCEPT:** Assignment-গুলো Lab-এর প্রমাণ। Lab-এ আপনি ধাপ অনুসরণ করেছেন; Assignment-এ আপনি নিজে সিদ্ধান্ত নিয়ে তার প্রমাণ দিয়েছেন। পোর্টফোলিওতে Lab-এর নোট নয়, Assignment-এর ডেলিভারেবল দেখান।

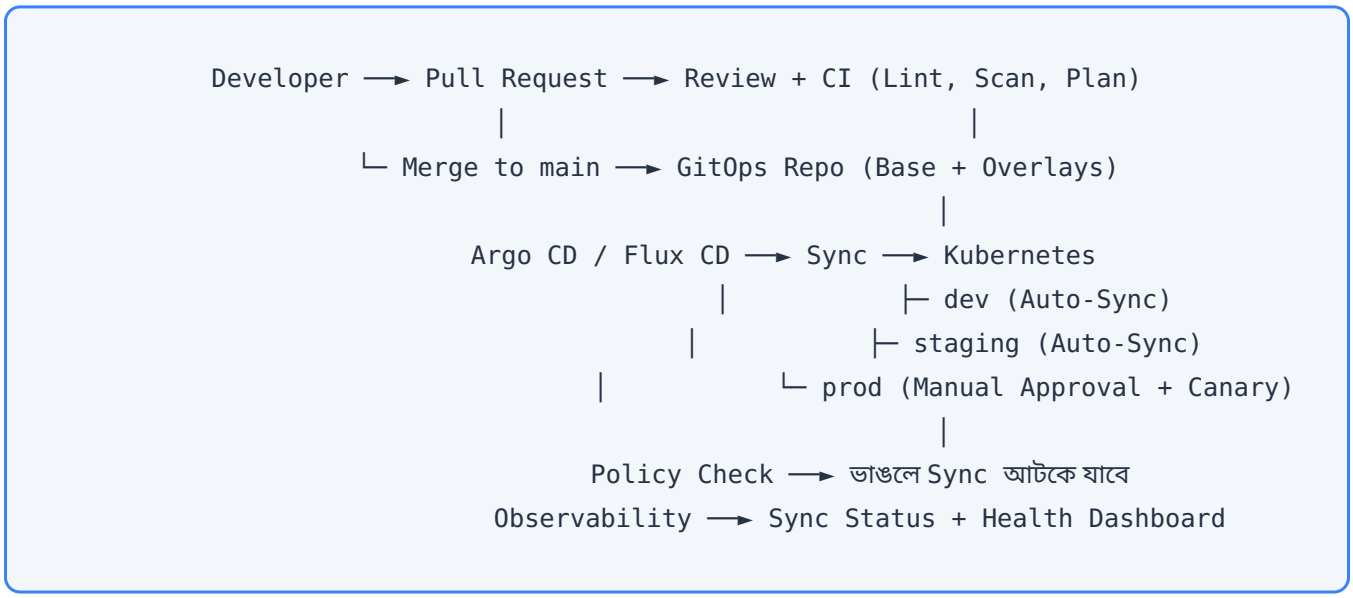
Project ০১ • Enterprise GitOps Deployment Platform

Project 01 — Enterprise GitOps Deployment Platform

স্ট্যাক: Kubernetes, Argo CD অথবা Flux CD, Helm, Kustomize। **পরিবেশ:** dev / staging / prod। **সময়:** ২–৩ সপ্তাহ (অংশ-সময়ে)। **খরচ-ঝুঁকি:** Low — লোকাল kind/Minikube-এ সম্পূর্ণ করা যায়; আসল ক্লাউডে করলে খরচ হতে পারে, তাই প্রথমে লোকালেই সম্পূর্ণ করুন।

১.১ প্রকল্পের বিবরণ

TechNova-র জন্য একটি Enterprise GitOps Deployment Platform তৈরি করুন — যেখানে প্রতিটি ডেপ্লয় Git-এর Commit থেকে আসে, প্রতিটি পরিবেশের কনফিগারেশন Git-এ থাকে, Sync স্বয়ংক্রিয়ভাবে পর্যবেক্ষিত হয়, ভুল হলে Rollback করা যায়, এবং Policy ভাঙা ডেপ্লয় আটকে যায়। এই প্রকল্পের শেষে আপনার হাতে থাকবে একটি চলমান প্ল্যাটফর্ম, একটি GitOps Repo, এবং একটি অপারেশনাল নথি — যা একজন নতুন প্রকৌশলী পড়ে প্ল্যাটফর্মটি চালাতে পারবেন।



১.২ পেশাদার প্রয়োজনীয়তা (Requirements)

আইডি	প্রয়োজনীয়তা	গ্রহণযোগ্যতা
REQ-01	সব ডেপ্লয় Git থেকে হবে; ক্লাস্টারে হাতে Apply নিষিদ্ধ	Git-এ Commit ছাড়া ক্লাস্টারে কোনো পরিবর্তন আসে না — পরীক্ষা করে প্রমাণিত
REQ-02	তিনটি পরিবেশ: dev, staging, prod — একই Base, ভিন্ন Overlay	প্রতিটি পরিবেশে ভিন্ন Replica, Image Tag ও Resource
REQ-03	স্বয়ংক্রিয় Sync ও Self-Heal (dev/staging); prod-এ অনুমোদন	হাতে মুছা Pod ফিরে আসে; prod-এ Sync-এর আগে অনুমোদন লাগে
REQ-04	Rollback — Git Revert দিয়ে আগের সংস্করণে ফেরা যায়	Revert-এর পর ১০ মিনিটের মধ্যে আগের সংস্করণ চলে

REQ-05	Policy Enforcement — Restricted Pod, অনুমোদিত Registry	নীতি-ভাঙা Manifest Sync-এ প্রত্যাখ্যাত হয়
REQ-06	Helm ও Kustomize — দুটোই ব্যবহৃত, কিন্তু এক অ্যাপে একটাই	Chart ও Overlay-র সিদ্ধান্ত-নোট লেখা আছে
REQ-07	ডকুমেন্টেশন — Architecture, Runbook, Handover	নতুন প্রকৌশলী নথি পড়ে ডেপ্লয় করতে পারেন

১.৩ আর্কিটেকচার

GitOps Repo (Monorepo): `apps/<service>/base`, `apps/<service>/overlays/{dev,staging,prod}`, `clusters/{dev,staging,prod}` (App-of-Apps), `platform/` (Argo CD, Policy, Observability)। Argo CD `argocd` নেমস্পেসে; প্রতিটি পরিবেশ আলাদা kind ক্লাস্টার অথবা একই ক্লাস্টারে আলাদা নেমস্পেস (লোকালে আলাদা ক্লাস্টার সুপারিশ করা হয়)। prod-এ Auto-Sync বন্ধ — Manual Sync ও অনুমোদন।

১.৪ বাস্তবায়ন পরিকল্পনা

মাইলফলক	কাজ	সময়
M1	GitOps Repo কাঠামো, Base + Overlay, নিয়ম-নথি	সপ্তাহ ১
M2	Argo CD ইনস্টল, dev Application Sync, Self-Heal পরীক্ষা	সপ্তাহ ১–২
M3	staging ও prod Application; prod-এ Manual Approval	সপ্তাহ ২
M4	Helm Chart, Policy, Rollback পরীক্ষা, নথি ও Handover	সপ্তাহ ২–৩

১.৫ রিপোজিটরি কাঠামো

```
technova-gitops/
├── apps/
│   ├── catalog-service/base/           # Deployment, Service
│   ├── catalog-service/overlays/dev/
│   ├── catalog-service/overlays/staging/
│   └── catalog-service/overlays/prod/
├── clusters/
│   ├── dev/apps.yaml                   # App-of-Apps
│   ├── staging/apps.yaml
│   └── prod/apps.yaml
├── charts/catalog-service/             # Helm Chart (যেখানে Helm ব্যবহৃত)
├── platform/
│   ├── argocd/                         # Argo CD Application
│   ├── policy/                         # Pod Security, NetworkPolicy
```

```
│   └── observability/                    # Alert নিয়ম
└── docs/
    ├── architecture.md
    ├── runbooks/deploy.md
    ├── runbooks/rollback.md
    └── handover.md
```

১.৬ কনফিগারেশন

```
# clusters/prod/apps.yaml — ধারণা (prod-এ Auto-Sync বন্ধ)
# Application: catalog-service-prod
# source: apps/catalog-service/overlays/prod, targetRevision: main
# syncPolicy: manual (automated নয়) — Sync-এর আগে মানুষ অনুমোদন দেবে
# syncOptions: CreateNamespace=true, Prune=true
# প্রতিটি Sync-এর আগে Argo CD UI-তে Diff দেখা বাধ্যতামূলক
```

১.৭ ডেপ্লয়মেন্ট

প্রথমে dev-এ Sync করুন; তারপর staging-এ; প্রতিটি পরিবেশে Health যাচাই করুন; শেষে prod-এ Manual Sync করুন। প্রতিটি ধাপের Sync Status ও Health নোট করুন।

```
kubectl get application -n argocd
kubectl get pods -n technova-app
# dev → staging → prod — এই ক্রমেই যাবে; prod-এ যাওয়ার আগে staging-এ পরীক্ষা বাধ্যতামূলক
```

১.৮ যাচাই (Validation)

- Git-এ Replica বদলে তিনটি পরিবেশে (ধাপে ধাপে) তা আসে কি না।
- ক্লাস্টারে হাতে একটি Pod মুছে Self-Heal হয় কি না।
- Git Revert করে Rollback হয় কি না — সময়সহ।
- নীতি-ভাঙা Manifest (Privileged Pod) Sync-এ আটকায় কি না।

১.৯ নিরাপত্তা মূল্যায়ন

Argo CD-র Admin পাসওয়ার্ড বদলানো হয়েছে; UI ইন্টারনেটে খোলা নয়; RBAC-এ কে Sync করতে পারে তা সীমিত; Secret Git-এ নেই; Image Tag `latest` নয়; Policy প্রতিটি পরিবেশে প্রয়োগ হয়। প্রতিটি আইটেমের প্রমাণ নথিতে লিখুন।

১.১০ খরচ মূল্যায়ন

লোকালে খরচ শূন্য। আসল ক্লাউডে এই প্ল্যাটফর্মের খরচের প্রধান অংশ: ক্লাস্টার, Managed ডেটাবেস ও Observability Storage। খরচ কমানোর উপায়: dev/staging-এ ছোট আকার, রাতে অপ্রয়োজনীয় পরিবেশ বন্ধ করা, Observability Retention কমানো। **সতর্কতা:** আসল ক্লাউডে ডেপ্লয় করার আগে বাজেট সতর্কতা (Alert) বসান।

১.১১ অপারেশনাল ডকুমেন্টেশন

- `docs/runbooks/deploy.md` — নতুন সংস্করণ ডেপ্লয় করার ধাপ।
- `docs/runbooks/rollback.md` — Rollback-এর ধাপ ও সিদ্ধান্ত-বিন্দু।
- `docs/runbooks/sync-failure.md` — Sync ব্যর্থ হলে কী করবেন।
- `docs/handover.md` — নতুন প্রকৌশলীর জন্য।

১.১২ চূড়ান্ত ডেলিভারেবল

- চলমান GitOps প্ল্যাটফর্ম (লোকাল) ও GitOps Repo।
- তিনটি পরিবেশের Sync প্রমাণ (স্ক্রিনশট-বর্ণনা বা লগ)।
- Rollback পরীক্ষার রিপোর্ট।
- Architecture, Runbook ও Handover নথি।
- Security ও Cost মূল্যায়ন।

> **WARNING:** prod-এ Auto-Sync চালু করবেন না — এই প্রকল্পে prod Manual Sync-ই সঠিক সিদ্ধান্ত। Auto-Sync prod-এ সুবিধাজনক মনে হলেও একটি ভুল Commit সরাসরি প্রোডাকশনে যাওয়ার পথ খুলে দেয়।

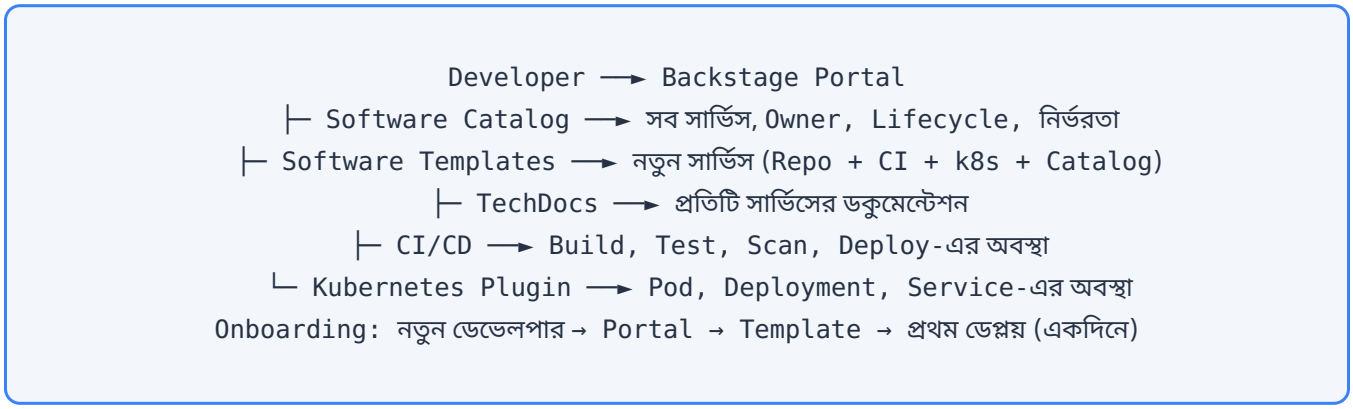
Project ০২ • Internal Developer Platform

Project 02 — Internal Developer Platform

স্ট্যাক: Backstage, Software Catalog, Software Templates, TechDocs, CI/CD, Kubernetes। **সময়:** ৩–৪ সপ্তাহ (অংশ-সময়ে)। **খরচ-ঝুঁকি:** Low — লোকাল Backstage-এ সম্পূর্ণ করা যায়।

২.১ প্রকল্পের বিবরণ

TechNova-র ডেভেলপারদের জন্য একটি Internal Developer Platform (IDP) তৈরি করুন — যেখানে একজন ডেভেলপার Portal-এ গিয়ে নতুন সার্ভিস তৈরি করতে পারেন, বিদ্যমান সার্ভিসের মালিক ও ডকুমেন্টেশন খুঁজে পান, CI/CD-র অবস্থা দেখেন, এবং Kubernetes-এ তাঁর সার্ভিসের অবস্থা জানেন — ক্লাস্টারে সরাসরি ঢুকতে না হয়। এই প্রকল্পের সাফল্য মাপা হবে একটি প্রশ্নে: একজন নতুন ডেভেলপার প্রথম দিনে, সাহায্য ছাড়া, একটি সার্ভিস তৈরি করে Staging-এ ডেপ্লয় করতে পারেন কি?



২.২ পেশাদার প্রয়োজনীয়তা (Requirements)

আইডি	প্রয়োজনীয়তা	গ্রহণযোগ্যতা
REQ-01	Software Catalog-এ অন্তত ৫টি সার্ভিস, প্রতিটির Owner ও Lifecycle	Owner ছাড়া কোনো সার্ভিস নেই
REQ-02	Software Template দিয়ে নতুন সার্ভিস তৈরি হয় (Repo + CI + k8s)	Template থেকে তৈরি সার্ভিস Staging-এ ডেপ্লয় হয়
REQ-03	TechDocs — প্রতিটি সার্ভিসের “কীভাবে চালাতে হয়”	নতুন ডেভেলপার Docs পড়ে সার্ভিস চালাতে পারেন
REQ-04	CI/CD Integration — Portal থেকে Build/Deploy-এর অবস্থা দেখা যায়	ব্যর্থ Build Portal-এ দেখা যায়
REQ-05	Kubernetes Integration — Pod ও Deployment-এর অবস্থা দেখা যায়	Pod-এর Status Portal-এ দেখা যায়
REQ-06	Developer Onboarding — নতুন ডেভেলপারের গাইড	গাইড অনুসরণ করে প্রথম ডেপ্লয় একদিনে হয়
REQ-07	Access Control — কে কী দেখতে ও করতে পারে	Authentication ছাড়া Portal খোলা নয়; ভূমিকা-ভিত্তিক প্রবেশাধিকার লেখা আছে

২.৩ আর্কিটেকচার

Backstage Portal (লোকালে `localhost:3000`); Catalog-এর উৎস Git Repo-র `catalog-info.yaml`; Template-এর Skeleton Repo; CI (GitHub Actions-ধরনের) — Build, Test, Image Scan, SBOM; GitOps Repo-তে Kubernetes Manifest; Argo CD দিয়ে Staging-এ Sync। প্রোডাকশনে Backstage-এর পেছনে ডেটাবেস (PostgreSQL) ও Authentication থাকবে — লোকালে ডিফল্ট সেটআপেই চলবে।

২.৪ বাস্তবায়ন পরিকল্পনা

মাইলফলক	কাজ	সময়
M1	Backstage চালু, Catalog-এ ৫টি সার্ভিস নিবন্ধন, TechDocs	সপ্তাহ ১
M2	Software Template (Node.js ও Python — অন্তত একটি) ও Skeleton	সপ্তাহ ২
M3	CI/CD Integration ও Kubernetes Plugin; Template থেকে Staging ডেপ্লয়	সপ্তাহ ৩
M4	Onboarding গাইড, Access Control, পরীক্ষা ও Handover	সপ্তাহ ৩–৪

২.৫ রিপোজিটরি কাঠামো

```
technova-idp/
├── technova-portal/                # Backstage অ্যাপ (app-config.yaml সহ)
├── catalog/
│   ├── catalog-service/catalog-info.yaml
│   ├── order-service/catalog-info.yaml
│   └── ... (মোট ৫টি সার্ভিস)
├── templates/
│   ├── node-service/template.yaml
│   └── node-service/skeleton/      # Dockerfile, CI, k8s, README
├── docs/
│   ├── onboarding.md               # নতুন ডেভেলপারের গাইড
│   ├── architecture.md
│   └── access-control.md
└── .github/workflows/              # CI — Build, Test, Scan
```

২.৬ কনফিগারেশন

```
# app-config.yaml — ধারণা (আসল Secret এখানে নয়)
# catalog:
#   locations:
#     - type: url
#       target: https://github.com/CHANGE-ME-ORG/technova-idp/blob/main/catalog/all.yam
```

```
# scaffolder:
#   templates: Template-এর অবস্থান
# auth: প্রোডাকশনে GitHub OAuth বা সমতুল্য — লোকালে Guest মোডে পরীক্ষা করুন
```

২.৭ ডেপ্লয়মেন্ট

Backstage লোকালে চালান; Catalog নিবন্ধন করুন; Template থেকে একটি পরীক্ষামূলক সার্ভিস তৈরি করুন; CI চালিয়ে Staging-এ ডেপ্লয় করুন; Portal-এ তার অবস্থা যাচাই করুন।

২.৮ যাচাই (Validation)

- নতুন ডেভেলপারের ভূমিকায় (নিজে, নথি পড়ে) Template থেকে সার্ভিস তৈরি ও Staging ডেপ্লয় — সময় মাপুন।
- Catalog-এ প্রতিটি সার্ভিসের Owner ও Docs আছে কি না।
- CI ব্যর্থ হলে Portal-এ তা দেখা যায় কি না।
- Kubernetes Plugin-এ Pod-এর Status দেখা যায় কি না।

২.৯ নিরাপত্তা মূল্যায়ন

Portal-এ Authentication বাধ্যতামূলক; Template থেকে তৈরি Repo-তে একই নিরাপত্তা মান (non-root, Secret প্লেসহোল্ডার, Policy-সম্মত Manifest); GitHub Token-এর স্কোপ সীমিত; Catalog-এ Secret বা অভ্যন্তরীণ শংসাপত্র নেই।

২.১০ খরচ মূল্যায়ন

লোকালে খরচ শূন্য। প্রোডাকশনে Backstage-এর খরচ: একটি ছোট সার্ভার/কনটেইনার, PostgreSQL ও Storage। খরচ কমানোর উপায়: Backstage-কে ছোট Replica-তে চালানো, TechDocs-এর Storage সীমিত রাখা।

২.১১ অপারেশনাল ডকুমেন্টেশন

- `docs/onboarding.md` — নতুন ডেভেলপারের ধাপে ধাপে গাইড।
- `docs/architecture.md` — Portal, Catalog, Template ও CI-এর সম্পর্ক।
- `docs/access-control.md` — কে কী করতে পারে।
- `docs/runbooks/template-failure.md` — Template ব্যর্থ হলে কী করবেন।

২.১২ চূড়ান্ত ডেলিভারেবল

- চলমান Backstage Portal (লোকাল) — Catalog, Template, TechDocs-সহ।
- Template থেকে তৈরি একটি সার্ভিসের Staging ডেপ্লয় প্রমাণ।
- Onboarding গাইড ও Access Control নথি।
- Architecture ও Runbook।

Project ০৩ • TechNova Enterprise Platform

Project 03 — TechNova Enterprise Platform

স্ট্যাক: Multi-Cluster Kubernetes, GitOps, Internal Developer Platform, Terraform, Ansible, Security Governance, Centralized Observability, SRE, FinOps, Disaster Recovery। **সময়:** ৪–৬ সপ্তাহ (অংশ-সময়ে)। **খরচ-ঝুঁকি:** Low — লোকাল অনুকরণে সম্পূর্ণ করা যায়; আসল মাল্টি-ক্লাউডে করলে উল্লেখযোগ্য খরচ হতে পারে, তাই প্রথমে লোকালেই সম্পূর্ণ করুন।

৩.১ প্রকল্পের বিবরণ

TechNova-র জন্য একটি সম্পূর্ণ Enterprise Platform তৈরি করুন — যা Project 01 (GitOps) ও Project 02 (IDP)-কে একসাথে জুড়ে, তার সাথে Terraform/Ansible দিয়ে অবকাঠামো, নিরাপত্তা শাসন (Governance), কেন্দ্রীয় Observability, SRE (SLO ও Error Budget), FinOps (খরচ-শাসন) ও Disaster Recovery যোগ করে। এটি এই Workbook-এর সবচেয়ে বড় প্রকল্প — এবং Capstone-এর মহড়া।

```
                    TechNova Enterprise Platform
        ┌─ Developer Experience: Backstage (Catalog, Template, TechDocs)
  ├─ Delivery: CI (Build/Scan/SBOM/Sign) → GitOps (Argo CD/Flux) → Multi-Cluster K8s
  ├─ Infrastructure: Terraform (Provision) + Ansible (Configure) + Crossplane (Claim)
            ├─ Networking: Gateway API, NetworkPolicy, mTLS-ধরনের সুরক্ষা
            ├─ Data: PostgreSQL StatefulSet + Backup + Restore Runbook
            ├─ Security: Policy, RBAC, Image Scan, Secret Scan, SBOM
     ├─ Observability: Prometheus + Grafana + SLO Dashboard + Alerts + Runbooks
          ├─ Reliability: SLO, Error Budget, Incident Management, DR Plan
             └─ Cost: FinOps Review, Quota, Limit, Tagging, বাজেট সতর্কতা
```

৩.২ পেশাদার প্রয়োজনীয়তা (Requirements)

আইডি	প্রয়োজনীয়তা	গ্রহণযোগ্যতা
REQ-01	Multi-Cluster: dev ও staging — আলাদা ক্লাস্টার	একই GitOps Repo থেকে দুটি ক্লাস্টারে ডেপ্লয়
REQ-02	GitOps: Argo CD বা Flux CD দিয়ে Sync ও Rollback	Git Revert-এ Rollback প্রমাণিত
REQ-03	IDP: Backstage Catalog, Template, TechDocs	Template থেকে নতুন সার্ভিস Staging-এ যায়
REQ-04	IaC: Terraform + Ansible — দায়িত্ব আলাদা	Plan/Apply ও Idempotence প্রমাণিত
REQ-05	Security Governance: Policy, RBAC, Scan, SBOM	নীতি-ভাঙা ডেপ্লয় আটকায়; SBOM আছে
REQ-06	Observability: Metrics, Logs, SLO Dashboard, Alert	Golden Signal ও SLO Dashboard দেখা যায়
REQ-07	SRE: SLO, Error Budget, Incident Runbook	Error Budget শেষ হলে কী হবে — নীতি লেখা আছে

REQ-08	FinOps: Cost Review, Quota, Tagging	নেমস্পেস-ভিত্তিক Resource পর্যালোচনা লেখা আছে
REQ-09	DR: Backup, Restore পরীক্ষা, RTO/RPO	Restore পরীক্ষা করে সময় মাপা আছে
REQ-10	ডকুমেন্টেশন: Architecture, ADR, Runbook, Handover	নতুন প্রকৌশলী নথি পড়ে প্ল্যাটফর্ম চালাতে পারেন

৩.৩ আর্কিটেকচার

দুটি kind ক্লাস্টার (dev, staging); প্রতিটিতে Argo CD, Gateway, NetworkPolicy, Pod Security, PostgreSQL StatefulSet, Prometheus + Grafana; GitOps Repo একটাই; Backstage Portal লোকালে; Terraform (Docker Provider দিয়ে লোকাল অনুকরণ) ও Ansible Playbook; SLO Dashboard ও Alert; DR Runbook। প্রোডাকশন ক্লাস্টার এই প্রকল্পে লোকালে অনুকরণ করা হয় — আসল প্রোডাকশনের জন্য Capstone-এর Readiness Review প্রয়োজন।

৩.৪ বাস্তবায়ন পরিকল্পনা

মাইলফলক	কাজ	সময়
M1	Multi-Cluster + GitOps + Gateway + Policy (Project 01-এর সম্প্রসারণ)	সপ্তাহ ১–২
M2	IDP (Project 02) যুক্ত করা; Template থেকে Staging ডেপ্লয়	সপ্তাহ ২–৩
M3	Terraform/Ansible, Observability, SLO, Security Scan, SBOM	সপ্তাহ ৩–৪
M4	FinOps, DR Simulation, Incident Runbook, Readiness Review, Handover	সপ্তাহ ৪–৬

৩.৫ রিপোজিটরি কাঠামো

```
technova-enterprise-platform/
├── gitops/                          # GitOps Repo (Base + Overlays, App-of-Apps)
├── charts/                          # Helm Charts
├── terraform/                       # main.tf, variables.tf
├── ansible/                         # playbook.yaml
├── crossplane/                      # Claim খসড়া
├── backstage/                       # technova-portal, catalog, templates
├── k8s/                             # Gateway, NetworkPolicy, Security, Stateful
├── observability/                   # Alert নিয়ম, Dashboard JSON
├── docs/
│   ├── architecture.md
│   ├── adr/                         # ADR-001 থেকে ADR-00N
│   ├── slo.md
│   ├── dr-plan.md
│   ├── governance.md
│   ├── security-review.md
│   ├── cost-review.md
```

```
│   ├── production-readiness.md
│   ├── runbooks/
│   └── handover.md
└── README.md                          # প্রকল্পের সারাংশ ও কীভাবে চালাতে হয়
```

৩.৬ কনফিগারেশন

প্রতিটি স্তরের কনফিগারেশন সংশ্লিষ্ট Lab-এর ফাইল থেকে নেওয়া — এই প্রকল্পে নতুন করে লেখা নয়, বরং একসাথে জোড়া। কোন ফাইল কোথা থেকে এসেছে, তার উৎস Lab-এর নম্বরসহ `docs/architecture.md`-এ লিখুন।

৩.৭ ডেপ্লয়মেন্ট

ক্রম: ক্লাস্টার → নেমস্পেস/Quota/Policy → Argo CD → Gateway → ডেটাবেস → অ্যাপ → Observability → SLO → IDP। প্রতিটি ধাপের পর Health যাচাই করুন; কোনো ধাপ ব্যর্থ হলে পরের ধাপে যাবেন না।

৩.৮ যাচাই (Validation)

- End-to-End: Template থেকে নতুন সার্ভিস → CI → GitOps → Staging ডেপ্লয় → Gateway দিয়ে অনুরোধ → ডেটাবেসে ডেটা।
- Failure: একটি Pod মুছে, একটি ক্লাস্টারের একটি সার্ভিস বন্ধ করে — প্ল্যাটফর্ম টিকে থাকে কি না।
- Rollback: Git Revert-এ আগের সংস্করণ ফেরে কি না।
- DR: Backup থেকে Restore করে সময় মাপা।
- Security: Scan ও Policy পরীক্ষা।

৩.৯ নিরাপত্তা মূল্যায়ন

পুরো প্ল্যাটফর্মের Security Review (`docs/security-review.md`): বাইরে খোলা Port, ServiceAccount-এর অনুমতি, Secret-এর অবস্থান, Image Scan ফল, Policy-র তালিকা। প্রতিটি Critical সমস্যার সংশোধন-পরিকল্পনা লিখুন।

৩.১০ খরচ মূল্যায়ন

লোকালে খরচ শূন্য; আসল ক্লাউডে এই প্ল্যাটফর্মের খরচের অনুমান নথিতে লিখুন — কিন্তু নির্দিষ্ট মূল্য দাবি করবেন না (মূল্য বদলায়)। বরং লিখুন: কোন রিসোর্স খরচ বাড়ায়, কোনটি কমায়, বাজেট সতর্কতা কোথায় বসবে, এবং dev/staging-এ কীভাবে সাশ্রয় হবে।

৩.১১ অপারেশনাল ডকুমেন্টেশন

- Architecture ও ADR সেট।
- Runbook: Deploy, Rollback, Sync Failure, Database Restore, Incident।
- SLO ও Error Budget নীতি।
- DR Plan।
- FinOps Review।
- Handover নোট।

৩.১২ চূড়ান্ত ডেলিভারেবল

- চলমান Enterprise Platform (লোকাল অনুকরণ) — দুটি ক্লাস্টার, GitOps, IDP, Observability-সহ।
- End-to-End ডেমো: Template থেকে Staging ডেপ্লয়।
- Failure ও DR পরীক্ষার রিপোর্ট।
- Security, Cost, SLO ও Readiness নথি।
- README ও Handover।

FINAL SERIES CAPSTONE • Volumes 1–16 — TechNova Enterprise-Grade Developer Platform and Production Infrastructure

Final Series Capstone — TechNova — Enterprise-Grade Developer Platform and Production Infrastructure

এই Capstone পুরো ১৬ খণ্ডের সিরিজের সমন্বিত প্রকল্প। এখানে আপনি আলাদা আলাদা টুলের সংগ্রহ দেখাবেন না — দেখাবেন **End-to-End Engineering Ownership**: একটি বাস্তবসম্মত MERN অ্যাপ্লিকেশনকে ধারণা থেকে প্রোডাকশন পর্যন্ত নিয়ে যাওয়া, যেখানে প্রতিটি সিদ্ধান্ত লেখা আছে, প্রতিটি পরিবর্তন পরীক্ষিত, প্রতিটি ব্যর্থতার জন্য পরিকল্পনা আছে, এবং প্রতিটি খরচের হিসাব আছে। এই Capstone-এর ফল একটি পোর্টফোলিও-যোগ্য, পর্যালোচনাযোগ্য ইঞ্জিনিয়ারিং ডেলিভারি — একটি চাকরির সাক্ষাৎকারে দেখানোর মতো কাজ।

> **CONCEPT:** Capstone-এর প্রশ্নটি “আপনি কতগুলো টুল জানেন?” নয় — প্রশ্নটি “একটি সিস্টেম ভাঙলে আপনি কী করবেন, আর ভাঙার আগেই কী করেছিলেন?”। টুলগুলো সেই উত্তরের প্রমাণ; উত্তরটি নিজেই ইঞ্জিনিয়ারিং।

> **WARNING:** এই Capstone লোকাল অনুকরণে সম্পূর্ণ করা যায় — এবং সেটাই সুপারিশ করা হয়। আসল ক্লাউডে প্রোডাকশন ডেপ্লয় করলে খরচ হতে পারে; আসল ক্লাউডে যাওয়ার আগে Production Readiness Review সম্পূর্ণ করুন এবং বাজেট সতর্কতা বসান। আসল ব্যবহারকারীর ডেটা, আসল পেমেন্ট বা আসল Secret এই Capstone-এ ব্যবহার করবেন না।

১.১ বাস্তবসম্মত MERN অ্যাপ্লিকেশন — TechNova Commerce

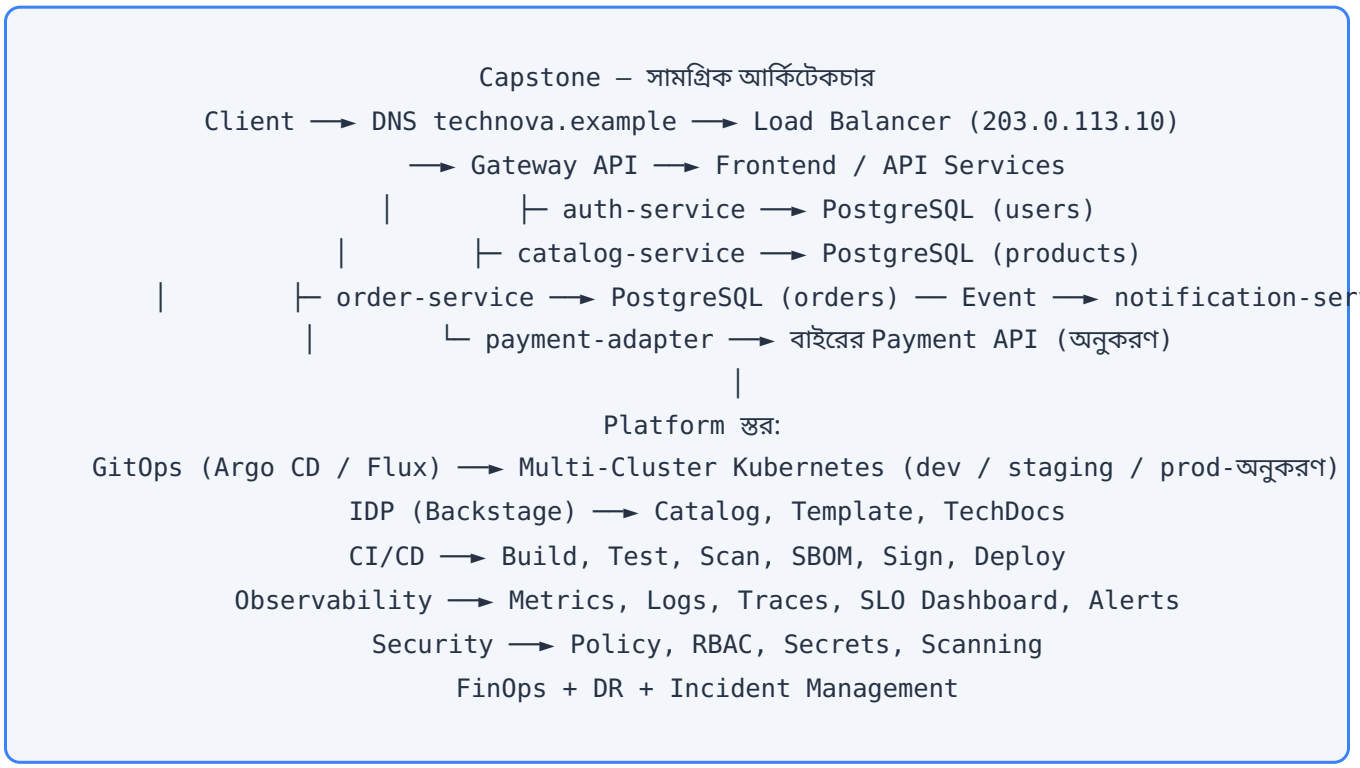
TechNova Commerce একটি কাল্পনিক ই-কমার্স অ্যাপ্লিকেশন: React Frontend, Express API, PostgreSQL (প্রধান ডেটা) ও MongoDB-ধরনের Document Store (ঐচ্ছিক; শেখার জন্য PostgreSQL-ই যথেষ্ট — দুটো ডেটাবেসের জটিলতা এড়াতে প্রথম সংস্করণে শুধু PostgreSQL ব্যবহার করুন)। ফিচার: ব্যবহারকারী নিবন্ধন ও লগইন, পণ্য তালিকা, অর্ডার তৈরি, পেমেন্ট অ্যাডাপ্টার (বাইরের Payment API-র অনুকরণ — আসল পেমেন্ট নয়), নোটিফিকেশন (Event-ভিত্তিক)। সার্ভিস: `frontend`, `auth-service`, `catalog-service`, `order-service`, `payment-adapter`, `notification-service`। প্রতিটি সার্ভিসের নিজের ডেটা মালিকানা; সার্ভিসগুলো শুধু API ও Event দিয়ে কথা বলে।

১.২ পেশাদার SRS (Software Requirements Specification)

SRS-এর কাঠামো (`docs/srs.md`): উদ্দেশ্য ও পরিধি; ব্যবহারকারী ও ভূমিকা (Customer, Admin, Platform Engineer, Developer); Functional Requirement (প্রতিটি ফিচারের জন্য: বিবরণ, ইনপুট, আউটপুট, Error); Non-Functional Requirement (Availability 99.9% লক্ষ্য, p95 Latency, Security, Scalability, Cost সীমা); অনুমান ও সীমাবদ্ধতা (যেমন: পেমেন্ট আসল নয়, অনুকরণ; ব্যবহারকারীর সংখ্যা অনুমান); গ্রহণযোগ্যতা মানদণ্ড। প্রতিটি Requirement-এর আইডি দিন (যেমন FR-01, NFR-01) — যাতে পরীক্ষা ও Traceability সম্ভব হয়।

১.৩ High-Level Design (HLD) ও Low-Level Design (LLD)

HLD (`docs/hld.md`): সিস্টেমের উপাদান, তাদের সম্পর্ক, ডেটা প্রবাহ, নেটওয়ার্ক কাঠামো, পরিবেশ (dev/staging/prod), নিরাপত্তা সীমানা, Observability কাঠামো। ডায়াগ্রামসহ। **LLD** (`docs/lld.md`): প্রতিটি সার্ভিসের API Contract, ডেটাবেস Schema, Event-এর গঠন, Error Handling, Retry ও Timeout নীতি, Kubernetes রিসোর্স (Deployment, Service, ConfigMap, Secret, PVC, HPA, NetworkPolicy), Helm Values ও Kustomize Overlay-র বিস্তারিত। HLD "কী", LLD "কীভাবে" — দুটো আলাদা নথি রাখুন।



১.৪ Architecture Decision Records (ADR)

অন্তত ছয়টি ADR লিখুন: ADR-001 Kubernetes প্ল্যাটফর্ম; ADR-002 ডেটাবেস স্থাপনা; ADR-003 GitOps টুল (Argo CD বনাম Flux — আপনার পছন্দ ও কারণ); ADR-004 Helm বনাম Kustomize; ADR-005 Observability স্ট্যাক; ADR-006 Disaster Recovery কৌশল। প্রতিটি ADR-এ Context, Decision, Alternatives, Consequences — এবং Status (Accepted / Superseded) থাকবে।

১.৫ নেটওয়ার্ক ও ক্লাউড আর্কিটেকচার

নেটওয়ার্ক নথি (`docs/network.md`): VPC-ধরনের ভাগ (লোকালে kind নেটওয়ার্কের অনুকরণ), Subnet-এর ধারণা, Gateway-এর অবস্থান, NetworkPolicy-র তালিকা, DNS (`technova.example`), TLS-এর পরিকল্পনা (প্রোডাকশনে), বাইরে খোলা Port-এর তালিকা (শুধু 80/443 — আর কিছু নয়)। ক্লাউড আর্কিটেকচার: Terraform-এর Variable কাঠামো দিয়ে পরিবেশভেদ; কোন রিসোর্স Managed সার্ভিসে যাবে, কোনটি ক্লাস্টারে — তার যুক্তি।

১.৬ Terraform অবকাঠামো

`terraform/`-এ: `main.tf`, `variables.tf`, `outputs.tf`, পরিবেশভেদে `.tfvars`। লোকালে Docker Provider দিয়ে অনুকরণ; আসল ক্লাউডে Apply করার আগে Plan পর্যালোচনা বাধ্যতামূলক। State Remote Backend-এর পরিকল্পনা লিখুন (লোকালে Local State; প্রোডাকশনে Remote, Encrypted, Locked)।

১.৭ Ansible অটোমেশন

`ansible/`-এ: Playbook — Base কনফিগারেশন, ব্যবহারকারী, নিরাপত্তা নিয়ম, অ্যাপ ডিরেক্টরি। Idempotence প্রমাণ করুন (দ্বিতীয়বারে `changed=0`)। Terraform ও Ansible-এর দায়িত্বের সীমা নথিতে লিখুন।

১.৮ Docker ইমেজ

প্রতিটি সার্ভিসের `Dockerfile`: Multi-Stage Build, non-root User, ছোট Base ইমেজ, `.dockerignore`। ইমেজে Secret নেই; Tag-এ Git Commit Hash ব্যবহার করুন — `latest` নয়।

```
# Dockerfile — ধারণা (প্রতিটি সার্ভিসে একই মান)
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
RUN npm run build
FROM node:20-alpine AS runtime
WORKDIR /app
USER 10001
COPY --from=build /app/dist ./dist
COPY --from=build /app/node_modules ./node_modules
EXPOSE 3000
CMD ["node", "dist/index.js"]
```

১.৯ Kubernetes ডেপ্লয়মেন্ট

`k8s/`-এ: প্রতিটি সার্ভিসের Deployment, Service, ConfigMap, Secret (প্লেসহোল্ডার), HPA, NetworkPolicy, PodDisruptionBudget; PostgreSQL StatefulSet ও PVC; Gateway ও HTTPRoute; ResourceQuota ও LimitRange। প্রতিটি Deployment-এ Resource Request/Limit, Readiness ও Liveness Probe, SecurityContext (Restricted) থাকবে।

১.১০ Helm ও Kustomize

Helm Chart প্রতিটি সার্ভিসের জন্য অথবা একটি Umbrella Chart; Kustomize Base + Overlay (dev/staging/prod)। কোন অ্যাপে কোনটি — তার সিদ্ধান্ত-নোট ADR-004-এ লিখুন। দুটো একই অ্যাপে মেশাবেন না।

১.১১ GitOps

GitOps Repo (Project 01-এর কাঠামো); Argo CD বা Flux CD দিয়ে Sync; dev/staging-এ Auto-Sync, prod-এ Manual Approval; App-of-Apps; Sync ও Self-Heal পরীক্ষার প্রমাণ।

১.১২ Internal Developer Portal

Backstage Portal: Catalog-এ সব সার্ভিস, Template দিয়ে নতুন সার্ভিস, TechDocs। Onboarding গাইডসহ। নতুন ডেভেলপারের প্রথম ডেপ্লয়ের সময় মাপুন।

১.১৩ CI/CD

Pipeline-এর ধাপ: Lint → Test → Build → Image Scan → SBOM → Sign → Registry → GitOps Repo-তে Image Tag আপডেট (Pull Request দিয়ে) → Argo CD Sync। প্রতিটি ধাপের ব্যর্থতায় Pipeline থামবে। Pipeline ফাইল (`.github/workflows/` বা সমতুল্য) ও তার নথি সংরক্ষণ করুন।

১.১৪ নিরাপত্তা স্ক্যানিং, SBOM ও Artifact Signing

Image Scan (Trivy), Secret Scan, Dependency Scan; SBOM (CycloneDX) প্রতিটি ইমেজের জন্য; Artifact Signing (Cosign-ধরনের) — স্বাক্ষর ছাড়া ইমেজ ডেপ্লয় হবে না (প্রোডাকশন নীতি হিসেবে লিখুন; লোকালে Signing-এর ধারণা ও কমান্ড দেখান)।

```
# CI-তে নিরাপত্তা ধাপ (ধারণা)
# 1. trivy image --severity CRITICAL,HIGH <image>
# 2. trivy image --format cyclonedx --output sbom.json <image>
# 3. cosign sign --key CHANGE-ME-COSIGN-KEY <image>
# 4. Critical সমস্যা পেলে Pipeline থামবে — Deploy হবে না
```

১.১৫ Secrets Management

Secret-এর নীতি (`docs/secrets.md`): Secret কখনো Git-এ নয়; লোকালে Kubernetes Secret; প্রোডাকশনে External Secrets / Sealed Secrets-এর পরিকল্পনা; Rotation-এর নিয়ম; কে কোন Secret দেখতে পারে — RBAC-সহ। সব Secret-এর নামের তালিকা ও তাদের উৎস (কোথা থেকে আসে) লিখুন — মান (Value) নয়।

১.১৬ Observability

Prometheus + Grafana; Logs সংগ্রহ; Trace-এর ধারণা; Golden Signal Dashboard; Alert নিয়ম — প্রতিটির Runbook লিংকসহ। একটি ভুল অনুরোধের Request ID দিয়ে Metrics → Logs → Trace-এর গল্প পড়ার অনুশীলন নথিতে লিখুন।

১.১৭ SLI/SLO Dashboard

`docs/slo.md`: প্রতিটি গুরুত্বপূর্ণ সার্ভিসের SLI, SLO, Error Budget; Grafana-তে Reliability Dashboard; Error Budget শেষ হলে কী হবে — নীতি।

১.১৮ Incident Management

Incident প্রক্রিয়া (`docs/incident.md`): Severity-র সংজ্ঞা (SEV-1 থেকে SEV-3), On-Call, যোগাযোগ-পথ, Status Page-এর ধারণা, Post-Incident Review (Blameless) ফরম্যাট। একটি অনুকরণীয় ঘটনা (যেমন: ডেটাবেস বন্ধ) দিয়ে পুরো প্রক্রিয়াটি অনুশীলন করুন ও তার Post-Incident Review লিখুন।

১.১৯ Backup ও Disaster Recovery

Backup নীতি (কী, কতবার, কোথায়), Restore পরীক্ষার সময়সূচি, RTO/RPO, DR Plan ও Runbook। লোকালে DR Simulation করে সময় মাপুন (Lab 22-এর মতো)।

১.২০ খরচ অপ্টিমাইজেশন (FinOps)

Cost Review (`docs/cost-review.md`): নেমস্পেস-ভিত্তিক Resource ব্যবহার, অব্যবহৃত Resource, অতিরিক্ত Replica, Observability Retention; সাশ্রয়ের পরিকল্পনা; বাজেট সতর্কতার পরিকল্পনা। নির্দিষ্ট মূল্য দাবি করবেন না — মূল্য বদলায়; বরং খরচের চালক (Driver) ও সাশ্রয়ের উপায় লিখুন।

১.২১ Production Readiness Review

Readiness চেকলিস্ট (`docs/production-readiness.md`) — Lab 24-এর চেকলিস্টের সম্প্রসারণ: ডেপ্লয়, নিরাপত্তা, পর্যবেক্ষণ, ডেটা, খরচ, নথি, Incident, DR — প্রতিটি আইটেমে প্রমাণ। কোনো আইটেমে প্রমাণ নেই মানে Readiness সম্পূর্ণ হয়নি।

১.২২ Runbook

`docs/runbooks/`-এ অন্তত ছয়টি Runbook: Deploy, Rollback, Sync Failure, Database Restore, High Error Rate, Pod CrashLoopBackOff। প্রতিটি Runbook-এ: লক্ষণ, যাচাই কমান্ড, ধাপ, কখন Escalate করবেন, কার সাথে যোগাযোগ করবেন।

১.২৩ চূড়ান্ত ইঞ্জিনিয়ারিং রিপোর্ট

`docs/final-engineering-report.md`: প্রকল্পের সারাংশ, আর্কিটেকচার, সিদ্ধান্ত (ADR-এর সারাংশ), পরীক্ষার ফল, নিরাপত্তা ও খরচ মূল্যায়ন, সীমাবদ্ধতা, ভবিষ্যৎ কাজ। এই রিপোর্টটি একজন পর্যালোচক পড়ে পুরো প্রকল্পটি বুঝতে পারবেন — কোড না খুলেই।

১.২৪ আর্কিটেকচার ডায়াগ্রাম, রিপোজিটরি কাঠামো ও মাইলফলক

```
technova-capstone/
├── docs/
│   ├── srs.md
│   ├── hld.md
│   ├── lld.md
│   ├── architecture.md
│   ├── adr/ADR-001.md ... ADR-006.md
```

```
│   ├── network.md
│   ├── secrets.md
│   ├── slo.md
│   ├── incident.md
│   ├── dr-plan.md
│   ├── cost-review.md
│   ├── security-review.md
│   ├── production-readiness.md
│   ├── final-engineering-report.md
│   └── runbooks/
├── apps/                                # MERN সার্ভিসগুলোর কোড
│   ├── frontend/
│   ├── auth-service/
│   ├── catalog-service/
│   ├── order-service/
│   ├── payment-adapter/
│   └── notification-service/
├── terraform/
├── ansible/
├── crossplane/
├── charts/
├── k8s/
├── gitops/
├── backstage/
├── observability/
├── .github/workflows/
└── README.md
```

মাইলফলক	কাজ	সময়
M1	SRS, HLD, LLD, ADR — নকশা সম্পূর্ণ	সপ্তাহ ১–২
M2	MERN অ্যাপ, Docker, Terraform, Ansible	সপ্তাহ ২–৩
M3	Kubernetes, Helm/Kustomize, GitOps, IDP, CI/CD	সপ্তাহ ৩–৫
M4	Security, SBOM, Signing, Observability, SLO	সপ্তাহ ৫–৬
M5	DR Simulation, Incident অনুশীলন, Readiness Review	সপ্তাহ ৬–৭
M6	Final Engineering Report, Handover, উপস্থাপনা-প্রস্তুতি	সপ্তাহ ৭–৮

১.২৫ পরীক্ষা কৌশল (Test Strategy)

পরীক্ষার স্তর	কী পরীক্ষা হয়	কোথায়
Unit Test	প্রতিটি সার্ভিসের ব্যবসায়িক যুক্তি	CI-তে, প্রতিটি Commit-এ
Integration Test	API Contract, ডেটাবেস সংযোগ	CI-তে, Staging-এ
Contract Test	সার্ভিস-টু-সার্ভিস চুক্তি ভাঙেনি তো	CI-তে
Load Test	Latency, Throughput, Error Rate (লোকালে)	Staging-অনুকরণে
Security Test	Image Scan, Secret Scan, Policy	CI-তে, প্রতিটি Build-এ
DR Test	Backup থেকে Restore	নিয়মিত অনুশীলনে
End-to-End Test	ব্যবহারকারীর পুরো পথ (নিবন্ধন → অর্ডার)	Staging-এ

১.২৬ নিরাপত্তা গ্রহণযোগ্যতা মানদণ্ড (Security Acceptance Criteria)

- কোনো Secret Git-এ নেই; Secret Scan পাস করে।
- কোনো Critical দুর্বলতাসহ ইমেজ প্রোডাকশনে যায় না।
- সব Pod Restricted Security Standard মানে।
- ডেটাবেস বাইরে থেকে পৌঁছানো যায় না; NetworkPolicy প্রয়োগ হয়।
- Argo CD ও Grafana-তে Authentication আছে; Admin পাসওয়ার্ড ডিফল্ট নয়।
- SBOM প্রতিটি ইমেজের জন্য আছে; স্বাক্ষর-নীতি লেখা আছে।

১.২৭ নির্ভরযোগ্যতা গ্রহণযোগ্যতা মানদণ্ড (Reliability Acceptance Criteria)

- একটি Pod মুছলে সার্ভিস চলে (Health Check প্রমাণিত)।
- Git Revert-এ Rollback হয় (সময়সহ প্রমাণিত)।
- Backup থেকে Restore পরীক্ষা করা হয়েছে।
- SLO Dashboard-এ Golden Signal দেখা যায়; Alert নিয়ম আছে।
- Canary পরিকল্পনা ও Rollback শর্ত লেখা আছে।
- Incident Runbook দিয়ে একটি অনুকরণীয় ঘটনা পরিচালনা করা হয়েছে।

১.২৮ খরচের সীমা (Cost Constraints)

Capstone লোকালে সম্পূর্ণ করা যায় — খরচ শূন্য। আসল ক্লাউডে যাওয়ার আগে: বাজেট সতর্কতা বসান; dev/staging-এ ছোট আকার ব্যবহার করুন; Observability Retention সীমিত রাখুন; অপ্রয়োজনীয় পরিবেশ বন্ধ করুন। কোনো নির্দিষ্ট মূল্য এই নথিতে দাবি করা হয়নি — ক্লাউডের মূল্য বদলায়, তাই মূল্য-তালিকার বদলে খরচের চালক ও সাশ্রয়ের উপায় নথিভুক্ত করুন।

১.২৯ ব্যর্থতার দৃশ্যকল্প (Failure Scenarios)

দৃশ্যকল্প	প্রত্যাশিত আচরণ	পরীক্ষা
একটি Pod ক্র্যাশ করে	Kubernetes নতুন Pod বানায়; সার্ভিস চলে	Pod মুছে পরীক্ষা
Catalog Service বন্ধ	Order তৈরি ব্যর্থ হয় — স্পষ্ট ভুল-জবাব; লগইন চলে	Service Scale ০ করে পরীক্ষা
ডেটাবেস বন্ধ	অ্যাপ ভুল-জবাব দেয়; Backup থেকে Restore হয়	StatefulSet Scale ০ করে Restore
ভুল ইমেজ ডেপ্লয়	Canary-তে ধরা পড়ে; Rollback হয়	ভাঙা Tag দিয়ে Canary
Git-এ ভুল Commit	Policy/Review-এ আটকায়; prod-এ Manual Sync আটকায়	নীতি-ভাঙা Manifest দিয়ে Sync
Argo CD বন্ধ	চলমান অ্যাপ চলে; নতুন ডেপ্লয় থামে	Argo CD Scale ০ করে পর্যবেক্ষণ

১.৩০ Rollback পরিকল্পনা

অ্যাপ Rollback: Git Revert → Argo CD Sync (prod-এ অনুমোদনসহ)। ডেটাবেস Rollback: Schema বদল Backward-Compatible রাখুন — পুরোনো কোড নতুন Schema-তে চলে; ডেটাবেসের Rollback শুধু Backup থেকে Restore — এবং তা শেষ উপায়। Canary Rollback: Argo Rollouts Abort। প্রতিটি Rollback-এর সময় ও সিদ্ধান্ত-বিন্দু Runbook-এ লিখুন।

১.৩১ চূড়ান্ত গ্রেডিং রুব্রিক

বিভাগ	ওজন	উৎকৃষ্ট কাজের লক্ষণ
Architecture & Design (SRS, HLD, LLD, ADR)	২০%	সিদ্ধান্তের যুক্তি স্পষ্ট; SPOF ও বিকল্প লেখা আছে
Implementation (MERN, Docker, K8s, Helm/ Kustomize)	২০%	চলমান প্ল্যাটফর্ম; End-to-End অনুরোধ সফল
GitOps & IDP (Argo CD/Flux, Backstage)	১৫%	Git থেকে ডেপ্লয়; Template থেকে নতুন সার্ভিস
CI/CD & Security (Scan, SBOM, Signing, Policy)	১৫%	Pipeline-এ নিরাপত্তা পরীক্ষা; SBOM আছে
Observability & Reliability (SLO, DR, Incident)	১৫%	Dashboard, Alert, DR পরীক্ষা, Incident অনুশীলন
Documentation & Ownership (Report, Runbook, Handover)	১৫%	নতুন প্রকৌশলী নথি পড়ে চালাতে পারেন; Final Report সম্পূর্ণ

পাসের শর্ত: প্রতিটি বিভাগে অন্তত মৌলিক স্তর (চলমান প্রমাণসহ) এবং Security ও Reliability Acceptance Criteria-র প্রতিটি আইটেমে প্রমাণ। কোনো বিভাগে “শুধু লেখা, কোনো প্রমাণ নেই” থাকলে সেই বিভাগ অসম্পূর্ণ ধরা হবে।

১.৩২ সমাধান নির্দেশিকা (Solution Guide) — পর্যালোচকের জন্য

এই অংশটি শিক্ষার্থীর জন্য নয় — পর্যালোচক ও স্ব-মূল্যায়নের জন্য। একটি ভালো সমাধান কেমন দেখায়, তার সংক্ষিপ্ত নির্দেশিকা:

- **SRS/HLD/LLD:** প্রতিটি Requirement-এর আইডি আছে; HLD-তে ডায়াগ্রাম আছে; LLD-তে API Contract ও Schema আছে — শুধু বর্ণনা নয়।
- **GitOps:** dev-এ Auto-Sync, prod-এ Manual; Git Revert-এর Rollback প্রমাণ (সময়সহ) আছে; Self-Heal পরীক্ষা আছে।
- **IDP:** Catalog-এ Owner ছাড়া সার্ভিস নেই; Template থেকে তৈরি সার্ভিস Staging-এ ডেপ্লয় হয়েছে; Onboarding গাইডে সময় মাপা আছে।
- **Security:** Trivy-র আসল ফলাফল সংযুক্ত; SBOM ফাইল আছে; Secret Scan পাস করেছে; Policy-ভাঙা Manifest প্রত্যাখ্যাত হওয়ার প্রমাণ আছে।
- **Observability:** Grafana Dashboard-এর JSON আছে; SLO নথিতে Error Budget-এর হিসাব আছে; Alert-এর Runbook লিংক আছে।
- **DR:** Backup থেকে Restore-এর মাপা সময় আছে; RTO/RPO-র সাথে তুলনা আছে।
- **সাধারণ ভুল (পর্যালোচক যা দেখবেন):** `latest` Tag; Secret Git-এ; লোকাল ফলকে প্রোডাকশন ক্ষমতা বলে দাবি; Rollback শুধু লেখা, পরীক্ষা করা হয়নি; Dashboard-এ ডেটা নেই; ADR-এ “কেন নয়” অংশ নেই। এই ভুলগুলোর যেকোনো একটি থাকলে সংশ্লিষ্ট বিভাগে পূর্ণ নম্বর দেওয়া যাবে না।
- **উৎকৃষ্ট সমাধানের চিহ্ন:** প্রতিটি দাবির পাশে প্রমাণ (লগ, স্ক্রিনশট-বর্ণনা, সময়); ব্যর্থতার পরীক্ষা করা হয়েছে; সীমাবদ্ধতা সততার সাথে লেখা আছে; Handover নোট পড়ে একজন অপরিচিত প্রকৌশলী প্ল্যাটফর্মটি চালাতে পারেন।

> **REAL WORLD:** একটি ভালো Capstone-এর শেষ লাইনটি “আমি সবকিছু জানি” নয় — “আমি এই সিস্টেমটির মালিক হতে পারি: ভাঙলে সারাতে পারি, বদলাতে পারি, আর পরের জনকে বুঝিয়ে দিতে পারি”। এই মালিকানাই পুরো সিরিজের লক্ষ্য ছিল।

১.৩৩ Capstone ডেলিভারেবল চেকলিস্ট

ডেলিভারেবল	অবস্থা
SRS, HLD, LLD, ADR সেট	☐ সম্পূর্ণ
MERN অ্যাপ (চলমান, লোকাল)	☐ সম্পূর্ণ
Terraform, Ansible, Docker, Kubernetes, Helm/Kustomize	☐ সম্পূর্ণ
GitOps (Argo CD/Flux) — Sync ও Rollback প্রমাণসহ	☐ সম্পূর্ণ
Backstage IDP — Catalog, Template, TechDocs, Onboarding	☐ সম্পূর্ণ
CI/CD — Scan, SBOM, Signing	☐ সম্পূর্ণ
Secrets Management নীতি	☐ সম্পূর্ণ
Observability, SLO Dashboard, Alert + Runbook	☐ সম্পূর্ণ

Incident Management ও Post-Incident Review	☐ সম্পূর্ণ
Backup, DR Plan, DR Simulation রিপোর্ট	☐ সম্পূর্ণ
Cost Review	☐ সম্পূর্ণ
Production Readiness Review	☐ সম্পূর্ণ
Runbook সেট (অন্তত ৬টি)	☐ সম্পূর্ণ
Final Engineering Report ও Handover	☐ সম্পূর্ণ

এই Workbook এখানেই শেষ — কিন্তু TechNova-র গল্প এখানে শেষ নয়। এই Capstone-এর প্ল্যাটফর্মটি আপনার পোর্টফোলিও, আপনার পরবর্তী চাকরির সাক্ষাৎকারের আলোচ্য বিষয়, এবং আপনার ভবিষ্যৎ দলের প্রথম দিনের ভিত্তি। শুভকামনা — আর মনে রাখবেন: ভালো প্ল্যাটফর্মের মাপকাঠি টুলের সংখ্যা নয়, ডেভেলপারের প্রথম ডেপ্লয় কতটা সহজ, আর প্রোডাকশন ভাঙলে কতটা দ্রুত সারে।